

人工智能的软件基础

徐辉

复旦大学 计算与智能创新学院

2026 年 6 月 5 日

目录

I	AI 软件的编程语言: Python	1
1	使用 Python 编写神经网络	2
1.1	问题定义与分析	2
1.2	模型构建	3
1.3	模型训练	7
2	Python 程序的解析和运行	14
2.1	Python 语言	14
2.2	语法解析	15
2.3	编译	19
2.4	程序执行	21
3	Python 与 Native Code 交互	25
3.1	解释执行的性能问题	25
3.2	本地代码	30
3.3	Python 与本地代码交互	31
4	Python 多线程	37
4.1	并发和并行	37
4.2	Python 中的并发机制	40
4.3	在本地代码中并行	44
II	AI 软件框架: PyTorch	49
5	PyTorch	50
5.1	PyTorch 软件栈	50
5.2	使用 PyTorch 编写 CNN	51
5.3	使用 PyTorch 编写 RNN	57
6	GPU 编程	62
6.1	了解 GPU	62
6.2	CUDA C++ 编程	63
6.3	实验与应用	70
7	算子开发	74
7.1	算子开发	74
7.2	Triton 算子开发	74
7.3	Triton 的 Autotune 机制	79
7.4	在 PyTorch 中使用自定义算子	80

8 算子融合	85
8.1 问题分析	85
8.2 计算图优化	87
8.3 性能分析工具	95
 III 大模型软件及其应用	 97
9 大语言模型	98
9.1 词元	98
9.2 注意力机制	100
9.3 Decoder-only Transformer	103
9.4 模型训练与测试	106
 10 大模型训练	 110
10.1 大模型的训练挑战	110
10.2 参数高效微调	111
10.3 并行和分布式训练	113
10.4 使用 Hugging Face 训练大模型	117
 11 大模型推理	 122
11.1 参数量化	122
11.2 KV Cache	125
11.3 FlashAttention	126
11.4 大模型部署	128
 12 智能体	 133
12.1 智能体的软件结构	133
12.2 案例：CLI 智能体	134

Part I

AI 软件的编程语言：Python

1 使用 Python 编写神经网络

本章学习目标：

- *** 理解神经网络的基本工作原理，包括网络结构、前向传播与反向传播机制。
- *** 能够阅读并理解使用 Python 编写的基础神经网络模型代码。
- *** 能够利用神经网络对简单数学函数进行建模与拟合。

1.1 问题定义与分析

本节以阶乘问题为例讲解如何用 Python 构建神经网络预测阶乘值。

问题 给定任意一个 x ，其阶乘 $x!$ 的计算方式为

$$x! = \prod_{i=1}^x i.$$

如何通过神经网络模型预测 x 的阶乘？

分析 从数学角度看，函数可以形式化为集合之间的映射。阶乘函数定义为从自然数集到自然数集的映射：

$$f : \mathbb{N} \rightarrow \mathbb{N}, \quad f(x) = x!.$$

在实际建模过程中，直接对 $x!$ 进行预测会面临数值大小迅速增长的问题。为缓解这一困难，通常对目标函数进行对数变换，将问题转化为对 $\log(x!)$ 的预测：

$$\log(x!) = \log \left(\prod_{i=1}^x i \right) = \sum_{i=1}^x \log i.$$

该变换将原本的乘法运算转化为加法形式，从而显著缓解输出数值的增长速率，使其更适合神经网络进行建模。我们可以使用神经网络构建 $x \rightarrow \log f(x)$ 的映射。

一般而言，利用神经网络解决这类问题通常包括以下几个步骤：

- 1) **模型构建**：设计神经网络的输入形式、网络结构及输出表示；
- 2) **模型训练**：基于实际数据调整模型参数，最小化模型输出与真实值之间的差异；
- 3) **效果验证**：评估模型的预测准确度和泛化能力。

1.2 模型构建

1.2.1 网络结构

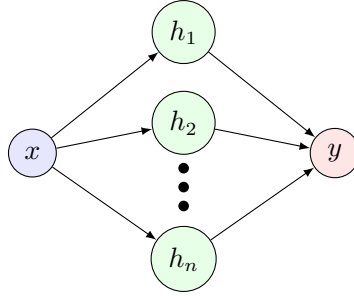


图 1.1: 前馈神经网络示例

我们采用前馈神经网络 (feed-forward neural network) 对该问题进行建模。前馈神经网络结构比较简单，其信息自输入层向输出层单向传播。图 1.1 是一个前馈神经网络，该模型由一个输入节点、一个隐藏层以及一个输出节点组成，其中隐藏层包含 $h_1, h_2, \dots, h_n$ 个神经元。每个隐藏层神经元执行如下非线性变换：

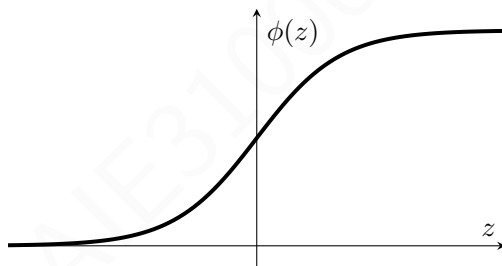
$$h_i = \sigma(w1_i \cdot x + b1_i),$$

其中 $\sigma(\cdot)$ 表示激活函数。输出层采用线性形式，对所有隐藏层神经元的输出进行加权求和作为 $\log(x!)$ 的预测结果。

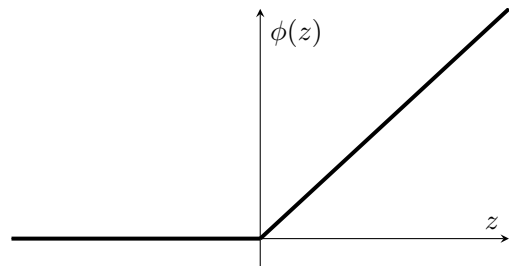
$$\hat{y} = \sum_{i=1}^n w2_i \cdot h_i + b2$$

1.2.2 激活函数

激活函数 $\sigma(\cdot)$ 通过对输入的线性组合进行非线性变换，从而为模型引入非线性能力，使得神经网络能够逼近复杂的非线性关系。若不引入激活函数，即使堆叠多个神经元，整个模型仍然等价于一个线性模型。常见的激活函数包括 Sigmoid 函数和 ReLU (Rectified Linear Unit)。



(a) Sigmoid 激活函数



(b) ReLU 激活函数

图 1.2: 常见激活函数示意图

Sigmoid 函数 Sigmoid 函数的定义为：

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

其输出值严格位于 $(0, 1)$ 区间内，因此常被解释为概率值。

ReLU ReLU 在输入为负时输出零，在输入为正时呈线性增长，其定义为：

$$\sigma(z) = \max(0, z)$$

1.2.3 Python 实现

在本节中，我们使用纯 Python（不依赖任何深度学习框架）实现上述模型。我们将模型设计为一个 class，并将其名称定义为 FactorialNN。接下来，我们先设计模型的主要规格和初始化方法，然后再定义其前向传播过程。

模型初始化

代码 1.1 定义了模型类 FactorialNN。其中，__init__ 函数是 Python 语言中的类构造器，用于在对象创建时对模型结构进行初始化。该函数接收 hidden_size 参数，用于定义隐藏层的神经元个数。例如，可以通过 FactorialNN(20) 创建一个模型实例，该代码会自动调用 __init__ 函数，并将隐藏层神经元个数设置为 20。也可以在创建实例时省略该参数，即直接调用 FactorialNN()，则模型将采用默认设置，此时隐藏层神经元个数为 10。

```
1 class FactorialNN:
2     def __init__(self, hidden_size=10):
3         """
4         __init__ 是 Python 类的构造函数 (constructor) ,
5         在创建对象时会被自动调用，用于初始化对象的成员变量。
6
7         参数:
8             hidden_size (int): 隐藏层神经元的个数。
9             若未显式指定，则默认取值为 10。
10        """
11
12        # 将传入的隐藏层规模保存为对象属性
13        self.hidden_size = hidden_size
14
15        # -----
16        # 输入层 -> 隐藏层 的权重和偏置初始化
17        # -----
18
19        # w1[i] 表示从输入  $x$  到第  $i$  个隐藏层神经元的权重
20        # 使用列表推导式生成 hidden_size 个随机初始权重
21        # 等价于如下 for 循环（仅为说明，非实际执行代码）：
22        #
23        #     w1 = []
24        #     for i in range(hidden_size):
25        #         w1.append(random.uniform(-1, 1))
26        #
27        # 列表推导式在语义上完全等价，但写法更加简洁、清晰
28        # random.uniform(-1, 1) 表示在  $[-1, 1]$  区间内均匀采样
29        self.w1 = [random.uniform(-1, 1) for _ in range(hidden_size)]
30
31        # b1[i] 表示第  $i$  个隐藏层神经元对应的偏置项
32        # 偏置同样采用随机初始化
33        self.b1 = [random.uniform(-1, 1) for _ in range(hidden_size)]
34
```

```

35 # -----
36 # 隐藏层 -> 输出层 的权重和偏置初始化
37 # -----
38
39 # w2[i] 表示第 i 个隐藏层神经元到输出层神经元的连接权重
40 self.w2 = [random.uniform(-1, 1) for _ in range(hidden_size)]
41
42 # b2 为输出层神经元的偏置项 (标量)
43 self.b2 = random.uniform(-1, 1)

```

代码 1.1: 定义神经网络模型结构

在 `__init__` 函数中, 我们定义并随机初始化了模型各层之间的权重和偏置。其中, 列表 `w1` 和 `b1` 分别表示输入层到隐含层的权重和偏置, `w2` 和 `b2` 则表示隐含层到输出层的权重和偏置。`uniform` 函数用于从指定区间内的均匀分布中随机采样初始参数值, 在本例中权重和偏置均初始化为区间 $[-1, 1]$ 内的随机数, 以避免所有神经元在初始阶段具有完全相同的行为。另外, 若将权重初始化为区间过大, 容易导致前向传播时激活值过大, 使非线性函数进入饱和区, 从而在反向传播阶段产生梯度消失等问题。

前向传播

下面, 我们为 `FactorialNN` 定义前向传播的计算过程, 即代码 1.2 中的 `forward` 函数。该函数描述了数据从输入层 $\rightarrow$ 隐含层 $\rightarrow$ 输出层的逐层计算过程。

```

1 class FactorialNN:
2     ...
3     def forward(self, x):
4         # 参数:
5         x: 模型输入, 对应一个样本
6
7         # ----- 隐藏层 -----
8         # z 用于存储隐藏层应用激活函数之前每个神经元的线性输出
9         # 用于反向传播计算梯度
10        z = []
11        # h 用于存储隐藏层每个神经元的输出
12        h = []
13
14        # 依次计算每个隐藏神经元的输出
15        for i in range(self.hidden_size):
16            # 线性变换
17            # z_i = w1[i] * x + b1[i]
18            # 其中:
19            # w1[i]: 输入到第 i 个隐藏神经元的权重
20            # b1[i]: 第 i 个隐藏神经元的偏置
21            zi = self.w1[i] * x + self.b1[i]
22            z.append(zi)
23            # 引入非线性激活函数, 使网络能够表示非线性函数
24            # 若使用 sigmoid 激活函数, 可替换为: h.append(sigmoid(z))
25            h.append(relu(zi))
26
27
28        # ----- 输出层 -----
29        # 输出层对隐藏层的输出进行线性组合

```

```

30     #  $y = \sum(w2[i] * h[i]) + b2$ 
31     # 其中:
32     #    $w2[i]$ : 第  $i$  个隐藏神经元到输出层的权重
33     #    $b2$ : 输出层偏置
34     y = sum(self.w2[i] * h[i] for i in range(self.hidden_size)) + self.b2
35
36     # 返回:
37     #   z: 隐藏层应用激活函数之前每个神经元的线性输出 (用于ReLU反向传播)
38     #   h: 隐藏层所有神经元的输出 (用于Sigmoid反向传播)
39     #   y: 模型的最终预测结果
40     return z, h, y

```

代码 1.2: 前向传播函数

1.3 模型训练

模型训练的的目的是得到一组参数值 θ ，使神经网络能够尽可能准确地预测输入 x 对应的输出 y 。训练过程通常通过最小化预测值 $\hat{y}$ 与真实值 y 之间的误差来实现，并利用梯度下降（Gradient Descent）对模型参数进行迭代优化。在此过程中，反向传播（Backpropagation）算法被用来计算损失函数对各参数的梯度，从而指导参数更新。

1.3.1 反向传播

反向传播是神经网络训练的核心机制，其本质是利用链式求导法则，将输出误差逐层向前传播，从而计算各层参数对损失函数的梯度。反向传播算法的核心思想是：

- 1) 误差计算：根据预测值 $\hat{y}$ 与真实值 y ，计算损失 $\mathcal{L}(\hat{y}, y)$ 。
- 2) 梯度计算：利用链式法则逐层计算损失函数对各参数 θ_i 的偏导数，从而得到梯度向量 $\nabla_{\theta} \mathcal{L} = \left(\frac{\partial \mathcal{L}}{\partial \theta_1}, \frac{\partial \mathcal{L}}{\partial \theta_2}, \dots \right)$ 。
- 3) 参数更新：根据梯度和学习率更新参数。

误差计算

在回归任务中，均方误差（Mean Squared Error, MSE）是一种常用的误差度量方式，其定义为：

$$\mathcal{L}_{\text{MSE}} = \frac{1}{N} \sum_{i=1}^N (\hat{y}_i - y_i)^2,$$

其中 y_i 表示第 i 个样本的真实值， $\hat{y}_i$ 表示模型对该样本的预测值。

通过对误差进行平方，MSE 不仅消除了误差的正负号影响，还对较大的预测偏差给予更高的惩罚。此外，平方运算保证了损失函数在整个定义域内可导，使其能够与梯度下降和反向传播算法自然结合，从而实现高效的参数优化。

为了说明梯度的计算，我们可以先考虑单个样本的平方误差（Squared Error, SE）：

$$\mathcal{L}_{\text{SE}_i} = (\hat{y}_i - y_i)^2.$$

对该单样本误差求偏导，得到：

$$\frac{\partial \mathcal{L}_{\text{SE}_i}}{\partial \hat{y}_i} = 2(\hat{y}_i - y_i),$$

在省略常数系数 2 时，可以简化为：

$$\frac{\partial \mathcal{L}_{\text{SE}_i}}{\partial \hat{y}_i} = \hat{y}_i - y_i.$$

此外，平均绝对误差（Mean Absolute Error, MAE）也是一种常用的误差统计方法，其定义为：

$$\mathcal{L}_{\text{MAE}} = \frac{1}{N} \sum_{i=1}^N |\hat{y}_i - y_i|.$$

MAE 通过对预测误差取绝对值来消除正负号的影响。若只考虑单个样本的绝对误差（Absolute Error, AE），其对预测值 $\hat{y}_i$ 的偏导为：

$$\frac{\partial \mathcal{L}_{\text{AE}_i}}{\partial \hat{y}_i} = \begin{cases} -1, & y < \hat{y}; \\ \text{undefined}, & \hat{y} = y; \\ 1, & y > \hat{y}. \end{cases}$$

梯度计算

在神经网络训练过程中，我们的目标是计算损失函数对模型参数的偏导数，从而利用梯度下降法对参数进行更新。以本章定义的阶乘预测模型为例，训练过程中需要计算的梯度包括

$$\frac{\partial \mathcal{L}}{\partial w1_i}, \quad \frac{\partial \mathcal{L}}{\partial b1_i}, \quad \frac{\partial \mathcal{L}}{\partial w2_i}, \quad \frac{\partial \mathcal{L}}{\partial b2}$$

需要注意的是，损失函数 $\mathcal{L}$ 并不是模型参数的显式函数，而是通过一系列中间变量（如隐藏层的输出以及模型的预测值 $\hat{y}$ ）间接依赖这些参数。因此，参数梯度的计算需要借助链式求导法则。

以输出层权重 $w2_i$ 为例，其对损失函数的影响路径可以表示为：

$$w2_i \longrightarrow \hat{y} \longrightarrow \mathcal{L}.$$

其中输出层的计算形式为：

$$\hat{y} = \sum_{i=1}^H w2_i \cdot h_i + b2.$$

根据链式法则，有：

$$\frac{\partial \mathcal{L}}{\partial w2_i} = \frac{\partial \mathcal{L}}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial w2_i}.$$

假设采用均方误差作为损失函数，可得：

$$\frac{\partial \mathcal{L}}{\partial \hat{y}} = \hat{y} - y, \quad \frac{\partial \hat{y}}{\partial w2_i} = h_i.$$

因此，输出层权重的梯度为：

$$\frac{\partial \mathcal{L}}{\partial w2_i} = (\hat{y} - y) \cdot h_i.$$

同理，可以计算：

$$\frac{\partial \mathcal{L}}{\partial b2} = \frac{\partial \mathcal{L}}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial b2} = \hat{y} - y.$$

对于输入层权重 $w1_i$ ，其对损失函数的影响需要经过多层计算，其依赖路径为：

$$w1_i \longrightarrow z_i \longrightarrow h_i \longrightarrow \hat{y} \longrightarrow \mathcal{L}.$$

其中

$$z_i = w1_i \cdot x + b1_i, \quad h_i = \sigma(z_i)$$

根据链式求导法则，损失函数对 $w1_i$ 的偏导数可写为：

$$\frac{\partial \mathcal{L}}{\partial w1_i} = \frac{\partial \mathcal{L}}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial h_i} \cdot \frac{\partial h_i}{\partial z_i} \cdot \frac{\partial z_i}{\partial w1_i}.$$

其中各项局部导数分别为：

$$\frac{\partial \mathcal{L}}{\partial \hat{y}} = \hat{y} - y, \quad \frac{\partial \hat{y}}{\partial h_i} = w2_i, \quad \frac{\partial h_i}{\partial z_i} = \sigma'(z_i), \quad \frac{\partial z_i}{\partial w1_i} = x.$$

其中， $\sigma'(z_i)$ 为激活函数的导数。将上述结果代入，可得：

$$\frac{\partial \mathcal{L}}{\partial w1_i} = (\hat{y} - y) \cdot w2_i \cdot \sigma'(z_i) \cdot x.$$

同理，可以计算

$$\frac{\partial \mathcal{L}}{\partial b1_i} = \frac{\partial \mathcal{L}}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial h_i} \cdot \frac{\partial h_i}{\partial z_i} \cdot \frac{\partial z_i}{\partial b1_i} = (\hat{y} - y) \cdot w2_i \cdot \sigma'(z_i).$$

若激活函数采用 ReLU，其定义为 $\text{ReLU}(z) = \max(0, z)$ 。因此，其关于 z 的导数为分段函数：

$$\frac{d}{dz}\text{ReLU}(z) = \begin{cases} 1, & z > 0; \\ 0, & z < 0. \end{cases}$$

注意，在 $z = 0$ 处，ReLU 函数不可导。实际的神经网络实现中，通常约定：

$$\frac{d}{dz}\text{ReLU}(0) = 0.$$

因此，在使用 ReLU 激活函数时，反向传播中的梯度传播表现为一种“门控”机制。当神经元被激活 ($z > 0$) 时，梯度可以无衰减地通过；当神经元未被激活 ($z \leq 0$) 时，梯度被直接截断。

如果激活函数采用 Sigmoid，其计算方式为 $\sigma(z) = \frac{1}{1+e^{-z}}$ 。对该函数关于 z 求导，有

$$\frac{d}{dz}\sigma(z) = \frac{d}{dz}(1+e^{-z})^{-1} = -(1+e^{-z})^{-2} \cdot (-e^{-z}) = \frac{e^{-z}}{(1+e^{-z})^2}.$$

由于

$$\sigma(z) = \frac{1}{1+e^{-z}}, \quad 1 - \sigma(z) = \frac{e^{-z}}{1+e^{-z}},$$

因此

$$\frac{e^{-z}}{(1+e^{-z})^2} = \sigma(z)(1 - \sigma(z)).$$

最终得到 Sigmoid 函数导数的简洁形式：

$$\frac{d}{dz}\sigma(z) = \sigma(z)(1 - \sigma(z)).$$

在神经网络的反向传播过程中，通常已知隐含层神经元的输出 $h = \sigma(z)$ ，因此其导数常直接写为

$$\frac{dh}{dz} = h(1 - h).$$

1.3.2 参数更新

在完成梯度计算之后，神经网络的参数通过梯度下降法进行更新，使得损失函数 $\mathcal{L}$ 在训练过程中不断减小，从而实现模型的优化。设学习率为 η ，则权重和偏置的更新公式为：

$$w2_i \leftarrow w2_i - \eta \frac{\partial \mathcal{L}}{\partial w2_i},$$

$$b2 \leftarrow b2 - \eta \frac{\partial \mathcal{L}}{\partial b2},$$

$$w1_i \leftarrow w1_i - \eta \frac{\partial \mathcal{L}}{\partial w1_i},$$

$$b1_i \leftarrow b1_i - \eta \frac{\partial \mathcal{L}}{\partial b1_i}.$$

1.3.3 Python 实现

在本节，我们用 Python 实现模型训练过程，包括反向传播学习算法、数据准备、模型训练和效果测试几部分。

代码 1.3和 1.4分别给出了 ReLU 和 Sigmoid 两种常用激活函数及其对应的导数实现。这里使用 0.0、1.0 是为了确保输出为浮点数，从而在反向传播计算梯度时保持数值类型一致。

```
1 def relu(x):
2     # 前向传播: 返回 ReLU 激活值
3     return max(0.0, x)
4
5 def relu_derivative(z):
6     # 反向传播: ReLU 的导数
7     return 1.0 if z > 0 else 0.0
```

代码 1.3: ReLU 激活函数及其导数

```
1 def sigmoid(x):
2     # 前向传播: 返回 Sigmoid 激活值
3     return 1.0 / (1.0 + math.exp(-x))
4
5 def sigmoid_derivative(h):
6     # 反向传播: Sigmoid 的导数
7     return h * (1 - h)
```

代码 1.4: Sigmoid 激活函数及其导数

代码 1.5 给出了 FactorialNN 的反向传播与参数更新过程。

```
1 class FactorialNN:
2     ...
3     def train(self, x, target, lr=0.01):
4         """
5         对单个样本执行前向传播、反向传播并更新网络参数。
6
7         参数:
8         x      : 输入单样本。
9         target : 样本的真实标签值。
10        lr     : float, 默认 0.01
11                学习率, 用于梯度下降更新权重和偏置。
12        """
13
14        # ----- 前向传播 -----
15        # z: 隐藏层应用激活函数前的输出列表
16        # h: 隐藏层输出列表
17        # y: 网络预测输出
18        z, h, y = self.forward(x)
19
20        # ----- 输出层梯度 -----
21        # 使用均方误差 (MSE)
22        dy = y - target
23        mse = dy ** 2
24
25        # ----- 隐藏层梯度 -----
```

```

26     dh = []
27     for i in range(self.hidden_size):          # 链式法则计算隐藏层梯度
28         #  $dL/dh_i = dL/dy * dy/dh_i * dh_i/dz_i$ 
29         #  $dL/dy = dy$ 
30         #  $dy/dh_i = self.w2[i]$ 
31         #  $dh_i/dz_i$  = 激活函数的导数
32         # 假设使用 ReLu 作为激活函数
33         grad = dy * self.w2[i] * relu_derivative(z[i])
34         # 如果使用 Sigmoid 作为激活函数:
35         #  $grad = dy * self.w2[i] * sigmoid\_derivative(h[i])$ 
36         dh.append(grad)
37
38
39     # ----- 更新隐藏层 -> 输出层参数 -----
40     for i in range(self.hidden_size):
41         self.w2[i] -= lr * dy * h[i]
42     self.b2 -= lr * dy
43
44     # ----- 更新输入层 -> 隐藏层参数 -----
45     for i in range(self.hidden_size):
46         self.w1[i] -= lr * dh[i] * x
47         self.b1[i] -= lr * dh[i]
48
49     # 返回损失值, 方便观察训练过程
50     return mse

```

代码 1.5: 反向传播及参数更新

代码 1.6 展示了训练数据准备过程。

```

1  MAX_N = 10 # 定义最大阶乘值 n
2
3  def log_factorial(n):
4      """
5      计算 log(n!)
6      使用对数避免阶乘值快速增长导致溢出
7      """
8      return math.log(math.factorial(n))
9
10 # 训练数据列表
11 # 输入: 归一化的 n (除以 MAX_N)
12 # 输出: log(n!), 作为回归目标
13 training_data = [
14     (n / MAX_N, log_factorial(n))
15     for n in range(1, MAX_N + 1)
16 ]

```

代码 1.6: 构造阶乘训练数据

神经网络初始化及训练过程如代码 1.7 所示, 每轮迭代遍历全部训练样本并更新参数, 同时可输出总损失观察训练收敛情况。

```

1  nn = FactorialNN(hidden_size=10) # 隐藏层 10 个神经元
2

```

```

3 for epoch in range(20000):          # 迭代次数
4     total_loss = 0.0                # 累计本轮总损失
5     for x, y in training_data:      # 遍历每个训练样本
6         total_loss += nn.train(x, y) # 前向传播 + 反向传播 + 参数更新
7
8     if epoch % 2000 == 0:           # 每2000轮打印一次
9         print(f"Epoch {epoch}, Loss = {total_loss:.4f}")

```

代码 1.7: 训练神经网络

训练完成后, 可通过前向传播得到预测的 $\log(n!)$, 再使用指数函数还原为阶乘值。代码 1.8 展示了测试过程及预测结果与真实值对比。

```

1 print("\nPrediction Results:")
2
3 for n in range(1, MAX_N + 1):
4     x = n / MAX_N                # 输入归一化
5     _, y_pred = nn.forward(x)    # 前向传播得到  $\log(n!)$  预测值
6     fact_pred = math.exp(y_pred) # 将预测值还原为  $n!$ 
7
8     # 打印预测结果与真实阶乘值对比
9     print(f"{n}!    {fact_pred:.1f} (true: {math.factorial(n)})")

```

代码 1.8: 测试神经网络预测效果

下面给出了使用 ReLU 和 Sigmoid 作为隐藏层激活函数时, 阶乘回归任务的训练过程与预测结果。可以看到, 不同激活函数在损失收敛速度以及拟合能力上存在显著差异。

```

1 #: python factorial-relu.py
2 Epoch 0, Loss = 661.5144
3 Epoch 2000, Loss = 5.0790
4 Epoch 4000, Loss = 4.8953
5 Epoch 6000, Loss = 4.8947
6 Epoch 8000, Loss = 4.8939
7 Epoch 10000, Loss = 4.8934
8 Epoch 12000, Loss = 4.8928
9 Epoch 14000, Loss = 4.8922
10 Epoch 16000, Loss = 4.8911
11 Epoch 18000, Loss = 4.8905
12
13 Prediction Results (ReLU):
14 1!    0.3    (true: 1)
15 2!    1.5    (true: 2)
16 3!    8.2    (true: 6)
17 4!   45.9    (true: 24)
18 5!   257.3   (true: 120)
19 6!  1442.9   (true: 720)
20 7!  8092.5   (true: 5040)
21 8! 45386.9   (true: 40320)
22 9! 254551.8  (true: 362880)
23 10! 1379211.7 (true: 3628800)

```

代码 1.9: ReLU 激活函数的训练与预测结果

```

1 #: python factorial-sigmoid.py
2 Epoch 0, Loss: 705.7546
3 Epoch 2000, Loss: 0.0695
4 Epoch 4000, Loss: 0.0622
5 Epoch 6000, Loss: 0.0573
6 Epoch 8000, Loss: 0.0531
7 Epoch 10000, Loss: 0.0493
8 Epoch 12000, Loss: 0.0459
9 Epoch 14000, Loss: 0.0429
10 Epoch 16000, Loss: 0.0404
11 Epoch 18000, Loss: 0.0383
12
13 Prediction Results (Sigmoid):
14 1!    0.9    (true: 1)
15 2!    2.2    (true: 2)
16 3!    6.5    (true: 6)
17 4!   24.2    (true: 24)
18 5!   115.0   (true: 120)
19 6!   684.2   (true: 720)
20 7!  4955.3   (true: 5040)
21 8!  41680.6  (true: 40320)
22 9!  381847.8 (true: 362880)
23 10! 3497393.9 (true: 3628800)

```

代码 1.10: Sigmoid 激活函数的训练与预测结果

练习

- 1) 设计一个神经网络程序，预测第 x 个斐波那契数，分析效果。
- 2) 设计一个神经网络程序，预测任意 x 的正弦值 $y = \sin(x)$ ，分析效果。

AIE310008 人工智能的软件基础

2 Python 程序的解析和运行

本章学习目标：

- ** 了解 Python 代码的解析与编译过程；
- ** 能够阅读并理解抽象语法树与字节码；
- ** 理解 Python 虚拟机执行字节码的机制。

2.1 Python 语言

Python 是一种解释执行的编程语言，以简洁易读的语法著称，是人工智能、数据科学和快速原型开发的首选语言。它拥有丰富的标准库和第三方生态，如 NumPy、Pandas、Matplotlib、Scikit-learn 和 PyTorch，使得数据处理、可视化、机器学习和深度学习都能高效实现。作为当前 AI 编程中最主要的语言，同学们通过本章内容学习将了解 Python 程序的运行原理及其优缺点，从而能够对现有 AI 软件生态和发展趋势形成一定的理解和分析能力。

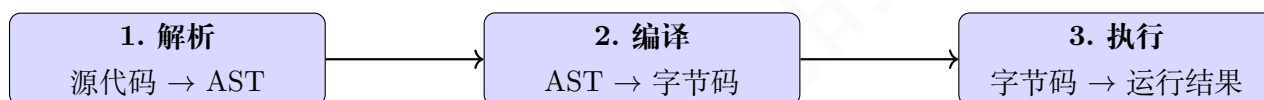


图 2.1: Python 程序运行过程

Python 程序的运行过程可以概括为三个主要阶段：

- 1) **语法分析 (Parsing)**: 解释器首先读取源代码，将其转换为抽象语法树 (Abstract Syntax Tree, AST)。AST 描述了程序的语法结构与层次关系，为后续编译阶段提供结构化表示。
- 2) **字节码生成 (Compilation)**: 抽象语法树被编译为字节码 (Bytecode)。字节码是一种与具体硬件平台无关的中间表示形式，通常以 `.pyc` 文件形式缓存。
- 3) **虚拟机执行 (Execution)**: Python 虚拟机 (Python Virtual Machine, PVM) 逐条解释执行字节码。执行过程中采用基于栈的执行模型，维护调用栈与运行时状态，从而完成表达式计算、变量管理以及函数调用等操作。

可以使用以下命令查看 Python 源代码在不同阶段的表示形式：

```
1 # 查看抽象语法树 (AST)
2 python -m ast toy.py
3 # 查看词元 (token) 序列
4 python -m tokenize toy.py
5 # 查看字节码
6 python -m dis toy.py
7 # 将 Python 源代码编译为字节码 (.pyc)
8 python -m compileall toy.py
```

代码 2.1: Python 代码分析与编译命令

2.2 语法解析

语法解析一般分为词法分析和句法分析两步。本节内容是传统编译原理课程的经典部分，如果同学们已经学习过编译，可以直接跳过。

2.2.1 词法分析

语法解析的第一步是词法解析 (Lexical Analysis)。词法解析的目标是将源程序的字符流 (Character Stream) 划分为一系列具有语法意义的词元 (Token)，并过滤掉对语法结构无关的细节，例如空白符和注释。每个词元通常由其类别以及可选的属性（如标识符名称、常量值）组成。

由于 Python 是一种基于缩进的编程语言，代码块的结构由行首缩进直接决定，而非通过显式的分隔符（如花括号）表示。因此，Python 的词法分析器不仅负责将字符流转换为关键字、标识符、常量等基本记号，还需要根据行首缩进层级的变化显式地产生 INDENT 和 DEDENT 记号，以刻画代码块的开始与结束。

```
1 def relu(x):
2     return max(0.0, x)
3
4 print(relu(100))
```

代码 2.2: python 代码示例

以代码 2.2 中的程序为例，其词法分析后的词元序列（抽象表示）如下所示：

```
1 ENCODING(utf-8)
2 NAME(def) NAME(relu) ( NAME(x) ) : NEWLINE
3 INDENT
4 NAME(return) NAME(max) ( NUMBER(0.0), NAME(x) ) NEWLINE
5 NL
6 DEDENT
7 NAME(print) ( NAME(relu) ( NUMBER(100) ) ) NEWLINE
8 ENDMARKER
```

代码 2.3: Code 2.2 的词法分析结果 (Token 序列)

词法分析通常基于正则表达式实现，用于描述和识别各类词元的词法模式。每一类词元都可以通过一个正则表达式表示，词法分析器通过匹配这些模式，将连续的字符归约为一个个词元。表 2.1 列举了正则表达式中单个字符的表示方法。

表 2.1: 单个字符元素正则表示方法

字符元素	表述形式	含义
特定字符	a	$x = a$
字符范围	$[ab]$	$x \in \{a, b\}$
字符范围	$[a - z]$	$x \in \{a, \dots, z\}$
字符范围	$[a - zA - Z]$	$x \in \{a, \dots, z, A, \dots, Z\}$
通配符	$.$	$x \in \Sigma$
排除特定字符	$\hat{a}$	$x \in \Sigma \setminus \{a\}$
空字符	ϵ	$x \in \emptyset$
特定字符或空	$a?$	$x = a \text{ or } x = \epsilon$

在此基础上，单个字符可以通过选择、连接和闭包等方式组合形成复杂模式。表 2.2 采用递归构造的方法定义了正则表达式的组合规则。

表 2.2: 正则表达式构造方法, 其中 S 和 T 为子正则表达式或单个字符

构造方式	符号	优先级	示例	含义
选择		1	$S T$	$x \in \{S \cup T\}$
连接		2	ST	$x \in \{st \mid s \in S, t \in T\}$
闭包	*	3	S^*	$x \in \{s_1..s_n \mid \forall 1 \leq i \leq n, s_i \in S, 0 \leq n < \infty\}$
正闭包 (扩展)	+	3	S^+	$x \in \{s_1..s_n \mid \forall 1 \leq i \leq n, s_i \in S, 1 \leq n < \infty\}$
区间闭包 (扩展)		3	$S^{\{min, max\}}$	$x \in \{s_1..s_n \mid \forall 1 \leq i \leq n, s_i \in S, min \leq n \leq max\}$

例如, 在 Python 的词法规则中, 标识符 (NAME) 和数值常量 (NUMBER) 可以分别用如下正则表达式近似描述:

$$\begin{aligned}\text{NAME} &\rightarrow [A-Za-z_][A-Za-z_0-9]^* \\ \text{NUMBER} &\rightarrow [0-9]^+(\.[0-9]^+)?\end{aligned}$$

其中, NAME 表示以字母或下划线开头、后续可以跟字母、数字或下划线的标识符, 用于表示变量名和函数名; NUMBER 表示整数或浮点数形式的数值常量。在代码 2.2 的示例程序中, relu、x 和 max 均被识别为 NAME, 而 0.0 和 100 被识别为 NUMBER。

需要注意的是, 在 Python 的词法分析阶段, 关键字 (如 def 和 return) 在词法分析输出中仍然被标记为 NAME 类型, 其关键字属性是在后续语法分析阶段根据具体字符串值进行识别的。因此, 在上述词元序列中, def 和 return 仍然以 NAME(def) 和 NAME(return) 的形式出现。在许多传统编程语言中, 这些关键字通常会在词法分析阶段直接识别。

除上述两类词元外, 词法分析器还会识别定界符和分隔符, 例如括号、逗号和冒号等。这些符号通常通过对固定字符的直接匹配生成, 其词法规则可以表示为:

$$\begin{aligned}\text{LPAREN} &\rightarrow (\\ \text{RPAREN} &\rightarrow) \\ \text{COMMA} &\rightarrow , \\ \text{COLON} &\rightarrow :\end{aligned}$$

换行符 NEWLINE 用于表示逻辑行结束, 其对应的字符通常为操作系统相关的行结束符, 例如:

$$\text{NEWLINE} \rightarrow \backslash n \mid \backslash r \backslash n$$

其中, \n 是类 Unix 系统 (如 Linux 和 macOS) 常用的行结束符, 而 \r\n 是 Windows 系统使用的行结束符。Python 在读取源文件时会统一处理这些不同形式的换行符。与之相关的还有 NL 词元, 它同样表示换行符, 但仅用于标记非逻辑行结束的换行, 例如空行或括号内部的换行。在这些情况下, 换行符不会结束当前语句, 因此词法分析器生成 NL 而不是 NEWLINE。

值得注意的是, NL 的表示和识别超出了正则表达式的能力, 因为它依赖于当前换行所在的上下文, 例如是否位于括号、方括号或花括号内部, 或者该行是否为空行。与之类似, INDENT 和 DEDENT 也无法直接由正则表达式描述, 它们通过比较相邻逻辑行的缩进宽度生成, 用于显式标记代码块的开始和结束。通过这些上下文敏感的词元, Python 的词法分析器能够正确处理缩进和空行, 从而为后续的语法分析提供准确的输入。

此外, Python 源代码中还可以包含注释 (COMMENT)。注释以字符 # 开始, 一直延续到当前行结束, 其词法规则可以近似表示为:

$$\text{COMMENT} \rightarrow \#[^\\n]^*$$

在词法分析的输出中，注释会被识别为独立的 `COMMENT` 词元，并且通常紧跟一个 `NEWLINE` 或 `NL` 词元，用于表示该注释所在行的结束。

与一些编程语言不同，Python 并没有专门的多行注释语法。虽然三引号字符串 (`'''...'''`) 有时被程序员用作“多行注释”，但在词法和语法层面上它们仍然被解析为字符串 (`STRING`)，而不是注释。字符串 (`STRING`) 用于表示文本常量，可以由单引号或双引号包围，也可以使用三引号表示多行字符串。有兴趣的同学可是尝试自己定义 Python 字符串的正则表达式。

通过这些正则表达式和规则，词法分析器可以将源代码中的字符序列归约为代码 2.3 所示的词元序列，为后续的句法分析提供输入。由于正则表达式只能刻画局部的、线性的结构，词法分析阶段并不负责处理嵌套关系或程序的层次结构，这些结构性信息将在句法分析阶段进一步处理。

2.2.2 句法分析

句法分析的目标是在词法分析产生的词元序列基础上，判断程序是否符合语言的语法规则，并据此构造程序的结构化表示，通常是一棵抽象语法树。语法解析阶段并不关心程序的“含义是否正确”（例如变量是否已定义、类型是否匹配），而只关注程序在结构上是否合法。大多数编程语言的语法可以形式化地描述为上下文无关文法（Context-Free Grammar, CFG）。一个上下文无关文法通常表示为一个四元组 $G = (V, \Sigma, P, S)$ ，其中：

- V 是非终结符集合；
- Σ 是终结符集合，对应词法分析阶段产生的词元；
- P 是产生式集合；
- $S \in V$ 是开始符号。

产生式通常写作 $S \rightarrow \beta$ ，其中左侧 S 是一个非终结符，右侧 β 是由终结符和非终结符组成的符号序列。通过一系列产生式的推导，语法解析器可以判断一个程序是否可以从开始符号 S 推导得到；若可以，则该程序在语法上是合法的。

以 Python 的一个简化子集为例，其核心语法规则可以形式化描述如下：

$$\begin{aligned} \text{Module} &\rightarrow \text{StmtList} \\ \text{StmtList} &\rightarrow \text{Stmt StmtList} \mid \epsilon \\ \text{Stmt} &\rightarrow \text{SimpleStmt} \mid \text{CompoundStmt} \\ \text{CompoundStmt} &\rightarrow \text{FunctionDef} \\ \text{FunctionDef} &\rightarrow \text{def Identifier (ParamList) : NEWLINE Block} \\ \text{Block} &\rightarrow \text{INDENT StmtList DEDENT} \\ \text{SimpleStmt} &\rightarrow \text{ReturnStmt NEWLINE} \mid \text{ExprStmt NEWLINE} \\ \text{ReturnStmt} &\rightarrow \text{return Expr} \\ \text{ExprStmt} &\rightarrow \text{Expr} \end{aligned}$$

语法解析的输出通常是一棵抽象语法树。抽象语法树会刻意丢弃与语义无关的语法细节（如括号、分号、换行），只保留程序的层次结构和关键构造。以代码 2.2 为例，经语法解析后，该程序可被表示为代码 2.2 所示的抽象语法树。

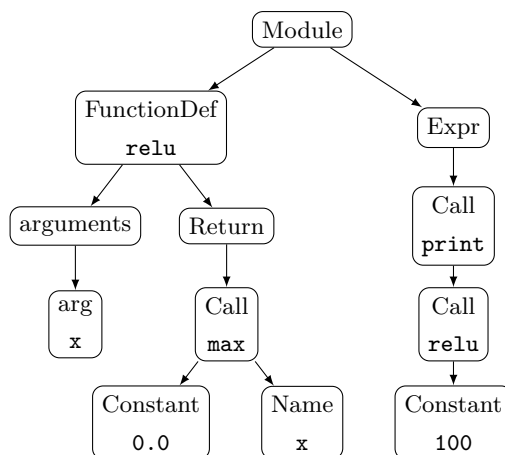


图 2.2: 代码 2.2 对应的抽象语法树

2.3 编译

在完成语法解析并构造抽象语法树之后，Python 并不会直接对抽象语法树进行解释执行，而是进入编译阶段。该阶段的主要任务是将抽象语法树转换为一种更适合虚拟机执行的中间表示。由于该中间表示由一组线性排列的指令组成，并以字节为基本单位对指令及其操作数进行编码，因此该表示通常被称为字节码。

从抽象层次上看，Python 的编译过程可以概括为：

抽象语法树 → 代码对象 → 字节码

代码对象 (Code Object) 是对一段可执行代码的封装，内部包含字节码指令序列以及与之配套的常量表、符号表、局部变量信息等元数据。以代码 2.2 为例，在编译阶段共产生两个代码对象。第一个是模块级代码对象，用于描述源文件中所有顶层语句的执行过程，包括函数对象的创建以及后续的函数调用。第二个是函数 `relu` 对应的代码对象，该代码对象仅包含函数体内部的字节码指令，并作为常量存储在模块级代码对象中。

2.3.1 字节码指令

字节码指令是 Python 程序执行的最小操作单元，每条指令由操作码 (Opcode) 和可选操作数 (Operand) 组成，用于描述程序在虚拟机上的具体执行步骤。编译器会遍历 AST，根据不同类型的语法节点生成对应的字节码指令。这一过程通常是结构化的：例如，函数定义节点会生成函数对象相关的指令，而表达式节点则会生成计算与调用相关的指令。

以代码 2.2 为例，其对应的字节码如代码 2.4 所示。

```
1 # -----代码对象1: 模块-----
2 # -----源代码第1行: 函数定义 relu-----
3     0 LOAD_CONST          0 (<code object relu ...>)
4     2 LOAD_CONST          1 ('relu')
5     4 MAKE_FUNCTION       0
6     6 STORE_NAME          0 (relu)
7 # -----源代码第4行: 调用 print(relu(100))-----
8     8 LOAD_NAME           1 (print)
9    10 LOAD_NAME           0 (relu)
10    12 LOAD_CONST          2 (100)
11    14 CALL_FUNCTION       1
12    16 CALL_FUNCTION       1
13    18 POP_TOP
14    20 LOAD_CONST          3 (None)
15    22 RETURN_VALUE
16 # -----代码对象2: 函数 relu 函数体-----
17 # -----源代码第2行: 函数 relu 函数实现-----
18    0 LOAD_GLOBAL          0 (max)
19    2 LOAD_CONST           1 (0.0)
20    4 LOAD_FAST             0 (x)
21    6 CALL_FUNCTION         2
22    8 RETURN_VALUE
```

代码 2.4: 代码 2.2 对应的字节码

上述字节码示例中的主要指令含义如下：

- **LOAD_CONST**: 将指定的常量从常量表加载到操作数栈上。参数表示常量表中的索引，例如数值、字符串或函数对象。
- **LOAD_NAME / LOAD_GLOBAL**: 将符号对应的对象从符号表加载到栈上。参数表示符号表中的索引。
- **STORE_NAME**: 将栈顶对象绑定到指定名字，并存入符号表。参数表示符号表中的索引。
- **MAKE_FUNCTION**: 根据栈顶的代码对象及默认参数信息创建函数对象，并压入栈。参数表示函数的标志或可选参数数量。
- **CALL_FUNCTION**: 从栈中弹出函数和指定数量的参数执行调用，并将调用结果压入栈。参数表示函数调用时的参数数量。
- **POP_TOP**: 弹出栈顶元素，通常用于丢弃表达式的返回值。此指令没有参数。
- **RETURN_VALUE**: 从当前栈帧返回栈顶对象作为返回值。此指令没有参数。
- **LOAD_FAST**: 将指定局部变量或参数从局部变量表加载到栈上。参数表示局部变量表中的索引。

2.3.2 代码对象中的表

每个代码对象除了存储字节码指令外，还维护几张重要的表，字节码指令在执行过程中需要访问这些表来获取操作数或目标对象：

- **常量表**: 存储字面量（如数值、字符串）和内部代码对象。指令 **LOAD_CONST n** 使用常量表索引 *n* 获取对应的常量。
- **符号表**: 存储模块或函数中使用的名字（函数名、全局变量名等）。指令 **LOAD_NAME n** 或 **STORE_NAME n** 使用符号表索引 *n*。函数内部也可以通过 **LOAD_GLOBAL n** 访问模块级符号表中的对象。
- **局部变量表**: 存储函数的局部变量名，包括参数和局部变量。指令 **LOAD_FAST n / STORE_FAST n** 使用局部变量表索引 *n*。

以代码对象 1 为例，其维护的表内容如下：

常量表		符号表		局部变量表 (co_varnames)	
Index	Value	Index	Name	Index	Variable
0	<code object relu>	0	relu		
1	'relu'	1	print		
2	100				
3	None				

图 2.3: 代码对象 1（模块）中各表的内容

代码对象 2（函数 `relu`）维护的表内容如下：

常量表		符号表		局部变量表	
Index	Value	Index	Name	Index	Variable
0	0.0	0	max	0	x

图 2.4: 代码对象 2（函数 `relu`）中各表的内容

2.4 程序执行

Python 字节码并不是由计算机或操作系统直接运行，而是通过一个称为 Python 虚拟机的软件层来完成。因此 Python 又被称为解释执行语言。

2.4.1 Python 虚拟机

虚拟机的核心作用是提供一个统一的执行环境，将高级 Python 字节码翻译为可以被计算机理解的指令。Python 虚拟机（PVM）采用栈式架构，程序的运行状态通过一个操作数栈保存。程序在执行时，会先将需要使用的操作数压入栈顶，字节码指令从栈顶弹出数据进行计算，结果再压回栈顶。

当 PVM 开始执行某个代码对象时，会创建一个执行帧（frame）。如果一个函数被多次调用，则该函数代码的对象可以对应多个帧。一个执行帧包含以下内容：指向代码对象的引用、指令计数器（program counter）、操作数栈等内容。

以代码对象 1 为例，其执行过程如下。图 2.5展示了执行帧操作数栈的变化情况。

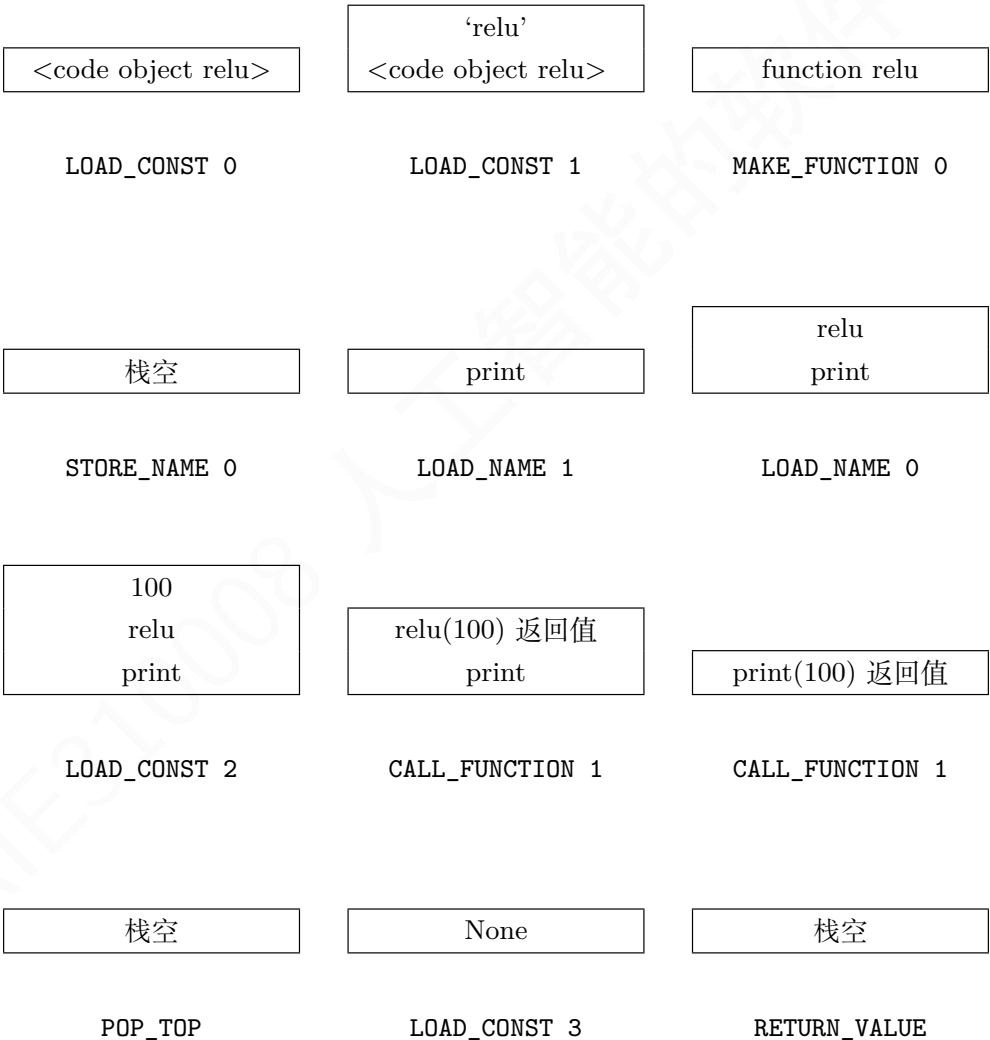


图 2.5: 代码对象 1（模块）执行过程中其操作数栈的变化示意图

- 1) `LOAD_CONST 0`: 取常量表索引 0 → `relu` 的函数体，入栈。
- 2) `LOAD_CONST 1`: 取常量表索引 1 → 函数名 `'relu'`，入栈。

- 3) MAKE_FUNCTION 0: 生成函数对象 (0 表示无默认参数), 弹出栈顶代码对象和名字, 将函数对象入栈。
- 4) STORE_NAME 0: 符号表索引 0 $\rightarrow$ relu, 将栈顶函数对象保存在全局符号表。
- 5) LOAD_NAME 1: 符号表索引 1 $\rightarrow$ print, 入栈。
- 6) LOAD_NAME 0: 符号表索引 0 $\rightarrow$ relu, 入栈。
- 7) LOAD_CONST 2: 常量表索引 2 $\rightarrow$ 100, 入栈作为函数参数。
- 8) CALL_FUNCTION 1: 调用 relu(100), 参数数量 1, 返回值压栈。
- 9) CALL_FUNCTION 1: 调用 print(返回值), 参数数量 1。
- 10) POP_TOP: 弹出 print 返回值 None。
- 11) LOAD_CONST 3: 常量表索引 3 $\rightarrow$ None, 作为模块返回值。
- 12) RETURN_VALUE: 返回栈顶值, 模块执行结束。

下面对代码对象 2 (函数 `relu`) 的执行流程进行分析。图 2.6 展示了执行帧操作数栈的变化情况。

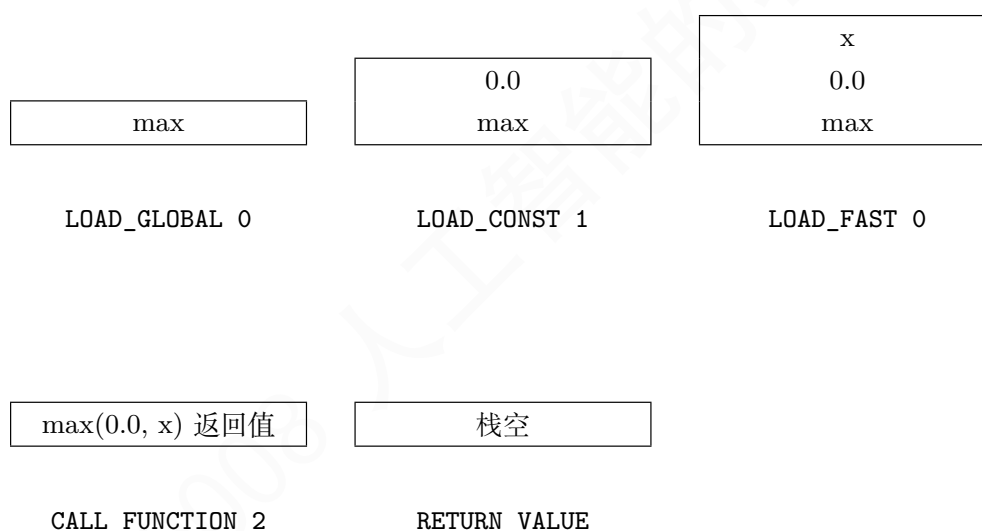


图 2.6: 代码对象 2 (函数 `relu`) 执行过程中其操作数栈的变化示意图

- 1) LOAD_GLOBAL 0: 从函数符号表加载 `max` 函数到栈顶。
- 2) LOAD_CONST 1: 将常量表索引 1 $\rightarrow$ 0.0 入栈。
- 3) LOAD_FAST 0: 将局部变量表索引 0 $\rightarrow$ 参数 `x` (例如 100) 入栈。
- 4) CALL_FUNCTION 2: 调用 `max(0.0, x)`, 参数数量 2, 返回值压入栈顶。
- 5) RETURN_VALUE: 返回栈顶值作为函数返回结果, 函数执行结束。

练习

- 1) 定义一个正则表达式，用于匹配 Python 的单引号、双引号及三引号字符串字面量，并使用 Python 的 `re` 模块对若干示例字符串进行测试，观察匹配结果是否符合预期。

```
1 import re
2
3 # 规则示例: pattern_name = r'[A-Za-z_][A-Za-z0-9_]*'
4 # TODO: 定义正则表达式匹配单/双/三引号字符串
5 pattern_string = r'填写你的正则'
6
7 # 示例字符串列表
8 examples = [
9     's1 = "Hello, world!"',
10    's2 = \'single quotes\'',
11    '''
12    s3 = multi
13        line
14        string
15    ''',
16 ]
17
18 for text in examples:
19     matches = re.findall(pattern, text, flags=re.DOTALL)
20     print(f"Text: {text!r}")
21     print(f"Matches: {matches}")
22     print("-" * 40)
```

代码 2.5: Python 字符串正则 PoC 框架

- 2) 以上节课的神经网络程序为例，
 - a) 分析代码与其词元序列及抽象语法树的对应关系；
 - b) 生成字节码，并解释其执行过程。
- 3) `python -m` 命令的功能本质上是通过调用相应模块实现的，如下代码所示。测试下列代码的效果。

```
1 import sys
2 import ast
3 import dis
4 import tokenize
5 from io import BytesIO
6
7 def usage():
8     print("Usage: python pycview.py <source.py> -ast|-ir|-tok")
9     sys.exit(1)
10
11 def load_source(filename):
12     with open(filename, "r", encoding="utf-8") as f:
13         return f.read()
14
15 def dump_ast(source, filename):
16     tree = ast.parse(source, filename=filename)
```

```

17     print(ast.dump(tree, indent=2))
18 def dump_ir(source, filename):    code = compile(source, filename, "exec")
19     dis.dis(code)
20
21 def dump_tokens(source):
22     reader = BytesIO(source.encode("utf-8")).readline
23     for tok in tokenize.tokenize(reader):
24         print(tok)
25
26 def main():
27     if len(sys.argv) != 3:
28         usage()
29     filename = sys.argv[1]
30     option = sys.argv[2]
31     if option not in ("-ast", "-ir", "-tok"):
32         usage()
33     source = load_source(filename)
34     if option == "-ast":
35         dump_ast(source, filename)
36     elif option == "-ir":
37         dump_ir(source, filename)
38     elif option == "-tok":
39         dump_tokens(source)
40
41 if __name__ == "__main__":
42     main()

```

代码 2.6: python 代码解析模块示例

3 Python 与 Native Code 交互

本章学习目标：

- *** 理解 Python 解释执行模型对程序性能的影响及其根本原因；
- *** 掌握 Python 调用 native code 的基本方式及其底层执行过程；
- *** 理解装饰器的工作原理及其在跨语言调用中的工程意义。

3.1 解释执行的性能问题

3.1.1 解释执行 vs 编译执行

程序语言的执行方式通常可以分为两大类：解释执行与编译执行。

- **解释执行**：程序在运行时由解释器逐条读取源代码（或中间表示），并立即执行相应语义。我们在上一章学习的 Python 采用的正是这种解释执行模型：源代码首先被翻译为字节码，随后由 PVM 逐条解释执行。
- **编译执行**：程序运行前，由编译器将源代码整体翻译为面向目标硬件架构的机器代码（即 native code 或本地代码），程序运行时由处理器直接执行这些指令，而无需解释器参与。C/C++、Rust 等语言通常采用这种执行模式。由于编译过程发生在程序运行之前，因此这种方式也称为 Ahead-of-Time (AOT) 编译。

解释执行的主要优点是执行过程灵活，便于交互式编程和动态特性支持，开发效率高。但每条指令的执行都伴随着解释、动态类型检查，运行时开销较大。编译执行运行阶段无需解释和类型检查等操作，且在编译阶段可进行全局优化（如内联、常量传播、寄存器分配等），因此运行性能更好。

3.1.2 案例分析

为具象化说明解释执行机制对 Python 数值计算性能的影响，本节设计了一组一维向量点积实验，对比纯 Python 实现、基于 NumPy 的实现以及手写 C 函数实现之间的运行开销差异。三种实现均采用相同的数学计算公式，以确保性能差异主要来源于执行模型与运行时机制，而非算法复杂度或计算逻辑本身。

一维向量点积的定义如下：

$$\text{dot}(a, b) = \sum_{i=0}^{N-1} a_i \cdot b_i,$$

其中 a 与 b 为长度为 N 的实数向量。

实验组 1: 纯 Python 实现 该实现完全使用 Python 语言编写，其循环控制与数值运算均由 Python 解释器执行。在 Python 中，对象（object）是程序运行时创建的基本实体，每个对象都具有三个基本属性：标识、类型和值。因此，Python 程序中的变量并不直接存储数据本身，而是保存对对象的引用。

在这种模型下，每一次循环迭代都会触发解释器的字节码调度，并涉及动态类型检查以及对象级别的算术运算。因此，该实现的整体运行性能在很大程度上依赖于 Python 解释执行的效率。

```
1 def dot_python(a, b):
2     # 使用 Python 的浮点对象进行累加
3     s = 0.0
4
5     # Python for 循环
6     # 每一次迭代都会经历解释器调度和动态类型检查
7     for i in range(len(a)):
8         # a[i] 和 b[i] 均为 Python 对象，
9         # 乘法与加法均为对象级别运算
10        s += a[i] * b[i]
11
12    return s
```

代码 3.1: 纯 Python 实现的一维向量点积函数

实验组 2: 基于 NumPy 的实现 NumPy 是 Python 生态中最核心的科学计算库之一，提供了高效的多维数组对象以及大量底层优化的数值计算函数。该实现调用 NumPy 提供的 dot 接口。虽然函数调用发生在 Python 层，但实际的数值计算由底层库实现，从而避免了 Python 解释器在循环与算术运算中的参与。

```
1 def dot_numpy(a, b):
2     # np.dot 在 Python 层仅作为接口存在
3     # 实际计算由底层库实现
4     return np.dot(a, b)
```

代码 3.2: 基于 NumPy 的一维向量点积函数

实验组 3: 基于手写 C 函数的实现 该实现直接使用 C 语言编写点积函数，所有循环与数值运算均在本地机器码层面执行，不存在解释执行、动态类型检查或对象封装开销。该实现可作为理想的本地执行性能基线，对比 Python 解释执行所引入的额外成本。

```
1 double dot_c(const double *a, const double *b, size_t n) {
2     // 使用原生 double 类型进行累加
3     double s = 0.0;
4
5     // 纯 C 语言 for 循环，直接编译为机器指令
6     for (size_t i = 0; i < n; i++) {
7         s += a[i] * b[i];
8     }
9
10    return s;
11 }
```

代码 3.3: C 语言实现的一维向量点积函数

实验过程和结果 代码 3.4为实验的具体过程。为确保计算规模足够大，提升结果的显著性，实验中向量长度设置为 $N = 5 \times 10^6$ 。所有向量元素均为双精度浮点数，并通过固定随机种子生成，以保证实验结果的可重复性。

```
1 # Data preparation
2 N = 5_000_000      # 向量长度
3 SEED = 42
4 random.seed(SEED)
5
6 # Python list: 元素为 Python float 对象 (双精度浮点数)
7 a_list = [random.random() for _ in range(N)]
8 b_list = [random.random() for _ in range(N)]
9
10 # NumPy array: 连续内存中的 double 类型 (双精度浮点数)
11 a_np = np.array(a_list, dtype=np.float64)
12 b_np = np.array(b_list, dtype=np.float64)
13
14 # Benchmark helper
15 def benchmark(func, *args):
16     start = time.perf_counter()
17     func(*args)
18     end = time.perf_counter()
19     return end - start
20
21 # Benchmark
22 # 纯 Python: 解释器逐元素执行
23 t_py = benchmark(dot_python, a_list, b_list)
24
25 # NumPy: Python 层函数调用, 底层 C 实现
26 t_np = benchmark(dot_numpy, a_np, b_np)
27
28 # C 实现: 全部在本地机器码中执行
29 # 此处省略了加载C语言库函数的代码
30 t_c = benchmark(dot_c, a_np, b_np)
31
32 print(f"Dot via Python: {t_py:.4f} s")
33 print(f"Dot via NumPy : {t_np:.4f} s")
34 print(f"Dot via C      : {t_c:.4f} s")
```

代码 3.4: 点积运算性能对比实验

实验结果如图 3.1所示。

```
1 #: python bench-dot.py
2 Dot via Python : 0.3283 s
3 Dot via NumPy  : 0.0048 s
4 Dot via C      : 0.0048 s
```

图 3.1: 点积运算性能测试结果

从结果可以观察到，不同实现方式之间存在数量级上的性能差异。纯 Python 实现的运行时间约为 0.3 秒，明显慢于另外两种实现。相比之下，NumPy 和原生 C 实现的运行时间都约为 0.005 秒，性能提升接

近两个数量级，表明 NumPy 在该算例中和原生 C 代码的性能类似。该实验清晰地表明：在数值密集型计算中，使用编译执行语言而非解释执行语言是提升性能的关键手段。

3.1.3 更多探讨

可以把 Python 实现为编译执行吗？从理论和工程实践角度看，答案是“可以，但需要付出代价”。Python 之所以难以像 C 或 Rust 那样采用纯粹的提前编译执行，根本原因在于其高度动态的语言特性，例如动态类型、动态属性或结构体字段、反射机制以及运行时生成代码等（如代码 3.5 所示）。这些特性使得编译器在编译期难以确定变量类型、对象布局和调用目标，从而难以稳定地产生高效的本地机器码。

```
1 class Person:
2     def __init__(self, name):
3         self.name = name
4
5 p = Person("Alice")
6
7 # 动态属性绑定：运行时添加新的属性
8 p.age = 20
9 print("Age:", p.age)
10
11 # 反射：通过字符串访问和修改属性
12 print("Name:", getattr(p, "name"))
13 setattr(p, "name", "Bob")
14 print("Updated name:", p.name)
15
16 # 运行时代码生成：动态创建并执行代码
17 code = """
18 def greet(person):
19     print("Hello,", person.name)
20 """
21 exec(code)
22
23 greet(p)
```

代码 3.5: Python 动态特性示例：动态属性、反射与运行时代码生成

在实际系统中，AOT 编译 Python 的主要思路是通过引入静态类型信息或限制动态行为，将 Python 约化为可静态分析的子集，从而在编译期生成高效的本地机器码。Cython 正是这一思路的典型代表：当代码中提供了足够的类型注解时，Cython 可以在编译期生成接近 C 语言性能的本地代码，但其代价是牺牲部分 Python 的动态性。

```
1 def dot_python(double[:] a, double[:] b):
2     cdef Py_ssize_t i, n = a.shape[0]
3     cdef double s = 0.0
4     for i in range(n):
5         s += a[i] * b[i]
6     return s
```

代码 3.6: Cython 点积函数

介于纯解释执行与提前编译执行之间，还存在一种基于即时编译（Just-In-Time, JIT）的执行方式。JIT 编译器在程序运行过程中对热点代码进行分析，并将频繁执行的代码片段即时编译为本地机器码，从而在保留语言动态特性的同时降低解释执行带来的运行时开销。常见的相关工具包括 PyPy 和 Numba。其

中, PyPy 是一种替代性的 Python 解释器实现, 通过内置的追踪式 JIT 在运行时优化热点代码; 而 Numba 是面向数值计算的 JIT 编译库, 通常通过函数装饰器将 Python 函数即时编译为机器码, 特别适合加速基于 NumPy 的计算任务。代码 3.7 和 3.8 展示了一组使用 Numba 的性能测试及其结果。

```
1 from numba import njit
2
3 N = 5_000_000
4
5 def dot_python(a, b):
6     s = 0.0
7     for i in range(len(a)):
8         s += a[i] * b[i]
9     return s
10
11 @njit
12 def dot_numba(a, b):
13     s = 0.0
14     for i in range(len(a)):
15         s += a[i] * b[i]
16     return s
17
18 # 注意: 这里如果用 list 几乎没有优化效果。
19 a_np = np.array([random.random() for _ in range(N)])
20 b_np = np.array([random.random() for _ in range(N)])
21
22 # Python benchmark
23 t0 = time.perf_counter()
24 dot_python(a_list, b_list)
25 t1 = time.perf_counter()
26
27 # Numba 第一次调用 (包含 JIT 编译)
28 t2 = time.perf_counter()
29 dot_numba(a_np, b_np)
30 t3 = time.perf_counter()
31
32 # Numba 第二次调用 (已编译)
33 t4 = time.perf_counter()
34 dot_numba(a_np, b_np)
35 t5 = time.perf_counter()
36
37 print("Python time:", t1 - t0)
38 print("Numba first call (compile + run):", t3 - t2)
39 print("Numba second call (run only):", t5 - t4)
```

代码 3.7: Numba JIT 实现及其基准测试

```
1 #: python bench-numba.py
2 Python time: 1.0850062351673841
3 Numba first call (compile + run): 0.46062095928937197
4 Numba second call (run only): 0.006434974260628223
```

代码 3.8: Numba JIT 基准测试结果

3.2 本地代码

所谓本地代码，是指针对具体硬件体系结构（如 x86_64、ARM）生成的机器指令序列。程序运行时，这些指令由处理器直接执行，而无需解释器或虚拟机在运行过程中参与指令调度。本地代码通常由编译器在程序运行前生成，其执行效率主要取决于处理器微架构以及编译器的优化策略。

以代码 3.3 中的 C 语言点积函数为例，编译器会将高级语言中的循环与算术表达式翻译为一段连续的汇编指令。代码 3.9 展示了该函数在 ARM 架构（AArch64）下生成的汇编代码片段。可以看到，函数首先在栈上分配局部变量空间并保存参数，随后通过加载指令（如 `ldr`）读取数组元素，通过浮点运算指令 `fmadd` 完成乘加操作，并利用分支指令（如 `b`、`b.hs`）实现循环控制。整个点积计算过程被映射为一系列显式的内存访问、算术运算和控制流指令，每条指令均由 CPU 直接执行，不再涉及动态类型检查或解释执行带来的额外开销。

```
1      sub    sp, sp, #48      ; 为函数栈帧分配 48 字节空间
2      str    x0, [sp, #40]    ; 保存参数 a 指针
3      str    x1, [sp, #32]    ; 保存参数 b 指针
4      str    x2, [sp, #24]    ; 保存参数 n (数组长度)
5      movi   d0, #0000000000000000
6      str    d0, [sp, #16]    ; s = 0.0 (累加器)
7      str    xzr, [sp, #8]    ; i = 0 (循环计数器)
8      b      .LBB0_1         ; 跳转到循环判断
9  .LBB0_1:                    ; 循环入口 (Loop Header)
10     ldr     x8, [sp, #8]     ; x8 = i
11     ldr     x9, [sp, #24]    ; x9 = n
12     subs   x8, x8, x9       ; 比较 i 和 n
13     b.hs   .LBB0_4         ; 如果 i >= n 跳出循环
14     b      .LBB0_2         ; 否则进入循环体
15
16  .LBB0_2:                    ; 循环体: 计算 s += a[i] * b[i]
17     ldr     x8, [sp, #40]    ; x8 = a 指针
18     ldr     x9, [sp, #8]     ; x9 = i
19     ldr     d0, [x8, x9, lsl #3] ; d0 = a[i] (double, 8 字节)
20
21     ldr     x8, [sp, #32]    ; x8 = b 指针
22     ldr     x9, [sp, #8]     ; x9 = i
23     ldr     d1, [x8, x9, lsl #3] ; d1 = b[i]
24
25     ldr     d2, [sp, #16]    ; d2 = 当前累加值 s
26     fmadd   d0, d0, d1, d2   ; d0 = a[i] * b[i] + s (FMA 指令)
27     str     d0, [sp, #16]    ; 更新 s
28     b      .LBB0_3         ; 继续循环
29
30  .LBB0_3:                    ; 循环尾: i++
31     ldr     x8, [sp, #8]     ; x8 = i
32     add     x8, x8, #1       ; i = i + 1
33     str     x8, [sp, #8]    ; 保存更新后的 i
34     b      .LBB0_1         ; 回到循环判断
35
36  .LBB0_4:                    ; 循环结束
37     ldr     d0, [sp, #16]    ; 返回值 s
38     add     sp, sp, #48     ; 恢复栈指针
39     ret
```

代码 3.9: 代码 3.3 对应的汇编代码 (ARM)

3.3 Python 与本地代码交互

为减少解释器和动态类型系统带来的运行时开销，Python 生态通常将核心计算下沉至本地代码，从而在保持语言易用性的同时获得接近底层语言的执行效率。Python 调用本地代码通常通过外部函数接口 (Foreign Function Interface, FFI) 实现。FFI 是一种允许程序调用由其它语言实现函数的机制，使高层语言能够在运行过程中执行本地代码，并在调用完成后继续自身的执行流程。

3.3.1 Python 调用本地代码

在 Python 中，调用已经编译好的本地代码库（以 C 为例）通常包括两步：

- 1) **编译 C 代码为动态库**：使用编译器（如 GCC 或 Clang）将 C 源文件编译为共享库（动态链接库）。

```
1 clang -O3 -shared -fPIC dot.c -o libdot.so
```

其中：

- -O3 开启最高优化等级，提高运行速度；
- -shared 生成动态库而非可执行文件；
- -fPIC 保证生成位置无关代码，可被动态链接。

- 2) **在 Python 中加载并调用动态库**：Python 可通过 ctypes 模块在运行时加载并调用已编译的本地代码，其基本过程如下：

- 使用 LoadLibrary 加载动态库，完成运行时链接；
- 通过函数名获取库中导出的 C 函数符号；
- 显式指定函数的返回类型 (restype) 与参数类型 (argtypes)；
- 将 Python 对象中可暴露的连续内存区域地址转换为 C 指针并传递给本地函数；
- 本地代码直接在原生机器指令上执行并返回结果。

```
1 import ctypes
2
3 # 加载动态库
4 lib = ctypes.cdll.LoadLibrary("./libdot.so")
5
6 # 设置函数返回类型和参数类型
7 lib.dot_c.restype = ctypes.c_double
8 lib.dot_c.argtypes = [ ctypes.POINTER(ctypes.c_double), ctypes.POINTER(ctypes.c_double),
9                        ctypes.c_int,
10 ]
11
12 # 封装为 Python 函数，或直接在 Python 代码中调用
13 def dot(a, b):
14     return lib.dot_c(
15         a.ctypes.data_as(ctypes.POINTER(ctypes.c_double)),
16         b.ctypes.data_as(ctypes.POINTER(ctypes.c_double)),
17         a.size
18     )
```

代码 3.10: 静态绑定：使用 ctypes 调用 C 实现的点积函数

代码 3.10 展示了调用本地函数 `dot_c` 的方法。`dot_c` 与 C 代码中定义的函数名一一对应，其函数接口及类型信息在构建阶段即已确定。这种调用方式属于静态绑定。在该用例中，本地函数调用被封装为普通的 Python 函数，并在封装层中完成参数转换与类型检查。

在 Python 中，对象的具体类型通常只有在运行时才能确定，因此在调用本地代码时，需要根据数据类型选择合适的函数实现。与静态绑定相对的是动态绑定 (Dynamic Binding)。如代码 3.11 所示，在动态绑定方式下，程序可以在运行时根据对象的类型选择不同的点积函数实现，例如 `dot_float` 或 `dot_double`。

```
1 lib = ctypes.cdll.LoadLibrary("./libdot.so")
2 def get_dot_func(a: np.ndarray):
3     """
4     根据数组类型动态选择调用的 C 函数
5     """
6     if a.dtype == np.int32:
7         func_name = "dot_int"
8         dot_func.restype = ctypes.c_int
9         ptr_type = ctypes.POINTER(ctypes.c_int)
10    elif a.dtype == np.float64:
11        func_name = "dot_double"
12        dot_func.restype = ctypes.c_double
13        ptr_type = ctypes.POINTER(ctypes.c_double)
14    else:
15        raise TypeError("Unsupported dtype")
16
17    dot_func = getattr(lib, func_name)
18    dot_func.argtypes = [ptr_type, ptr_type, ctypes.c_size_t]
19    return dot_func, ptr_type
20
21 def dot(a: np.ndarray, b: np.ndarray):
22     """
23     Python 封装函数，内部调用对应的 C 实现
24     """
25     dot_func, ptr_type = get_dot_func(a)
26
27     return dot_func(
28         a.ctypes.data_as(ptr_type),
29         b.ctypes.data_as(ptr_type),
30         a.size
31     )
```

代码 3.11: 动态绑定：使用 ctypes 调用 C 实现的点积函数

3.3.2 通过 Python 装饰器交互

虽然用户可以直接通过接口调用本地代码，但这种方式通常需要显式处理函数签名、参数转换以及调用细节，对使用者而言存在一定的工程负担。基于 Python 装饰器的方式提供了另一种更高层次的交互抽象。对于添加了装饰器的函数，Python 拦截并替换原始 Python 函数对象，并按照装饰器的逻辑对该函数的内容进行编译和运行。

代码 3.12 定义了三个 Python 函数，并通过 `@kernel` 装饰器标记。每个函数描述一个简单的向量计算操作，分别对应逐元素向量乘法、向量加法以及乘加运算。在这种模式下，函数体主要用于表达计算逻辑，而具体的执行方式由装饰器及其背后的 DSL 运行时系统负责处理。

```

1 @kernel
2 def vec_elem_mul(a, b):
3     # element-wise multiplication
4     return a * b
5
6 @kernel
7 def vec_elem_add(a, b):
8     # element-wise addition
9     return a + b
10
11 @kernel
12 def vec_elem_fma(a, b, c):
13     # element-wise fused multiply-add: a * b + c
14     return (a * b) + c

```

代码 3.12: 通过装饰器使用 DSL

装饰器 `@kernel` 的实现如代码 3.13 所示。该装饰器并不直接执行被修饰函数的 Python 代码，而是基于抽象语法树对函数体进行解析与分析，将其视为一段领域专用语言（DSL：Domain-Specific Language）。在函数定义阶段，装饰器首先获取并解析函数源码，提取函数参数与核心计算表达式；随后对表达式对应的 AST 进行遍历与降级（lowering），将高层的算术运算映射为预先实现的本地向量算子。在运行时，装饰器生成的包装函数负责完成参数准备与调度，并将实际计算委托给本地代码执行。

```

1 def kernel(func):
2     # -----
3     # 第一阶段：解析 Python 抽象语法树 (AST)
4     # -----
5     src = inspect.getsource(func) # 获取被修饰函数的源码字符串
6     tree = ast.parse(src) # 将源码解析为 Python 抽象语法树
7     func_def = tree.body[0] # 取出函数定义节点
8     return_stmt = func_def.body[0] # 函数体内部只有一条return语句
9     expr = return_stmt.value # 提取赋值语句右侧的表达式 AST
10    arg_names = [arg.arg for arg in func_def.args.args] # 提取函数参数名列表
11
12    # -----
13    # 第二阶段：AST 降级 (Lowering)；将高层表达式映射为本地算子调用
14    # -----
15    def lower(node, env, n):
16        """
17        递归遍历表达式 AST，将其转换为对本地向量算子的调用。
18        参数：
19            node : 当前 AST 节点
20            env  : 变量名到数组的映射环境
21            n    : 向量长度
22        """
23
24        # 如果节点是变量名，直接从环境中取出对应数组
25        if isinstance(node, ast.Name):
26            return env[node.id]
27
28        # 如果节点是二元运算（如加法、乘法）
29        if isinstance(node, ast.BinOp):

```

```

30     left = lower(node.left, env, n) # 递归处理左右子表达式
31     right = lower(node.right, env, n)
32     out = np.empty_like(left) # 为结果分配输出数组
33
34     # 根据运算类型选择对应的本地算子
35     if isinstance(node.op, ast.Mult):
36         lib.vec_elem_mul(
37             left.ctypes.data_as(ctypes.POINTER(ctypes.c_double)),
38             right.ctypes.data_as(ctypes.POINTER(ctypes.c_double)),
39             out.ctypes.data_as(ctypes.POINTER(ctypes.c_double)),
40             n,
41         )
42     elif isinstance(node.op, ast.Add):
43         lib.vec_elem_add(
44             left.ctypes.data_as(ctypes.POINTER(ctypes.c_double)),
45             right.ctypes.data_as(ctypes.POINTER(ctypes.c_double)),
46             out.ctypes.data_as(ctypes.POINTER(ctypes.c_double)),
47             n,
48         )
49     else:
50         # 当前示例不支持的运算符
51         raise NotImplementedError("Unsupported operator")
52
53     return out
54
55     # 不支持的 AST 节点类型
56     raise NotImplementedError("Unsupported AST node")
57
58     # -----
59     # 第三阶段：运行时包装函数
60     # -----
61     def wrapper(*args):
62         """
63         wrapper 是最终返回给用户调用的函数。
64         """
65
66         # 将输入参数统一转换为 float64 的 NumPy 数组
67         arrays = [np.asarray(a, dtype=np.float64) for a in args]
68         n = arrays[0].size # 假定所有向量长度相同
69         env = dict(zip(arg_names, arrays)) # 构造变量名到数组的映射环境
70
71         # 对表达式 AST 进行降级并执行本地计算
72         result = lower(expr, env, n)
73         return result
74
75     # 返回包装后的函数，替代原始 Python 函数
76     return wrapper

```

代码 3.13: 定义装饰器：调用 Native Code

装饰器 @kernel 的执行流程可以概括为以下几个步骤：

- 1) **解析 Python 抽象语法树**：使用 `inspect.getsource(func)` 获取函数源码，并通过 `ast.parse` 将

其转换为抽象语法树。由于 DSL 函数仅包含一条返回表达式（如 `return a * b`），因此只需提取该返回语句对应的表达式节点即可。

- 2) **定义 AST 节点处理规则**：通过递归函数 `lower` 遍历 AST 节点。对于变量节点 (`ast.Name`)，直接从运行时环境中获取对应的 NumPy 数组；对于二元运算节点 (`ast.BinOp`)，递归计算左右子表达式，并调用本地算子完成对应的逐元素运算，例如 `lib.vec_elem_mul` 表示逐元素乘法，`lib.vec_elem_add` 表示逐元素加法。
- 3) **构造运行时包装函数并返回**：定义 `wrapper` 函数，将传入实参统一转换为 NumPy 数组，并构建变量名到数组的映射环境 `env`。随后调用 `lower` 对表达式进行降级并执行对应的本地计算，最终返回计算结果。

练习

- 1) 设计针对向量进行 ReLU 的本地代码，并在 Python 中调用。(难度 **)
- 2) 调研与思考：if 和 for 循环怎么通过装饰器 DSL 实现？(难度 ***)
- 3) 进阶：在第一节课中设计的机器学习程序中，哪些计算密集部分适合使用本地代码实现？尝试对其进行改造，并对比改造前后的性能差异。(难度 ***)

4 Python 多线程

本章学习目标：

- *** 理解并发程序和并程序的区别
- *** 理解 Python 多线程的基本执行模型和性能瓶颈
- *** 理解基于本地代码进行并行优化的方法

4.1 并发和并行

4.1.1 基本概念

在现代计算系统中，并发（Concurrency）是提升程序响应性和资源利用率的重要手段。一个并发程序允许多个任务在逻辑上同时推进，而不要求它们在物理时间上真正同时执行。需要区分的是，并发并不等同于并行（Parallelism）。

- **并发**：多个任务在时间上交错执行，强调程序结构
- **并行**：多个任务在多个处理器或核心上同时执行，强调硬件能力

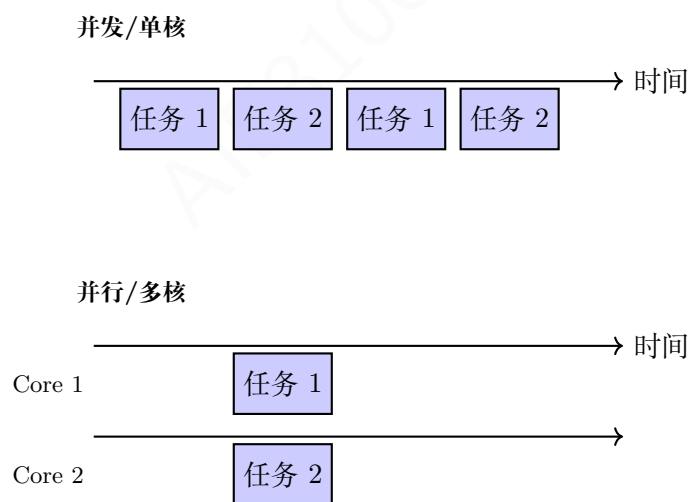


图 4.1: 并发与并行的区别示意图

如图 4.1所示，在单核处理器上，操作系统可以通过时间片轮转的方式调度多个线程，使它们在逻辑上并发执行；而在多核处理器上，多个线程可能真正同时运行。并行可以看做是并发的一种特殊形式。

4.1.2 数据依赖与并发安全

并发的关键在于保证任务在任意切换时程序语义仍然正确。换言之，当在不同任务之间进行调度切换时，程序的最终执行结果不应依赖于具体的调度顺序。当多个任务共享同一份数据，并且至少有一个任务对该数据进行写操作时，就产生了数据依赖。如果这种依赖关系未被正确处理，不同的任务切换顺序可能导致不同的执行结果，从而破坏程序的确定性。

```

1 # 全局共享变量，多个并发任务都会访问和修改
2 def task():
3     global counter
4     for _ in range(N):          # 第一步：从共享变量 counter 中读取当前值
5         tmp = counter
6
7         # 此处若发生任务切换，另一个任务可能也读取到相同的 counter，并更新其值
8
9         # 第二步：基于旧值进行计算并写回，覆盖另一个任务的更新结果
10        counter = tmp + 1
11        # 该“读-算-写”过程不是原子操作，可能导致更新丢失

```

代码 4.1: 累加操作

代码 4.1 给出了一个存在数据依赖的并发程序示例。假设图 4.1 中的线程任务 1 与任务 2 同时执行该函数，并由系统在运行过程中进行时间片轮转调度。在该示例中，变量 `counter` 是一个全局共享变量，任务 1 与任务 2 都会对其进行读写操作。一次看似简单的累加语句在语义上可被拆解为三个步骤：读取当前值、基于旧值计算新值、将结果写回共享变量。由于这一“读-算-写”过程并非原子操作，系统可能在任意两个步骤之间切换正在执行的任务。

例如，任务 1 在完成对 `counter` 的读取后尚未写回新值时被切换，任务 2 也可能读取到相同的旧值并完成写回操作。随后任务 1 恢复执行并写回其计算结果，从而覆盖任务 2 的更新。这种由于任务切换顺序不同而导致更新丢失的现象，正是并发程序中典型的数据竞争问题。因此，该示例清楚地表明：当并发任务之间存在未受控的共享可变状态时，程序的最终结果将依赖于具体的调度顺序，从而破坏程序行为的确定性。

代码 4.1 中，`for` 循环体内的代码即使合并为一条 `counter += 1`，仍然存在数据竞争隐患。如代码 4.2 的字节码所示，该语句在底层会被分解为“读-算-写”三个步骤：其中，`LOAD_GLOBAL counter` 对应“读”操作，`BINARY_ADD` 完成“算”操作，而 `STORE_GLOBAL counter` 则对应“写”操作。由于这三个步骤并非原子执行，在并发环境下可能被其他线程打断，从而引发数据竞争问题。

```

1      0 LOAD_GLOBAL          0 (range)      # 读取全局变量 range
2      2 LOAD_CONST          1 (10000)     # 常量 10000 入栈
3      4 CALL_FUNCTION        1             # 调用 range(10000)
4      6 GET_ITER             # 获取迭代器
5      8 FOR_ITER             6 (to 22)     # for 循环迭代，取出下一个元素；退出循环则跳转到 22
6     10 STORE_FAST          0 (_)         # 将当前迭代值存入局部变量 _
7
8     12 LOAD_GLOBAL          1 (counter)    # 【读】读取全局变量 counter 的当前值
9     14 LOAD_CONST          2 (1)         # 常量 1 入栈
10    16 BINARY_ADD           # 【算】执行 counter + 1
11    18 STORE_GLOBAL         1 (counter)    # 【写】将结果写回 counter
12
13    20 JUMP_ABSOLUTE        4 (to 8)      # 跳回循环开始
14    22 LOAD_CONST          0 (None)      # 加载 None
15    24 RETURN_VALUE         # 返回

```

代码 4.2: `task` 函数内 `for` 循环改为 `counter += 1` 后的字节码

为了安全地实现并发，程序必须对共享数据的访问进行显式控制，其中最常见的手段是使用锁 (Lock) 等同步机制。锁通过在临界区执行期间排斥性地授予对共享资源的访问权限，保证任意时刻只有一个任务能够执行涉及共享可变状态的操作，从而避免数据竞争的发生。在前述代码示例中，若对变量 `counter` 的

“读-算-写”过程加以锁保护，即可将其视为一个不可分割的原子操作。这样，无论操作系统如何在任务 1 与任务 2 之间切换，都不会出现更新丢失的问题，程序的最终结果也将不再依赖于具体的调度顺序。

4.1.3 数据依赖与并行性

数据依赖不仅影响并发程序的正确性，也直接决定了程序能够利用并行计算资源的程度。当一个计算过程中的某一步依赖于前一步的结果时，该过程在逻辑上就必须顺序执行，从而限制了其并行性；反之，若各个计算步骤之间相互独立，则可以安全地并行执行。

代码 4.3 所示的向量点积计算是典型的存在数据依赖的示例。在该实现中，每次循环迭代都会读取并更新变量 `result`，且其当前值依赖于此前所有迭代的累积结果。此类循环携带依赖使得循环迭代之间无法相互独立执行，因此难以直接并行化；否则将破坏计算结果的正确性。

```
1 def dot_product(a, b):
2     result = 0
3     for i in range(len(a)):
4         # result 的当前值依赖于前一次迭代的计算结果
5         result += a[i] * b[i]
6     return result
```

代码 4.3: 点积运算存在数据依赖关系

相比之下，代码 4.4 所示的逐元素相乘函数，其中每个元素的计算彼此独立。在该实现中，不同迭代之间不存在共享可变状态，也不存在跨迭代的数据依赖。因此，每一次循环迭代都可以被视为一个独立的任务，可以在多核处理器上并行执行。

```
1 def vector_multiply(a, b):
2     result = [0] * len(a)
3     for i in range(len(a)):
4         # 每个位置的计算仅依赖 a[i] 和 b[i]
5         result[i] = a[i] * b[i]
6     return result
```

代码 4.4: 逐元素相乘示例

上述对比表明，并行性的关键并不在于是否使用循环或计算量大小，而在于是否存在未消除的数据依赖。通过消除共享状态、重构计算结构等方法，许多原本顺序的算法都可以被改写为高效且安全的并行形式。

4.2 Python 中的并发机制

在并发程序中，常见的执行单元包括进程、线程和协程，Python 也不例外。其主要关系如图 4.2所示。

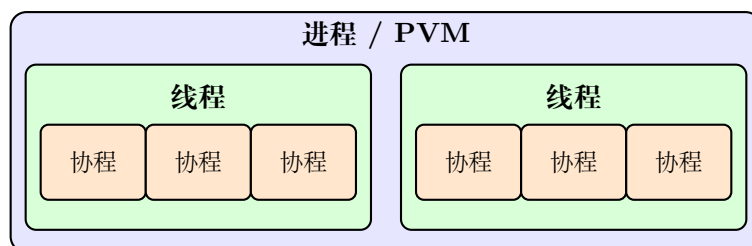


图 4.2: 进程、线程与协程的层次关系示意图

- **进程 (Process)** 是操作系统进行资源分配和调度的基本单位。不同进程之间拥有独立的地址空间，彼此隔离，通信需要借助进程间通信机制 (IPC)。在常见实现中，一个 PVM 往往以单个操作系统进程作为承载单元。
- **线程 (Thread)** 是进程内的执行单元，是操作系统调度的基本执行实体。同一进程中的多个线程共享地址空间和大部分资源，但各自拥有独立的执行上下文。线程由操作系统负责调度，线程间通信开销较小，但需要显式处理同步与数据竞争问题。
- **协程 (Coroutine)** 是一种运行在用户态的轻量级执行单元，由程序显式控制调度与切换。协程通常不依赖操作系统线程切换，创建和切换开销极低，适合处理大量并发任务。

本节将简要介绍 Python 中常见的并发机制，并结合其运行时特性说明它们在实际应用中的适用场景。

4.2.1 多线程

多线程通过在同一进程内创建多个执行线程来实现并发。所有线程共享同一地址空间，因此数据共享较为方便，但也更容易引入数据竞争问题。

代码 4.5展示了一段使用两个线程的代码。假设每个线程执行的任务 `task` 是代码 4.1的累加运算，则结果一般等于或小于 $2N$ 。如果出现小于 $2N$ 的情况，则是由于数据竞争导致。

```
1 import threading
2
3 # task: 对共享变量 counter 执行累加操作
4 t1 = threading.Thread(target=task) # 创建第一个线程
5 t2 = threading.Thread(target=task) # 创建第二个线程
6
7 t1.start() # 启动线程 t1
8 t2.start() # 启动线程 t2
9
10 t1.join() # 等待线程 t1 执行结束
11 t2.join() # 等待线程 t2 执行结束
12
13 # 输出最终结果
14 print(counter)
```

代码 4.5: 多线程下的累加操作

Python 的多线程机制在 CPython 实现中无法实现真正意义上的并行计算，其主要原因在于全局解释器锁 (GIL, Global Interpreter Lock) 的存在。GIL 是 CPython 解释器内部采用的一种互斥机制，其核心语义是：在任意时刻，至多只有一个线程能够执行 Python 字节码。因此，Python 的多线程更适用于 I/O 密集型任务，如网络通信、文件读写等，在这些场景下线程可以在等待 I/O 时主动让出执行权，从而提高程序的整体响应性。

GIL 是一种解释器级别的执行互斥机制，而不是语言层面的并发控制原语。结合前面的示例可以看到，即便在存在 GIL 的情况下，多个线程仍然可能在共享变量的“读-算-写”过程中交错执行，从而导致更新丢失等并发错误。在 CPython 中，线程切换可能发生在以下情形：

- 执行一定数量的 Python 字节码指令（时间片耗尽）；
- 执行阻塞操作（如 I/O）；
- 显式调用会让出执行权的函数，如 `time.sleep()`。

```
1 def task():
2     global counter
3     for _ in range(10000):
4         tmp = counter
5         time.sleep(0.00001)
6         counter = tmp + 1
```

代码 4.6: 通过 sleep 触发线程切换

代码 4.6 在累加过程中引入了 `time.sleep()` 调用，该操作恰好位于对共享变量 `counter` 的读操作与写操作之间。在多线程环境下执行 `task` 函数时，`time.sleep()` 会主动释放 GIL，从而触发线程切换，使得其他线程有机会在当前线程完成写回之前介入执行，进而导致数据竞争的发生。

4.2.2 锁

如前所述，尽管 GIL 保证了同一时刻仅有一个线程执行字节码，但它并不能保证多条语句或由多条字节码构成的复合操作具有原子性。对于涉及共享变量读写的代码片段，这类可能被多个线程同时访问的区域称为临界区 (critical section)。若临界区中的操作不是原子执行的，则在执行过程中可能发生线程切换，从而导致数据竞争问题。

为了解决这一问题，Python 提供了显式的同步机制，例如 `threading.Lock`，用于对临界区进行互斥保护，从而确保其中操作的原子性：

```
1 import threading
2 lock = threading.Lock()
3 counter = 0
4
5 def task():
6     global counter
7     for _ in range(10000):
8         with lock: # 加锁，保证更新原子性
9             counter += 1
```

代码 4.7: 使用 Lock 保护共享变量

在上述代码中：

- `lock = threading.Lock()` 创建一个互斥锁；

- `with lock`: 在进入代码块时自动获取锁，在退出时释放锁；
- 任意时刻仅有一个线程可以持有该锁，从而保证临界区内操作的互斥执行；
- 因此，即使线程发生切换，`counter += 1` 也不会被中断，能够正确更新共享变量。

4.2.3 多进程

多进程通过创建多个独立的操作系统进程来实现并发执行。每个进程都拥有独立的 Python 解释器和内存空间，因此不存在 GIL 的限制，能够在多核处理器上实现真正的并行计算。然而，由于进程之间默认不共享内存，进程间的数据交换需要借助管道、队列或共享内存等进程间通信（IPC）机制，其开销相对较高，编程复杂度也更大。因此，多进程通常适用于计算量较大、任务粒度较粗的 CPU 密集型任务。

代码 4.8 展示了一个简单的多进程示例，其中创建了两个子进程分别执行累加任务。需要注意的是，传入子进程的 `counter` 变量在进程间是以副本形式存在的，各个进程对其修改互不影响。因此，即使每个子进程内部均执行了 N 次累加操作，主进程中的 `counter` 仍保持初始值不变，最终输出结果为 0。

```

1 from multiprocessing import Process # 导入多进程模块
2
3 N = 1000
4 counter = 0 # 主进程中的全局变量
5
6 def task(counter):
7     # 注意：这里的 counter 是从主进程拷贝来的副本（值传递）
8     for _ in range(N):
9         counter += 1 # 只修改当前子进程中的局部变量
10
11 if __name__ == "__main__":
12     # 创建两个进程
13     processes = [
14         Process(target=task, args=(counter,))
15         for _ in range(2)
16     ]
17
18     for p in processes:
19         p.start() # 启动子进程（各自拥有独立内存空间）
20
21     for p in processes:
22         p.join() # 等待子进程执行结束
23
24     # 主进程中的 counter 未被修改
25     print("multiprocessing:", counter) # 仍然为 0

```

代码 4.8: 多进程下的累加操作

4.2.4 协程

协程是一种运行在用户态的轻量级执行单元，通常由运行时系统（如 `asyncio`）负责调度。与线程不同，协程的切换是协作式的，只有在显式的挂起点（如 `await`）处才会发生任务切换。因此，协程在单线程中即可实现高并发执行，但无法利用多核实现真正的并行计算。

代码 4.9 展示了一个基于协程的累加示例，其中创建了两个协程任务并发执行。与多线程类似，协程之间共享同一地址空间中的变量 `counter`。在不引入挂起操作的情况下，各协程对 `counter` 的更新按顺

序执行，最终结果为 $2N$ 。然而，如果在累加过程中引入 `await` 操作，则当前协程会在执行过程中主动让出执行权，使得其他协程有机会在“读-改-写”序列尚未完成时介入执行，从而发生执行交错并引发数据竞争。在这种情况下，多个协程可能基于相同的旧值进行更新，导致更新丢失，最终结果可能退化为 N 。

```
1 import asyncio
2
3 N = 1000
4 counter = 0
5
6 async def task():
7     global counter
8     for _ in range(N):
9         tmp = counter
10        # await asyncio.sleep(0) # 主动让出执行权
11        counter = tmp + 1
12
13 async def runner():
14     tasks = [asyncio.create_task(task()) for _ in range(2)]
15     await asyncio.gather(*tasks)
16
17 asyncio.run(runner()) # 启动事件循环并执行协程
18 print("asyncio:", counter)
```

代码 4.9: 协程下的累加操作

4.3 在本地代码中并行

通过前面的讨论可以看到，在 CPython 实现中，多线程由于 GIL 的限制难以实现真正的计算并行。因此，通常需要借助多进程来实现并行计算。然而，进程间的数据交换需要依赖进程间通信机制，其实现相对复杂且开销较高。

本节将以矩阵乘法为例介绍如何在本地代码中实现并行计算。给定矩阵 $A \in \mathbb{R}^{M \times K}$ 和 $B \in \mathbb{R}^{K \times N}$ ，其乘积 $C = A \cdot B \in \mathbb{R}^{M \times N}$ 定义为：

$$C_{ij} = \sum_{k=0}^{K-1} A_{ik} \cdot B_{kj}.$$

在 Python 中，最直接的实现是三重循环：

```
1 def matmul(A, B, M, K, N):
2     C = [[0.0]*N for _ in range(M)]
3     for i in range(M):
4         for j in range(N):
5             s = 0.0
6             for k in range(K):
7                 s += A[i][k] * B[k][j]
8             C[i][j] = s
9     return C
```

代码 4.10: Python 矩阵乘法

4.3.1 矩阵乘法分块思想

在矩阵乘法中， C_{ij} 的计算依赖于 $A_{i,:}$ 和 $B_{:,j}$ ，但不同的 C_{ij} 之间是相互独立的。利用这一特点，可以将矩阵划分为若干子块（block），并以块为单位进行计算，从而提高数据局部性并便于并行化。

具体而言，将矩阵按一定的块大小划分后，矩阵乘法可以表示为块级别的运算：

$$C_{ij}^{(block)} = \sum_k A_{ik}^{(block)} \cdot B_{kj}^{(block)}.$$

其中，每个 C 的子块由对应行块与列块之间的多个子块乘积累加得到，而非单次块乘即可完成。

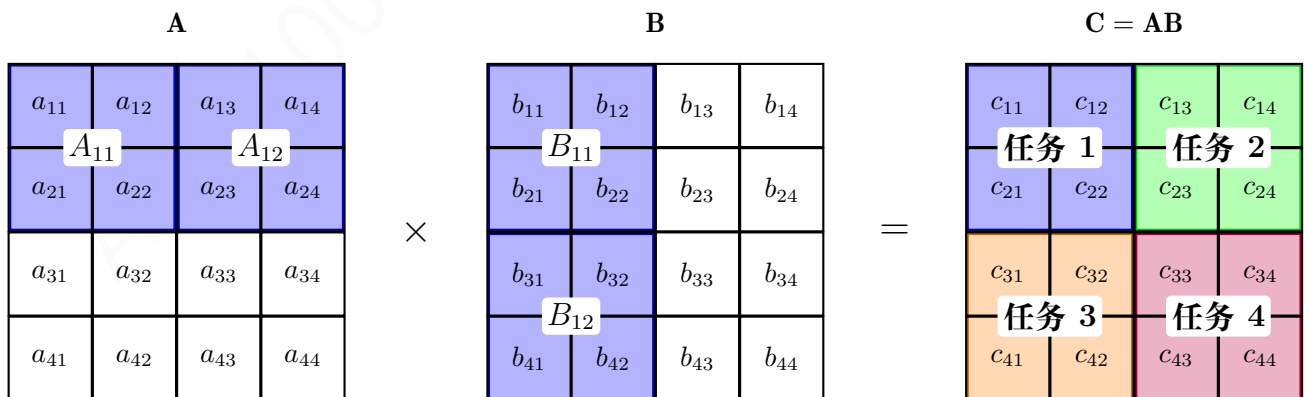


图 4.3: 分块矩阵乘法

如图 4.3所示，矩阵 C 被分为四个子块，每个子块可以独立计算，从而实现并行计算。其中每个子块又可以分解为多个部分进行累加，例如任务 1 对应的 C_{11} 是由两部分计算组成： $A_{11}B_{11}$ 和 $A_{12}B_{21}$ ，在内层循环中依次累加得到最终结果。这种分块加累加的方式既保证了计算正确性，又有利于充分利用缓存和并行化。

4.3.2 分块矩阵乘法 C 实现

下面给出一个最小可运行的 C 语言分块矩阵乘法实现，利用 OpenMP 对外层循环进行多线程并行计算，从而自动加速矩阵运算。OpenMP (Open Multi-Processing) 是一种在 C/C++ 中实现多线程并行计算的标准，它通过简单的编译指令和库函数，就可以让循环或代码块在多个线程上同时执行，提高 CPU 多核利用率。

```
1  #include <stddef.h>
2  #include <omp.h>
3
4  // 分块矩阵乘法函数
5  // A: M×K 矩阵
6  // B: K×N 矩阵
7  // C: M×N 矩阵
8  // 按行优先存储
9  // M, K, N: 矩阵维度
10 // BS: 块大小 (Block Size)
11 void matmul_blocked(const double* A, const double* B, double* C,
12                     int M, int K, int N, int BS) {
13
14     omp_set_num_threads(4); // 手动设置并发线程数
15
16     #pragma omp parallel for collapse(2) schedule(static)
17     // 遍历 C 的块 (输出块)
18     for (int ii = 0; ii < M; ii += BS) { // 行块
19         for (int jj = 0; jj < N; jj += BS) { // 列块
20             for (int kk = 0; kk < K; kk += BS) { // 累加维度块
21
22                 // 处理边界 (防止越界)
23                 int i_max = (ii + BS < M) ? ii + BS : M;
24                 int j_max = (jj + BS < N) ? jj + BS : N;
25                 int k_max = (kk + BS < K) ? kk + BS : K;
26
27                 for (int i = ii; i < i_max; i++) {
28                     for (int j = jj; j < j_max; j++) {
29                         double sum = C[i*N + j];
30                         for (int k = kk; k < k_max; k++) {
31                             sum += A[i*K + k] * B[k*N + j];
32                         }
33                         C[i*N + j] = sum;
34                     }
35                 }
36
37             }
38         }
39     }
40 }
```

代码 4.11: C 分块矩阵乘法

上述代码中，OpenMP 语句的含义如下：

- `omp_set_num_threads(n)`：在代码中手动指定线程数。也可以通过环境变量 `OMP_NUM_THREADS` 设

置, 例如 `export OMP_NUM_THREADS=4`。如果不设置并行度, OpenMP 会自动使用系统默认的线程数, 这通常是 CPU 的物理或逻辑核心数。

- `#pragma omp parallel for`: 表示紧随其后的循环将被多线程并行执行, 每个线程负责循环的一部分迭代。
- `collapse(2)`: 将外层两层循环 (例如 `ii` 和 `jj` 控制的行/列子块循环) 合并成一个大循环, 以便线程可以均匀分配任务, 提高负载均衡。
- `schedule(static)`: 表示迭代按照固定块平均分配给各线程, 适合每次迭代耗时相近的情况, 例如矩阵分块乘法。

编译上述代码时, 需要打开 `-fopenmp` 选项:

```
1 clang -O3 -shared -fPIC -fopenmp $1 -o $2
```

4.3.3 性能对比实验

下面展示如何在 Python 中调用之前 C 语言实现的本地分块矩阵乘法函数 `matmul_blocked`, 将 NumPy 矩阵传入本地函数进行高性能计算, 并将结果返回到 Python。实验中, 我们对比了三种实现的性能:

- 纯 Python 实现: 直接使用嵌套循环计算矩阵乘法, 作为基准。
- 本地 C 分块矩阵乘法 + OpenMP 并行: 利用我们之前实现的 `matmul_blocked` 函数, 通过 OpenMP 并行化外层循环, 实现多线程加速。
- NumPy 矩阵乘法 (BLAS 库): 调用 NumPy 的矩阵乘法操作 (底层使用高性能 BLAS), 作为参考。

通过该实验可以直观地观察不同实现的性能差异, 从而验证本地分块矩阵乘法和多线程加速的效果。

```
1 M, K, N = 256, 256, 256
2 BS = 64
3
4 # -----
5 # Prepare data
6 # -----
7 A_np = np.random.rand(M, K).astype(np.float64)
8 B_np = np.random.rand(K, N).astype(np.float64)
9 C_np = np.zeros((M, N), dtype=np.float64)
10
11 # Python list
12 A_py = A_np.tolist()
13 B_py = B_np.tolist()
14
15 # -----
16 # 1. Pure Python
17 # -----
18 def matmul(A, B, M, K, N):
19     C = [[0.0] * N for _ in range(M)]
20     for i in range(M):
21         for j in range(N):
22             s = 0.0
```

```

23         for k in range(K):
24             s += A[i][k] * B[k][j]
25         C[i][j] = s
26     return C
27 start = time.time()
28 C_py = matmul(A_py, B_py, M, K, N)
29 t_naive = time.time() - start
30 print(f"Python time:      {t_naive:.3f}s")
31
32 # -----
33 # 2. native blocked (C + OpenMP)
34 # -----
35 lib = ctypes.CDLL("./libmatmul.so")
36
37 lib.matmul_blocked.argtypes = [
38     ctypes.POINTER(ctypes.c_double), # A
39     ctypes.POINTER(ctypes.c_double), # B
40     ctypes.POINTER(ctypes.c_double), # C
41     ctypes.c_int, # M
42     ctypes.c_int, # K
43     ctypes.c_int, # N
44     ctypes.c_int, # BS
45 ]
46
47 start = time.time()
48 lib.matmul_blocked(
49     A_np.ctypes.data_as(ctypes.POINTER(ctypes.c_double)),
50     B_np.ctypes.data_as(ctypes.POINTER(ctypes.c_double)),
51     C_np.ctypes.data_as(ctypes.POINTER(ctypes.c_double)),
52     M, K, N, BS,
53 )
54 t_native = time.time() - start
55 print(f"Blocked C (OpenMP): {t_native:.3f}s")
56
57 # -----
58 # 3. NumPy (BLAS)
59 # -----
60 start = time.time()
61 C_ref = A_np @ B_np
62 t_numpy = time.time() - start
63 print(f"NumPy (BLAS):      {t_numpy:.3f}s")

```

代码 4.12: Python 调用本地分块矩阵乘法

实验结果参考如下:

```

1 #: python bench-matmul.py
2 Python time:      1.331s
3 Blocked C (OpenMP):    0.005s
4 NumPy (BLAS):        0.008s

```

代码 4.13: Python 调用本地分块矩阵乘法

练习

- 1) 针对矩阵乘法实验，分析不同参数设置下的优化效果，并总结性能规律。
 - 线程数
 - 矩阵规模
 - 分块大小
- 2) 针对点积运算，实现基于 OpenMP 并行化版本，验证计算正确性，并评估性能提升效果。

Part II

AI 软件框架：PyTorch

5 PyTorch

本章学习目标：

- *** 熟悉 PyTorch 软件栈的基本结构，能够阅读并理解基于 PyTorch 编写的模型代码；
- *** 掌握 CNN 模型的基本原理与实现方法，能够使用 PyTorch 构建并实现 CNN 模型；
- *** 掌握 RNN 模型的基本原理与实现方法，能够使用 PyTorch 构建并实现 RNN 模型。

5.1 PyTorch 软件栈

PyTorch¹ 是一个开源的深度学习框架，由 Facebook AI Research (FAIR) 团队主导开发。其软件栈设计强调灵活性、易用性与高性能，广泛应用于学术研究与工业部署。从底层硬件到高层 API，PyTorch 构建了一个层次清晰、模块解耦的软件体系，使得开发者既能快速原型设计，也能高效部署模型。

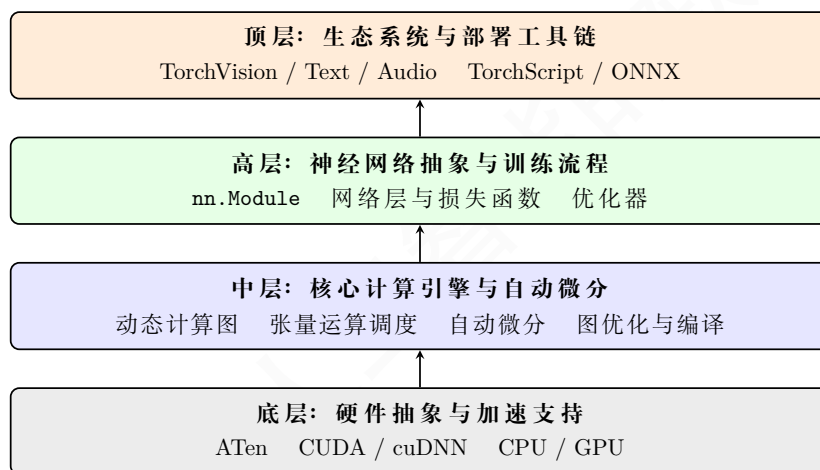


图 5.1: PyTorch 软件栈的分层结构示意图

如图 5.1 所示，整个软件栈可大致分为四个层次：

- **底层：硬件抽象与加速支持。**该层直接面向计算硬件，负责张量存储、内存管理以及并行计算的具体实现。通过统一的后端抽象机制，框架能够在 CPU、GPU 及多种专用加速器上提供一致的张量操作接口，并充分利用诸如 CUDA、cuDNN、cuBLAS 等高度优化的底层库，以实现高性能计算。
 - ATen：提供统一的张量接口与算子调度机制；在 GPU 后端实现中，会调用 cuDNN / cuBLAS 等库来完成具体计算；
 - cuDNN/cuBLAS：构建在 CUDA 之上的高性能算子库，提供针对深度学习与线性代数运算的优化实现，供 ATen 在 GPU 后端调用；
 - CUDA：底层的通用 GPU 并行计算平台，是 cuDNN / cuBLAS 等高性能库的运行基础。
- **中层：核心计算引擎与自动微分系统。**这一层以计算图为核心，将模型计算统一表示为由张量运算构成的计算图结构。在前向传播过程中，计算图按执行顺序动态构建；在反向传播过程中，基于该计算图自动完成梯度计算，从而支持模型训练。

¹PyTorch: <https://pytorch.org>

- 动态计算图：以计算图为核心抽象，在前向传播过程中动态记录张量之间的运算关系，刻画整个计算流程；
- 张量运算：构成计算图的基本单元，各类算子作用于张量并产生新的张量，是前向计算的具体实现；
- 自动微分：基于计算图，通过反向传播机制自动计算各节点的梯度，实现从输出到输入的梯度传递。
- 图优化与编译 (`torch.compile`)：对计算图进行捕获与优化，并生成高效执行代码，以提升模型训练与推理性能；
- **高层：神经网络抽象与训练机制。**该层面向模型构建与训练过程，提供对神经网络结构、损失函数以及参数优化过程的高层抽象，使用户能够以模块化方式定义模型，并便捷地完成训练与评估。
 - 模型基础抽象 (`nn.Module`)：神经网络模型的基础抽象，通常在实现具体网络时通过继承使用，提供统一的参数管理与运行机制，从而支持模型的模块化构建、组织与训练；
 - 网络层与损失函数：提供常用神经网络层（如卷积层、全连接层等）以及损失函数，用于构建模型并定义优化目标；
 - 优化器：根据计算得到的梯度对模型参数进行更新（如 Adam、SGD 等）。
- **顶层：生态系统与部署工具链。**该层提供丰富的领域扩展库以及模型导出与部署工具，支持模型从开发到实际应用的完整流程。
 - 领域扩展库 (`TorchVision` / `TorchText` / `TorchAudio`)：面向计算机视觉、自然语言处理和语音等领域，提供数据集、预处理方法及常用模型；
 - 模型表示与导出 (`TorchScript` / `ONNX`)：支持将模型转换为中间表示或标准交换格式。其中，`TorchScript` 用于将模型转化为可脱离 Python 运行的静态表示，以便在不同运行环境中部署；`ONNX` 用于将模型导出为跨框架的通用格式，从而实现不同深度学习框架之间的模型迁移与部署。

在 PyTorch 中，构建神经网络的核心是继承 `torch.nn.Module` 类并实现其 `forward` 方法。接下来通过两个经典模型讲解如何使用 PyTorch 编写神经网络，包括卷积神经网络 (CNN) 和循环神经网络 (RNN)。

5.2 使用 PyTorch 编写 CNN

卷积神经网络 (Convolutional Neural Network, CNN) 是一类专门用于处理图像等具有网格结构数据的深度神经网络模型。本节首先介绍 CNN 的核心操作-卷积，然后以 LeNet-5 为例讲解如何使用 PyTorch 编写 CNN 模型。

5.2.1 卷积

图像可以表示为一个像素网格，其中灰度图像的每个像素包含一个强度值，而彩色图像的每个像素则包含对应于红、绿、蓝 (RGB) 三个通道的三个数值。卷积操作使用小矩阵（卷积核）在输入图像上滑动，并在每一个位置对覆盖区域内的像素进行加权求和，从而实现局部模式的检测。

形式化地，设输入图像为 I ，卷积核 K 的大小为 $h \times w$ ，则输出特征图 O 定义为：

$$O(i, j) = \sum_{u=0}^{h-1} \sum_{v=0}^{w-1} K(u, v) \cdot I(i+u, j+v).$$

输出中的每一个元素 $O(i, j)$ 都是卷积核与输入图像中对应局部区域之间的点积。卷积核在图像上不断滑动，最终生成输入图像相对该卷积核的特征图。

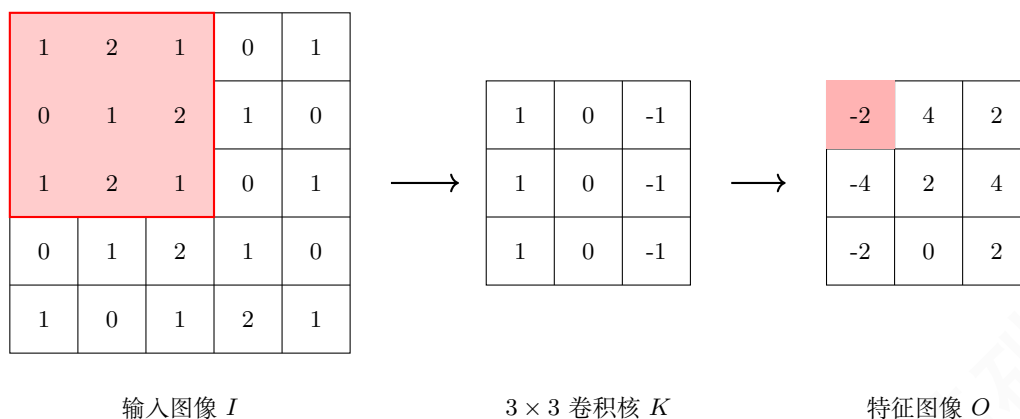


图 5.2: 卷积示例

图 5.2 展示了一个具体示例：一个 5×5 的输入图像与一个 3×3 的卷积核进行卷积运算，最终得到 3×3 的特征图像。

- 左侧输入图像 I 中每个元素为具体的像素值。红色高亮区域表示当前与卷积核进行计算的区域；
- 中间卷积核 K 中每个元素为参数值；
- 右侧特征图 O 中每个元素为卷积核与对应图像区域的点积。红色高亮表示与当前区域对应的结果。

5.2.2 LeNet-5 模型构建

LeNet-5 是由 Yann LeCun 等人在 1998 年提出的经典卷积神经网络，最初用于手写数字识别或 MNIST 数据集。

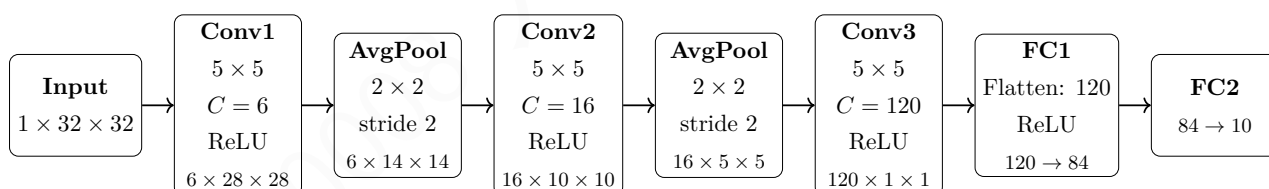


图 5.3: LeNet-5 网络结构

LeNet-5 的主要结构如图 5.3 所示。每层的功能和作用如下：

- **输入层 (Input):** 接受单通道灰度图像，尺寸为 32×32 。
- **卷积层 1 (Conv1):** 使用 5×5 卷积核，将 1 通道映射为 6 个特征通道，并通过 ReLU 激活函数增加非线性表达能力，输出特征图尺寸为 28×28 。
- **池化层 1 (AvgPool):** 使用 2×2 平均池化，步幅为 2，将特征图尺寸从 28×28 缩小到 14×14 。
- **卷积层 2 (Conv2):** 使用 5×5 卷积核，将 6 通道映射到 16 通道，ReLU 激活。输出尺寸为 10×10 。
- **池化层 2 (AvgPool):** 使用 2×2 平均池化，步幅为 2，将尺寸从 10×10 缩小到 5×5 。
- **卷积层 3 (Conv3):** 使用 5×5 卷积核，将 16 通道映射到 120 通道，输出尺寸 1×1 。
- **全连接层 1 (FC1):** 将 Conv3 输出展平为长度 120 的向量，经过 ReLU 激活，映射为 84 维特征。

- **全连接层 2 (FC2):** 将 84 维特征映射到 10 个输出节点，分别对应 10 类分类结果，实现最终分类。

LeNet-5 的设计思想是逐层提取特征：低层卷积捕捉基本边缘和纹理，中层卷积组合局部特征，高层卷积和全连接层整合全局模式。池化层通过下采样提高平移不变性并降低计算量，全连接层将局部特征整合为分类结果。这种结构在手写数字识别和小尺寸图像分类任务中表现优秀。

代码 5.1 展示了基于 PyTorch 的 LeNet-5 实现，主要包括 `__init__` 和 `forward` 两个函数。这两个函数分别定义了网络的层结构和前向传播流程：`__init__` 方法中声明了卷积层 (`conv1`、`conv2`、`conv3`) 和全连接层 (`fc1`、`fc2`)，对应图 5.3 中的各层；而 `forward` 方法则实现了数据从输入层流经各卷积层、池化层和全连接层到输出层的过程，完整体现了图中箭头表示的前向传播路径。

```

1 import torch
2 import torch.nn as nn
3 import torch.nn.functional as F
4
5 class LeNet5(nn.Module):
6     def __init__(self):
7         # 调用父类 (nn.Module) 构造函数
8         super(LeNet5, self).__init__()
9         # 卷积层: 输入 1 通道 -> 输出 6 通道, 卷积核 5*5
10        self.conv1 = nn.Conv2d(1, 6, kernel_size=5)
11        # 卷积层: 输入 6 通道 -> 输出 16 通道, 卷积核 5*5
12        self.conv2 = nn.Conv2d(6, 16, kernel_size=5)
13        # 卷积层: 输入 16 通道 -> 输出 120 通道, 卷积核 5*5
14        self.conv3 = nn.Conv2d(16, 120, kernel_size=5)
15        # 全连接层: 输入 120 -> 输出 84
16        self.fc1 = nn.Linear(120, 84)
17        # 全连接层: 输入 84 -> 输出 10 (分类数量)
18        self.fc2 = nn.Linear(84, 10)
19
20    def forward(self, x):
21        # Conv1 -> ReLU -> AvgPool (2x2)
22        x = F.relu(self.conv1(x))
23        x = F.avg_pool2d(x, 2)
24        # Conv2 -> ReLU -> AvgPool (2x2)
25        x = F.relu(self.conv2(x))
26        x = F.avg_pool2d(x, 2)
27        # Conv3 -> ReLU
28        x = F.relu(self.conv3(x))
29        # 展平操作, (BatchSize, C, H, W) => (BatchSize, C * H * W)
30        x = x.view(x.size(0), -1)
31        # 全连接层 FC1 -> ReLU -> FC2
32        x = F.relu(self.fc1(x))
33        x = self.fc2(x)
34        return x

```

代码 5.1: 使用 PyTorch 搭建 LeNet-5

5.2.3 MNIST 数据集准备

模型训练前的一个步骤是准备数据。这里我们使用 LeNet-5 模型解决手写数字识别问题，因此采用经典的 MNIST (Modified National Institute of Standards and Technology) 数据集。MNIST 包含 70,000 张灰度图像 (60,000 张训练图像和 10,000 张测试图像)，每张图像为 28×28 像素，分为 10 类数字 (0-9)。

在 PyTorch 中, `torchvision.datasets.MNIST` 提供了对 MNIST 的访问接口, 支持在线下载以及本地缓存 (如果此前已下载过数据, 则会直接使用本地文件)。通过 `download=True` 参数, 用户可以控制数据是否自动下载到指定路径。如果是使用自定义数据集, 需要通过 PyTorch 中 `torch.utils.data` 模块的 `Dataset` 抽象实现, 我们在下一章 RNN 的例子中会使用这种方式。

```
1 from torch.utils.data import DataLoader
2 from torchvision import datasets, transforms
3
4 # -----数据预处理方式-----
5 # transforms.Compose 用于将多个图像预处理操作按顺序组合
6 transform = transforms.Compose([
7     transforms.Resize((32, 32)),          # 调整图像尺寸以匹配 LeNet-5 输入
8     transforms.ToTensor(),                # 将 PIL 图像转换为 PyTorch Tensor
9     transforms.Normalize((0.5,), (0.5,))  # 灰度图像标准化, 输出约在 [-1, 1]
10 ])
11
12 # -----数据集定义-----
13 # datasets.MNIST 是 torchvision 提供的 Dataset 实现。
14 train_dataset = datasets.MNIST(
15     root='./data',                        # 数据集本地存放的根目录
16     train=True,                            # 训练集 (60,000 张图像)
17     download=True,                         # 若本地不存在则自动下载
18     transform=transform                   # 样本级预处理流程
19 )
20
21 test_dataset = datasets.MNIST(
22     root='./data',                        # 与训练集共用同一数据根目录
23     train=False,                          # 测试集 (10,000 张图像)
24     download=True,
25     transform=transform
26 )
27
28 # -----数据加载器定义-----
29 # DataLoader 负责将 Dataset 组织成批量数据, 并提供随机打乱等机制。
30 train_loader = DataLoader(
31     train_dataset,
32     batch_size=64,                        # 训练阶段常用较大的 batch size
33     shuffle=True                          # 每个 epoch 开始前打乱样本顺序
34 )
35
36 test_loader = DataLoader(
37     test_dataset,
38     batch_size=4,
39     shuffle=False                         # 测试集不需要打乱顺序
40 )
```

代码 5.2: MNIST 数据集加载

5.2.4 LeNet-5 模型训练

模型训练过程首先对输入数据进行前向传播以产生预测结果，然后通过损失函数度量预测值与真实标签之间的差异，并利用反向传播算法结合自动微分机制计算模型参数的梯度。随后，优化器根据梯度及其优化策略对模型参数进行更新。PyTorch 提供了损失函数、自动微分以及多种优化器的高效实现，用户只需构建模型结构、选择合适的损失函数与优化器，即可通过迭代式训练循环（多个 epoch）不断优化模型参数。

代码 5.3 实现了上述训练流程，并最终将训练得到的模型参数保存至 `lenet5_mnist.pth` 文件中。

```
1 import torch
2 import torch.nn as nn
3 import torch.optim as optim
4
5 # ----- 模型、损失函数和优化器 -----
6 model = LeNet5() # 实例化 LeNet-5 网络模型
7 criterion = nn.CrossEntropyLoss() # 多分类任务常用的交叉熵损失函数
8 optimizer = optim.Adam(model.parameters(), lr=0.001) # 选取 Adam 优化器，学习率为 0.001
9
10 # ----- 训练循环 -----
11 num_epochs = 5
12 for epoch in range(num_epochs):
13     model.train() # 将模型设置为训练模式
14     running_loss = 0.0
15
16     # 遍历 DataLoader 提供的每一个 batch
17     for batch_idx, (inputs, labels) in enumerate(train_loader):
18         optimizer.zero_grad() # 清空上一轮迭代的梯度
19         outputs = model(inputs) # 前向传播，计算模型输出
20         loss = criterion(outputs, labels) # 计算预测结果与真实标签之间的损失
21         loss.backward() # 反向传播，计算梯度
22         optimizer.step() # 根据梯度更新模型参数
23
24     running_loss += loss.item()
25     if batch_idx % 100 == 99: # 每处理 100 个 batch 输出一次平均损失
26         print(
27             f'Epoch [{epoch+1}/{num_epochs}], '
28             f'Batch [{batch_idx+1}], '
29             f'Loss: {running_loss/100:.4f}'
30         )
31     running_loss = 0.0
32
33 print("训练完成！")
34 torch.save(model.state_dict(), "lenet5_mnist.pth") # 保存模型参数
```

代码 5.3: LeNet-5 训练示例

PyTorch 的模型包括 `train` 和 `eval` 两种模式，分别用于训练阶段和推理阶段。在 `train` 模式下，模型中的某些层（如 Dropout 和 Batch Normalization）会启用训练行为；而在 `eval` 模式下，这些层会切换为推理行为，从而保证模型输出的稳定性。在本实验所实现的 LeNet-5 模型中，由于网络结构未包含 Dropout 或 Batch Normalization 等在训练与推理阶段行为不同的层，因此 `model.train()` 与 `model.eval()` 在功能上无显著差异。

5.2.5 LeNet-5 模型测试

在模型效果测试环节，我们首先加载训练好的模型参数，然后使用前面加载好的 MNIST 测试集验证模型效果。代码 5.4 展示了具体实现过程。

```
1 import torch
2
3 # ----- 创建模型 -----
4 model = LeNet5() # 构造模型结构
5 model.load_state_dict(torch.load("lenet5_mnist.pth")) # 加载模型参数
6 model.eval() # 设置为评估模式，关闭 dropout/batchnorm 等训练专用操作
7
8 # ----- 前向推理 -----
9 for batch_idx, (input_batch, label_batch) in enumerate(test_loader):
10     output_batch = model(input_batch) # 前向传播
11     predicted_classes = torch.argmax(output_batch, dim=1) # 获取预测类别
12
13     # 输出结果
14     print(f"Batch {batch_idx+1}")
15     print("输出 logits:", output_batch)
16     print("输出形状:", output_batch.shape) # [batch_size, 10]
17     print("预测类别:", predicted_classes)
18     print("真实标签:", label_batch)
19     print("-"*40)
```

代码 5.4: 使用 LeNet-5 对 MNIST 数据集进行前向推理

5.3 使用 PyTorch 编写 RNN

5.3.1 RNN 的基本原理

循环神经网络 (Recurrent Neural Network, RNN) 是一类适合处理序列数据的神经网络。

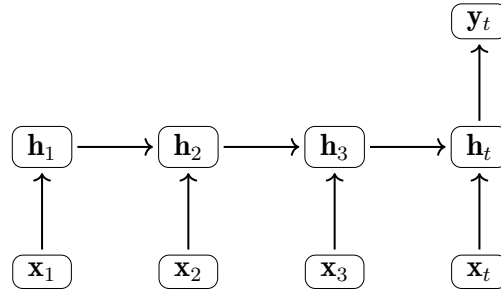


图 5.4: RNN 展开结构示意图

如图 5.4所示, 在每一个时间步 t , RNN 会接收当前输入 x_t 和上一时间步的隐藏状态 h_{t-1} , 计算当前隐藏状态 h_t :

$$h_t = \tanh(W_x x_t + W_h h_{t-1} + b_h),$$

其中:

- $x_t \in \mathbb{R}^d$ 是当前时间步的输入向量;
- $h_t \in \mathbb{R}^h$ 是当前时间步的隐藏状态;
- $W_x \in \mathbb{R}^{h \times d}$ 和 $W_h \in \mathbb{R}^{h \times h}$ 是可训练权重;
- $b_h \in \mathbb{R}^h$ 是偏置项。

RNN 的输出 y_t 通常通过隐藏状态映射得到:

$$y_t = W_{hy} h_t + b_y.$$

RNN 的关键特点是参数共享, 同一组 W_x, W_h, W_y 被重复使用于每个时间步, 这使得 RNN 可以处理任意长度的序列。

5.3.2 RNN 模型构建

代码 5.5 展示了如何使用 PyTorch 搭建一个用于回归的单层 RNN 模型。该模型包括一个 RNN 层和一个输出的全连接层。RNN 层是模型的核心，用于处理序列数据，通过隐藏状态 h_t 在时间维上记忆前面输入的信息，从而捕捉序列的动态规律。输出层将序列最后一个时间步的隐藏状态映射为最终的预测值，适用于下一步预测或回归任务。

```
1 import torch
2 import torch.nn as nn
3
4 # SimpleRNN 是一个单层 RNN 回归模型，用于预测序列的下一步值
5 class SimpleRNN(nn.Module):
6     def __init__(self, input_size=1, hidden_size=32, output_size=1):
7         super(SimpleRNN, self).__init__()
8         self.hidden_size = hidden_size
9
10        # RNN 层
11        self.rnn = nn.RNN(
12            input_size=input_size,
13            hidden_size=hidden_size,
14            batch_first=True      # 表示输入和输出形状为 (batch, seq_len, input_size)
15        )
16
17        # 输出层，将最后一个时间步的 hidden state 映射为预测值
18        self.fc = nn.Linear(hidden_size, output_size)
19
20    def forward(self, x):
21        # x: (batch_size, seq_len, input_size)
22        out, h_n = self.rnn(x)      # out: (batch, seq_len, hidden_size)
23        out = out[:, -1, :]        # 取最后一个时间步的输出
24        out = self.fc(out)         # 输出预测值
25        return out
```

代码 5.5: 使用 PyTorch 搭建 RNN

在 `forward` 方法中，输入张量的形状为 `(batch_size, seq_len, input_size)`，经过 RNN 层得到每个时间步的隐藏状态后，取最后一个时间步的隐藏状态再通过全连接层输出预测结果。参数 `batch_first=True` 指定输入张量的第 0 维为 batch size，第 1 维为序列长度，第 2 维为特征维度，这样可以让训练循环更直观，减少张量维度混淆的可能。该模型结构简单，非常适合像正弦序列这种平滑、规律的时间序列预测任务。

5.3.3 数据集准备

```
1 from torch.utils.data import Dataset, DataLoader
2 import torch
3 import math
4 # =====
5 # 自定义数据集: SineDataset
6 # =====
7 class SineDataset(Dataset):
8     """
9     用于生成正弦序列数据的 Dataset
10    每个样本包含一个长度为 seq_len 的正弦序列 x, 以及下一步预测值 y
11    可以选择为训练集添加噪声, 测试集保持干净
12    """
13    def __init__(self, seq_len=20, num_samples=1000, noise_std=0.0):
14        self.data = [] # 存储输入序列, shape: (seq_len, 1)
15        self.targets = [] # 存储目标值, 即序列下一步, shape: (1,)
16        self.seq_len = seq_len
17        self.noise_std = noise_std # 噪声标准差, 训练集可设置非零, 测试集设为0
18
19        for _ in range(num_samples):
20            start = torch.rand(1).item() * 2 * math.pi # 随机起点: [0, 2] 之间
21
22            # 生成长度为 seq_len 的正弦序列, 并加上高斯噪声
23            x = torch.tensor([
24                math.sin(start + j*0.1) + noise_std * torch.randn(1).item()
25                for j in range(seq_len)
26            ])
27            y = torch.tensor([math.sin(start + seq_len*0.1)]) # 不加噪声, 保持真实值
28
29            self.data.append(x.unsqueeze(-1)) # unsqueeze(-1) 将 x 变为 (seq_len, 1)
30            self.targets.append(y)
31
32    def __len__(self):
33        """返回样本数量"""
34        return len(self.data)
35
36    def __getitem__(self, idx):
37        """返回第 idx 个样本"""
38        return self.data[idx], self.targets[idx]
39
40 # -----数据加载器定义-----
41 seq_len = 20 # 序列长度
42 train_dataset = SineDataset(seq_len=seq_len, num_samples=1000, noise_std=0.02) # 训练集, 加噪声
43 train_loader = DataLoader(train_dataset, batch_size=16, shuffle=True)
44 test_dataset = SineDataset(seq_len=seq_len, num_samples=200, noise_std=0.0) # 测试集, 不加噪声
45 test_loader = DataLoader(test_dataset, batch_size=4, shuffle=False)
```

代码 5.6: 使用 PyTorch 搭建 RNN

5.3.4 模型训练与测试

代码 5.7 展示了如何对我们前面定义的 SimpleRNN 模型进行训练和测试。训练和测试过程使用上一步自定义的 SineDataset 数据集。整体流程与 LeNet-5 的训练和测试类似。

```
1 model = SimpleRNN()
2 criterion = nn.MSELoss()          # 回归任务常用均方误差损失
3 optimizer = optim.Adam(model.parameters(), lr=0.01)
4
5 num_epochs = 10
6 for epoch in range(num_epochs):
7     model.train()                  # 设置模型为训练模式
8     running_loss = 0.0
9
10    for batch_idx, (inputs, targets) in enumerate(train_loader):
11        optimizer.zero_grad()      # 清空上一轮梯度
12        outputs = model(inputs.float()) # 前向传播
13        loss = criterion(outputs, targets.float()) # 计算损失
14        loss.backward()             # 反向传播
15        optimizer.step()            # 参数更新
16
17        running_loss += loss.item()
18
19    # 每个 epoch 输出一次平均损失
20    print(f"Epoch [{epoch+1}/{num_epochs}], Loss: {running_loss/len(train_loader):.4f}")
21
22 print("训练完成！")
23
24 torch.save(model.state_dict(), "simple_rnn_sine.pth") # 保存模型参数
25
26 model.eval() # 切换到评估模式，关闭 dropout / batchnorm 等
27 test_loss = 0.0
28
29 with torch.no_grad(): # 测试阶段不需要计算梯度
30     for batch_idx, (inputs, targets) in enumerate(test_loader):
31         outputs = model(inputs.float()) # 前向传播
32         loss = criterion(outputs, targets.float()) # 计算损失
33         test_loss += loss.item()
34
35 # 输出平均损失
36 print(f"测试集平均损失: {test_loss/len(test_loader):.4f}")
```

代码 5.7: RNN 模型训练和测试

练习

- 1) 相比手写 LeNet、RNN 等神经网络，PyTorch 简化了哪些步骤？
- 2) 分析 PyTorch 中 `nn.Conv2d`、`nn.Linear` 等算子的实现原理。
- 3) 改动 LeNet 的卷积核大小以及通道数，观察模型效果的变化。
- 4) 改动 RNN 的隐藏层维度，观察模型效果的变化。

6 GPU 编程

本章学习目标：

- *** 了解 GPU 的基本特点和 SIMT 执行模型
- *** 掌握 CUDA 编程模型中 `thread`、`block` 与 `grid` 的概念，能阅读简单的 CUDA 程序
- * 了解 NVIDIA GPU 的中间指令表示：PTX 指令集
- *** 掌握在 PyTorch 中使用 GPU 进行计算的基本方法

6.1 了解 GPU

6.1.1 GPU 体系结构

现代图形处理器（Graphics Processing Unit, GPU）最初设计用于加速图形渲染，但因其高度并行的架构，逐渐成为通用并行计算（General-Purpose GPU, GPGPU）的重要平台。与 CPU 强调低延迟、复杂控制流和单线程性能不同，GPU 采用“吞吐量优先”的设计理念，通过集成数千个轻量级计算核心，同时处理大量相似的计算任务。

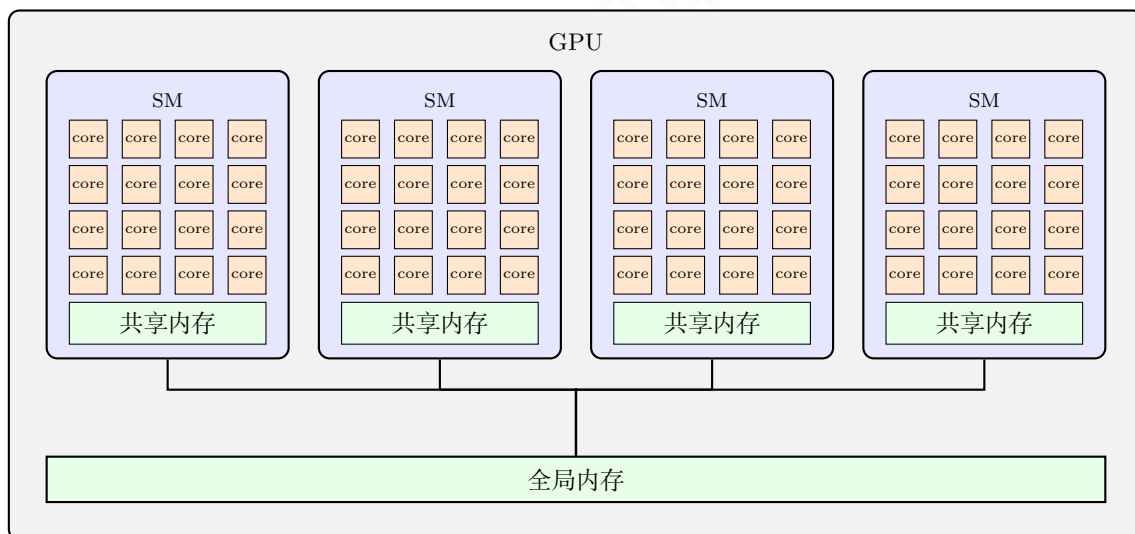


图 6.1: GPU 架构示意图

如图 6.1 所示，NVIDIA GPU 的硬件架构基于流式多处理器（Streaming Multiprocessor, SM）。一个 GPU 通常由多个 SM 组成。每个 SM 内部包含多个 CUDA 核心、共享内存以及线程调度器等硬件资源。CUDA（Compute Unified Device Architecture）是 NVIDIA 提供的 GPU 并行计算平台和编程模型，它允许开发者直接在 GPU 上编写通用计算程序。CUDA 利用 SM 的多核特性，将大量任务分解为多个线程，在硬件上实现大规模数据并行计算。每个 SM 可以同时执行多个线程，提高计算吞吐量。

GPU 的内存层次结构也与 CPU 显著不同，主要包括全局内存和共享内存。全局内存容量大（GB 级），但访问延迟高。共享内存位于 SM 内部，低延迟、高带宽，由线程块内所有线程共享。理解 GPU 的硬件特性对于编写高效 CUDA 程序至关重要。程序员需合理组织线程结构、优化内存访问模式，并充分利用共享内存与计算资源，才能充分发挥 GPU 的并行计算能力。

6.1.2 SIMT 执行模型

SIMT (Single Instruction, Multiple Threads) 是 GPU 的核心执行模型，也是 CUDA 的基本设计理念。在 SIMT 模型下，多个线程共享同一条指令流，但每个线程拥有独立的寄存器和局部状态，可以处理不同的数据。

在 CUDA 编程中，GPU 的并行性并不是通过显式创建线程实现的，而是通过一种分层的抽象执行模型来描述。CUDA 的执行层次结构可以总结为：

Grid $\rightarrow$ Block $\rightarrow$ Warp $\rightarrow$ Thread

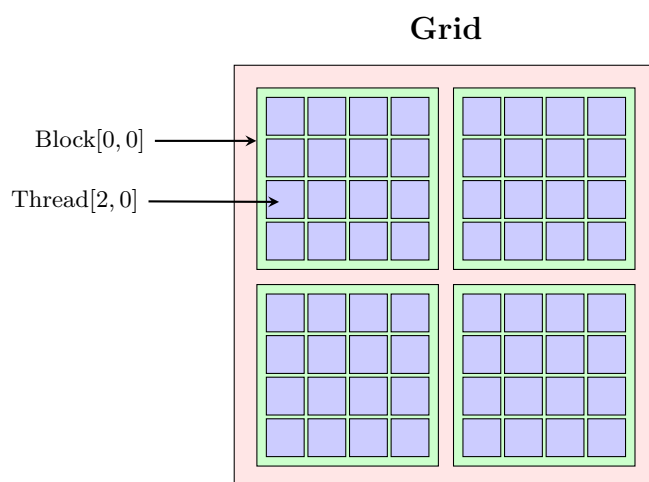


图 6.2: CUDA 层次结构示意图，每个 Grid 含有 2×2 Blocks，每个 Block 含 4×4 Threads

为了直观理解，假设我们在 GPU 上执行一个向量逐元素相乘任务。给定两个长度为 N 的向量 A 和 B ，计算向量 C ，其中

$$C_i = A_i \cdot B_i$$

- **Grid (网格)**: 一次核函数启动所创建的所有 Block 的集合，用于覆盖整个向量计算任务。例如，当向量长度为 $N = 1024$ 时，Grid 需要包含足够多的 Block 来覆盖这 N 个元素。
- **Block (线程块)**: 由多个线程组成的执行单元，负责处理向量的一段连续区间。例如，每个 Block 包含 256 个线程，则一个 Block 可以处理 256 个元素。若 $N = 1024$ ，则需要 $\frac{1024}{256} = 4$ 个 Block。
- **Warp (线程束)**: 硬件调度单位，通常包含 32 个线程。一个包含 256 个线程的 Block 会被划分为 $\frac{256}{32} = 8$ 个 Warp。Warp 内的线程执行同一条乘法指令，但各自作用于不同的向量元素。
- **Thread (线程)**: CUDA 的最小执行单元。在逐元素相乘中，通常每个线程负责计算一个元素： $C_i = A_i \cdot B_i$ 由于各元素之间互不依赖，这种计算模式天然适合大规模并行。

CUDA 的 SM 与 Block 的映射并非一一对应。一个 Block 在执行期间被分配到某个 SM 上 (Block 只驻留在一个 SM)，但一个 SM 可以同时容纳多个 Block，只要硬件资源允许 (如寄存器数量、共享内存容量、最大线程数等)。例如，如果一个 SM 支持最多 2048 个并发线程，而每个 Block 有 512 个线程，则该 SM 最多可同时执行 4 个 Block。这种多对一的调度机制可以通过大量线程隐藏内存访问延迟，提高 GPU 利用率。如果 Block 的线程数超过 SM 的资源限制，就无法完全调度整个 Block，需要调整 Block 大小。

6.2 CUDA C++ 编程

CUDA C++ 编程主要分为两步：编写核函数和调用核函数。

- **编写核函数**：核函数是在 GPU 上运行的函数，每个线程都会独立执行它。
- **调用核函数**：程序员从 CPU 端启动核函数，并指定 Grid 和 Block 的布局。调用核函数会在 GPU 上创建相应数量的线程，每个线程执行核函数中的代码。CPU 还负责准备输入数据、将数据从主机内存传输到 GPU 内存，以及在核函数执行完成后将结果传回主机。

6.2.1 编写核函数

代码 6.1 展示了如何使用 CUDA C++ 编写一个简单的核函数，实现向量的逐元素相乘。该核函数使用 `__global__` 关键字定义，表示该函数在 GPU 上执行，并由 CPU 端发起调用。

```

1  #include <cuda_runtime.h>
2
3  #define N 1024 // 向量长度
4
5  __global__ void ele_mul(float* a, float* b, float* c) {
6      // 计算全局线程索引
7      // blockIdx.x: 当前线程所在 Block 的索引
8      // blockDim.x: 每个 Block 中线程的数量
9      // threadIdx.x: 当前线程在 Block 内的索引
10     int i = blockIdx.x * blockDim.x + threadIdx.x;
11
12     // 边界检查：确保线程索引不超过向量长度
13     if (i < N) {
14         c[i] = a[i] * b[i];
15     }
16 }

```

代码 6.1: 使用 CUDA C++ 编写向量逐元素相乘核函数

在该示例中，我们采用一维 Grid 和一维 Block 的组织方式，将向量计算任务映射到一维线程结构上。可以将整个 Grid 理解为对向量索引空间的划分：每个 Block 负责处理向量中的一段连续区间，而 Block 内的线程则分别对应区间中的具体元素。通过这种分层的索引计算方式，GPU 能够将向量逐元素相乘这一数据并行任务自然地映射到大量线程上，实现高效并行计算。每个线程通过表达式 `blockIdx.x * blockDim.x + threadIdx.x` 计算自己负责的全局向量索引。其中，`blockIdx.x` 表示当前 Block 在 Grid 中的编号，`blockDim.x` 表示每个 Block 中的线程数量，`threadIdx.x` 表示线程在 Block 内的局部编号。三者组合形成“块偏移量 + 块内偏移量”的全局索引计算方式。

由于 Grid 和 Block 的尺寸通常按需向上取整，可能会产生多余线程，因此代码中通过条件判断 `i < N` 进行边界检查，确保线程只访问合法的向量元素，从而避免越界访问。

CUDA 为不同层次的并行结构提供了内建索引变量，用于描述 Grid、Block 和 Thread 之间的层级关系。按照自顶向下的结构，可以理解为：

- **gridDim**：表示网格 (grid) 在三个维度上的尺寸，即整个核函数启动时包含的 block 数量。`gridDim.x`、`gridDim.y`、`gridDim.z` 分别对应 X、Y、Z 方向上的 block 数目。Grid 是一次核函数启动所创建的最大执行单元。
- **blockIdx**：表示当前 block 在整个 grid 中的三维索引，包含 x、y、z 分量，分别表示该 block 在网格 X、Y、Z 方向上的位置。通过 `blockIdx` 可以确定当前 block 负责的数据区域。
- **blockDim**：表示线程块 (block) 在三个维度上的尺寸，即每个 block 中包含的线程数量。`blockDim.x`、`blockDim.y`、`blockDim.z` 分别对应各维度的线程数量，用于描述 block 内部的线程组织方式。

- `threadIdx`: 表示线程在其所属 block 中的三维索引, 包含 x、y、z 分量, 分别表示线程在 block 内 X、Y、Z 方向上的位置。结合 `blockIdx` 和 `blockDim`, 可以计算出线程在整个数据空间中的全局位置。

代码 6.2 展示了一个二维矩阵乘法的例子。每个线程负责计算结果矩阵 C 中的一个元素 $C_{row,col}$ 。线程首先通过如下公式计算自己对应的矩阵坐标:

$$\begin{aligned} \text{row} &= \text{blockIdx.y} \times \text{blockDim.y} + \text{threadIdx.y} \\ \text{col} &= \text{blockIdx.x} \times \text{blockDim.x} + \text{threadIdx.x} \end{aligned}$$

这里, 行索引 `row` 和列索引 `col` 是分别独立计算的: `row` 仅依赖纵向的 `blockIdx.y` 与 `threadIdx.y`, 而 `col` 仅依赖横向的 `blockIdx.x` 与 `threadIdx.x`。这种方式使每个方向的线程分布可以单独控制, 从而灵活地组织二维 Grid 和二维 Block, 实现对矩阵每个元素的精确映射。

在核函数内部, 线程首先进行边界检查, 确保索引不越界, 然后通过循环计算对应的点积, 并将结果写回全局内存。这种二维索引计算方式体现了 GPU 的二维线程组织结构, 同时保证了每个线程独立处理矩阵中的一个元素。

```

1  #define N 1024    // 矩阵维度: N * N
2
3  __global__ void matmul(float* A, float* B, float* C) {
4      // 计算当前线程负责的矩阵坐标
5      int row = blockIdx.y * blockDim.y + threadIdx.y;
6      int col = blockIdx.x * blockDim.x + threadIdx.x;
7
8      // 边界检查: grid/block 的尺寸通常向上取整, 因此可能会产生越界线程
9      if (row < N && col < N) {
10         float sum = 0.0f;
11         for (int k = 0; k < N; k++) {
12             sum += A[row * N + k] * B[k * N + col];
13         }
14         C[row * N + col] = sum;
15     }
16 }

```

代码 6.2: 使用 CUDA C++ 矩阵乘法核函数

6.2.2 调用核函数

主机端程序调用核函数包括完成以下几个步骤：

- 1) 在 CPU (Host) 端分配并初始化数据；
- 2) 在 GPU (Device) 端分配显存，并将数据从主机拷贝到设备；
- 3) 配置 Grid 和 Block 的尺寸，启动核函数；
- 4) 将计算结果从设备拷贝回主机，并释放相关资源。

代码 6.3以向量逐元素相乘为例说明核函数的完整调用流程。

```
1 int main() {
2     size_t size = N * sizeof(float); // 计算向量所需内存
3
4     //-----CPU端内存分配与初始化-----
5     float *ha, *hb, *hc;
6     ha = (float*)malloc(size); // 为向量 A 分配内存
7     hb = (float*)malloc(size); // 为向量 B 分配内存
8     hc = (float*)malloc(size); // 为向量 C 分配内存
9     for (int i = 0; i < N; i++) {
10         ha[i] = 2.0f; // 初始化向量 A
11         hb[i] = 3.0f; // 初始化向量 B
12     }
13
14     //-----GPU端内存分配和数据拷贝-----
15     float *da, *db, *dc;
16     cudaMalloc(&da, size); // 为向量 A 分配内存
17     cudaMalloc(&db, size); // 为向量 B 分配内存
18     cudaMalloc(&dc, size); // 为向量 C 分配内存
19     cudaMemcpy(da, ha, size, cudaMemcpyHostToDevice); // 将向量A从CPU拷贝到GPU
20     cudaMemcpy(db, hb, size, cudaMemcpyHostToDevice); // 将向量B从CPU拷贝到GPU
21
22     //-----配置并启动核函数-----
23     int threadsPerBlock = 256; // 每个 Block 含 256 个线程
24     int numBlocks = (N + threadsPerBlock - 1) / threadsPerBlock; // 计算所需 Block 数 (向上取整)
25
26     // 启动核函数, 通过<<gridDim, blockDim>>>设置核函数参数
27     ele_mul<<numBlocks, threadsPerBlock>>>(da, db, dc);
28
29     //-----结果拷贝回主机, 并释放资源-----
30     cudaMemcpy(hc, dc, size, cudaMemcpyDeviceToHost);
31
32     cudaFree(da);
33     cudaFree(db);
34     cudaFree(dc);
35     free(ha);
36     free(hb);
37     free(hc);
38     return 0;
39 }
```

代码 6.3: 调用 CUDA 向量乘法核函数

代码 6.8 给出二维矩阵乘法的核函数调用方式。相比一维向量计算，矩阵计算采用二维 Grid 与二维 Block 结构，以便更自然地映射到矩阵的行列坐标。

```

1  int main() {
2      // 省略内存分配与初始化步骤...
3
4      // 每个 block 包含 16 * 16 = 256 个线程
5      dim3 threadsPerBlock(16, 16);
6      // ↑ dim3 是 CUDA 提供的三维结构 (x, y, z)
7      // 这里只传了 (16,16)，等价于 (16,16,1)
8      // 表示一个二维线程块 (block)
9
10     // grid 尺寸向上取整，确保覆盖整个矩阵
11     dim3 numBlocks(
12         (N + threadsPerBlock.x - 1) / threadsPerBlock.x,
13         (N + threadsPerBlock.y - 1) / threadsPerBlock.y
14     );
15     // ↑ numBlocks 也是 dim3 类型，表示 grid 的尺寸 (block 的数量)
16     // 这里构造的是二维 grid，用于覆盖 N×N 的矩阵
17
18
19     //-----启动 GPU 核函数-----
20     matmul<<<numBlocks, threadsPerBlock>>>(dA, dB, dC);
21
22     // 省略后续步骤...
23 }

```

代码 6.4: 调用矩阵乘法核函数

6.2.3 Tile 编程

在 GPU 编程中，访存性能往往成为限制程序性能的主要瓶颈。传统的矩阵乘法实现中，每一次乘加操作都需要从全局内存读取数据，而全局内存具有较高的访问延迟，导致计算单元无法被充分利用。为了解决这一问题，CUDA 中常采用 Tile（分块）优化技术，通过引入共享内存来提高数据复用率，从而显著提升性能。

Tile 编程的核心思想是将大规模矩阵划分为若干个小块，每个线程块（block）负责计算输出矩阵中的一个子块。在计算过程中，线程块中的所有线程协同工作，将输入数据的一部分数据加载到共享内存中。由于共享内存具有远低于全局内存的访问延迟，这些数据可以被线程块内的多个线程重复使用，从而减少对全局内存的访问次数。

代码 6.5 展示了一种分块矩阵乘法的核函数实现，每个线程首先根据其在线程块和网格中的索引，确定其负责计算的输出矩阵元素 $C[row][col]$ 。随后，通过外层循环遍历所有 tile。在每一次迭代中，线程块加载矩阵 A 的一行子块和矩阵 B 的一列子块到共享内存数组 As 和 Bs 中。加载完成后，通过调用 `__syncthreads()` 进行线程同步，确保所有线程都完成数据加载后再进行后续计算。

在每个 tile 内部，线程通过一个长度为 `TILE_SIZE` 的循环，计算对应子块的部分乘积，即：

$$C[row][col] += \sum_{k=t \cdot TILE_SIZE}^{(t+1) \cdot TILE_SIZE - 1} A[row][k] \cdot B[k][col]$$

经过所有 tile 的累加后，即可得到最终的矩阵乘法结果。最后，线程将计算结果写回全局内存。相比朴素实现，Tile 优化通常可以带来数量级的性能提升，是 GPU 矩阵计算中的经典优化方法之一。

```

1 __global__ void matmul_tiled(float* A, float* B, float* C, int N) {
2     // 定义block共享内存, 每个block加载 A 和 B 的一个 TILE (子块)
3     __shared__ float As[TILE_SIZE][TILE_SIZE]; // A 的子块, TILE_SIZE 为常量
4     __shared__ float Bs[TILE_SIZE][TILE_SIZE]; // B 的子块, TILE_SIZE 为常量
5
6     // -----计算当前线程负责的 C 中元素位置-----
7     int row = blockIdx.y * TILE_SIZE + threadIdx.y;
8     int col = blockIdx.x * TILE_SIZE + threadIdx.x;
9
10    float value = 0.0f; // 用于保存C[row][col]的结果
11
12    // -----遍历所有 tile-----
13    for (int t = 0; t < (N + TILE_SIZE - 1) / TILE_SIZE; t++) {
14        // -----加载 A 的一个 tile 到共享内存-----
15        if (row < N && t * TILE_SIZE + threadIdx.x < N)
16            As[threadIdx.y][threadIdx.x] = A[row * N + t * TILE_SIZE + threadIdx.x];
17        else
18            As[threadIdx.y][threadIdx.x] = 0.0f;
19
20        // -----加载 B 的一个 tile 到共享内存-----
21        if (col < N && t * TILE_SIZE + threadIdx.y < N)
22            Bs[threadIdx.y][threadIdx.x] = B[(t * TILE_SIZE + threadIdx.y) * N + col];
23        else
24            Bs[threadIdx.y][threadIdx.x] = 0.0f;
25
26        __syncthreads(); // 同步线程, 确保整个 tile 已经加载完成
27
28        // -----计算当前 tile 的部分乘积-----
29        for (int k = 0; k < TILE_SIZE; k++) {
30            value += As[threadIdx.y][k] * Bs[k][threadIdx.x];
31        }
32
33        __syncthreads(); // 再次同步, 确保整个 tile 已经计算完成
34    }
35
36    // -----写回结果到全局内存-----
37    if (row < N && col < N) {
38        C[row * N + col] = value;
39    }
40 }

```

代码 6.5: 矩阵乘法性能实验

6.2.4 GPU 程序的编译过程

CUDA 程序通常使用 NVCC 编译器进行编译。首先, CUDA C/C++ 源代码 (.cu 文件) 会被编译为 PTX (Parallel Thread Execution)。PTX 是一种面向 NVIDIA GPU 的中间表示语言, 类似于汇编语言, 但具有一定的可移植性, 可以视为 GPU 的“虚拟指令集”。随后, PTX 会进一步被编译为特定 GPU 架构对应的机器码 (SASS)。如果目标 GPU 与编译时生成的机器码不兼容, 则可以使用保留在二进制中的 PTX, 在程序运行时由 GPU 驱动进行 JIT 编译, 以生成适配当前 GPU 的机器码。

下面展示了向量逐元素相乘核函数经 NVCC 编译器生成的 PTX 代码。该核函数接收三个浮点数组指针作为参数, 在边界检查后执行对应元素的乘法并写回结果。

```
1      // ---- 核函数参数: 三个指针 a, b, c ----
2      ld.param.u64    %rd1, [_Z7ele_mulPfS_S__param_0]; // a 指针
3      ld.param.u64    %rd2, [_Z7ele_mulPfS_S__param_1]; // b 指针
4      ld.param.u64    %rd3, [_Z7ele_mulPfS_S__param_2]; // c 指针
5
6      // ---- 线程层次索引 ----
7      mov.u32         %r2, %ctaid.x;    // blockIdx.x
8      mov.u32         %r3, %ntid.x;     // blockDim.x
9      mov.u32         %r4, %tid.x;      // threadIdx.x
10
11     // ---- 计算全局线程索引 tid = blockIdx.x * blockDim.x + threadIdx.x ----
12     mad.lo.s32       %r1, %r2, %r3, %r4;
13
14     // ---- 边界检查: if (tid > 1023) ----
15     setp.gt.s32      %p1, %r1, 1023;
16     @%p1 bra         $L__BB0_2;        // 越界线程直接返回
17
18     // ---- 计算 a[tid] 的地址 ----
19     cvta.to.global.u64 %rd4, %rd1;    // a 转为全局地址
20     mul.wide.s32      %rd5, %r1, 4;    // tid * sizeof(float)
21     add.s64           %rd6, %rd4, %rd5;
22
23     // ---- 计算 b[tid] 的地址 ----
24     cvta.to.global.u64 %rd7, %rd2;
25     add.s64           %rd8, %rd7, %rd5;
26
27     // ---- 加载 a[tid], b[tid] ----
28     ld.global.f32     %f1, [%rd8];    // b[tid]
29     ld.global.f32     %f2, [%rd6];    // a[tid]
30
31     // ---- 向量逐元素相乘 ----
32     mul.f32           %f3, %f2, %f1;  // a[tid] * b[tid]
33
34     // ---- 计算 c[tid] 的地址并写回 ----
35     cvta.to.global.u64 %rd9, %rd3;
36     add.s64           %rd10, %rd9, %rd5;
37     st.global.f32     [%rd10], %f3;    // c[tid] = result
38
39 $L__BB0_2:
40     ret;                // 结束
```

代码 6.6: 代码 6.1 向量逐元素相乘核函数对应的 PTX 指令

6.3 实验与应用

6.3.1 实验环境：在 Kaggle 平台调试 GPU 程序

Kaggle¹ 是一个免费的云端数据科学和深度学习平台，提供预装的 PyTorch 等深度学习框架，以及可直接使用的 GPU/TPU 计算资源，非常适合进行 CUDA 程序的实验与调试。

Kaggle 的主要开发环境是 Notebook，其基本组成如下：

- **Notebook**：一个完整的工作环境，用于组织代码、文本以及实验结果。
- **Cell**：Notebook 的基本执行单元，每个 Cell 可以独立运行。根据功能不同，主要分为：
 - **Code Cell**：用于编写和执行代码或命令；
 - **Markdown Cell**：用于编写说明文档。

Kaggle 平台对 GPU 资源的使用存在一定的额度限制。在使用 GPU 之前，需要在 Notebook 设置中手动开启，将 Accelerator 设置为 GPU。Code Cell 中的常用命令如下：

```
1 # 验证 GPU 是否可用
2 !nvidia-smi
3
4 # Cell 1: 创建 CUDA 代码文件
5 %%writefile ele_mul.cu
6 # 此处填写程序源代码
7
8 # Cell 2: 编译程序
9 !nvcc ele_mul.cu -o ele_mul
10
11 # Cell 3: 运行程序
12 !./ele_mul
13
14 # Cell 4: 查看PTX代码
15 !nvcc -ptx ele_mul.cu -o ele_mul.ptx
```

代码 6.7: Code Cell 中编译和运行 CUDA 程序的常用命令

需要注意的是，`%%writefile` 与 `!nvcc` 等命令必须位于不同的 Cell 中，否则可能会导致错误。

¹<https://www.kaggle.com/>

6.3.2 矩阵乘法性能实验

```
1 void matmul_cpu(float* A, float* B, float* C, int N) { // 普通CPU矩阵乘法 }
2 __global__ void matmul_naive(float* A, float* B, float* C, int N) { // 普通GPU矩阵乘法 }
3 template <int TILE_SIZE>
4 __global__ void matmul_tiled(float* A, float* B, float* C, int N) { // 分块矩阵乘法 }
5
6 int main() {
7     // 省略若干代码
8     // -----1: CPU 矩阵乘法测试-----
9     double t1 = cpu_time();
10    matmul_cpu(hA, hB, hC, N);
11    double t2 = cpu_time();
12    printf("CPU time: %f ms\n", t2 - t1);
13
14    // -----2: GPU 矩阵乘法测试-----
15    dim3 threads(16, 16);
16    dim3 blocks((N + 15) / 16, (N + 15) / 16);
17    cudaEventRecord(start);
18    matmul_naive<<<blocks, threads>>>(dA, dB, dC, N);
19    cudaEventRecord(stop);
20    cudaEventSynchronize(stop);
21    float time_naive;
22    cudaEventElapsedTime(&time_naive, start, stop);
23    printf("GPU Naive: %f ms\n", time_naive);
24
25    // -----3: GPU 分块矩阵乘法 (多种 TILE_SIZE 配置) 测试-----
26    #define RUN_TILE(T) { \
27        dim3 tpb(T, T); \
28        dim3 nb((N + T - 1) / T, (N + T - 1) / T); \
29        cudaEventRecord(start); \
30        matmul_tiled<T><<<nb, tpb>>>(dA, dB, dC, N); \
31        cudaEventRecord(stop); \
32        cudaEventSynchronize(stop); \
33        float t_ms; \
34        cudaEventElapsedTime(&t_ms, start, stop); \
35        printf("GPU Tiled (%d): %f ms\n", T, t_ms); \
36    }
37
38    RUN_TILE(8);
39    RUN_TILE(32);
40    // 省略若干代码
41 }
```

代码 6.8: 矩阵乘法性能实验

实验结果参考如下:

```
1 CPU time: 3682.179199 ms
2 GPU Naive: 1.731328 ms
3 GPU Tiled (8): 1.726048 ms
4 GPU Tiled (32): 1.040832 ms
```

代码 6.9: 矩阵乘法实验结果

6.3.3 在 PyTorch 中使用 GPU

在 PyTorch 中，GPU 计算主要依赖于 CUDA 来加速张量运算和神经网络训练。通过将模型和数据移动到 GPU，可以显著提高训练速度。

上一章的 LeNet-5 训练代码经过修改以支持 GPU，如代码 6.10 所示。代码训练逻辑与 CPU 版本一致，关键改动包括以下三点：

- **检测 GPU 设备**：使用 `torch.device("cuda" if torch.cuda.is_available() else "cpu")` 自动选择可用的 GPU 或回退到 CPU。
- **模型移动到 GPU**：通过 `model.to(device)` 将模型参数移动到 GPU，以便后续计算在 GPU 上进行。
- **数据移动到 GPU**：在每个训练批次中，将输入数据 `inputs` 和标签 `labels` 移动到 GPU，确保前向和反向传播在 GPU 上执行。

```
1 # ----- 检测 GPU 设备 -----
2 device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
3 print(f"Using device: {device}")
4
5 # ----- 实例化 LeNet-5 并移动到 GPU -----
6 model = LeNet5().to(device)
7 criterion = nn.CrossEntropyLoss()
8 optimizer = optim.Adam(model.parameters(), lr=0.001)
9
10 num_epochs = 5
11 for epoch in range(num_epochs):
12     model.train()
13     running_loss = 0.0
14
15     for batch_idx, (inputs, labels) in enumerate(train_loader):
16         # ----- 数据移动到 GPU -----
17         inputs = inputs.to(device)
18         labels = labels.to(device)
19
20         # 省略训练代码...
21
22 print("训练完成！")
23 torch.save(model.state_dict(), "lenet5_mnist.pth") # 保存模型参数到文件
```

代码 6.10: 基于 GPU 训练 LeNet-5

练习

- 1) 分析矩阵乘法实验中不同分块大小对性能的影响并探索原因。
- 2) 将上节课的 RNN 代码示例修改为 GPU 版本。

7 算子开发

本章学习目标：

- *** 使用 Triton 编写自定义 GPU 算子；
- *** 了解 Triton 的自动参数调优机制；
- *** 可以将 Triton 算子集成到 PyTorch 模型。

7.1 算子开发

算子开发是指针对深度学习模型中的基本计算单元（例如矩阵乘法、卷积等），在特定硬件平台上进行高性能实现与优化的过程。尽管直接使用 CUDA C++ 编写算子具有极高的灵活性，但其实现过程往往较为复杂。开发者需要手动设计线程块（block）与线程（thread）的划分策略，显式管理全局内存与共享内存之间的数据搬运，同时还需处理同步与并发问题，并通过反复实验进行性能调优。这种底层优化方式对开发者的硬件理解能力和并行编程经验提出了较高要求。

为降低算子开发的门槛，近年来出现了一类基于领域特定语言（Domain-Specific Language, DSL）的 AI 编译器系统。这类系统允许开发者使用更高层次的抽象来描述计算逻辑，由编译器自动完成并行化映射、调度优化以及目标代码生成，从而显著提升开发效率与代码可移植性。典型代表包括 OpenAI 的 Triton¹和 nVidia 公司新出的 cuTile²。这两款软件都是开源软件，可以与 PyTorch 无缝集成。

本章内容以 Triton 为例，详细讲解算子开发的过程。

7.2 Triton 算子开发

Triton 是一种嵌入在 Python 中的领域特定语言（DSL），专门用于高性能 GPU 算子的开发。开发流程上，用户通过 `@triton.jit` 装饰器定义 Kernel，随后由编译器自动生成优化的 CUDA/PTX 代码，从而避免手写底层 CUDA 的复杂性。在编程模型上，Triton 与 CUDA 存在一定的对应关系，但整体抽象更加简洁：

- **Program Instance 与并行映射**：Triton 以 program instance 作为基本执行单元（类似 CUDA 的 thread block），每个 instance 处理一块数据（tile）。通过 `tl.program_id(dim)` 获取其在 grid 中的位置（类似 `blockIdx`）并计算数据偏移；线程级并行由向量化操作隐式表达，无需显式管理 `threadIdx` 或线程布局。
- **掩码（Mask）机制**：通过掩码控制访存有效性，自动避免越界访问，替代 CUDA 中常见的边界判断，如 `if (idx < N)`。
- **高效内存访问与广播**：内建矢量化 `load/store` 和广播语义，便于表达连续访存和数据复用，从而提升内存带宽利用率。

总体而言，Triton 在保留 CUDA 核心执行模型的基础上，通过更高层的抽象，简化了并行映射与索引计算，同时仍能生成高性能代码。

¹<https://github.com/openai/triton>

²<https://github.com/nvidia/cutile-python>

7.2.1 编写算子

本节先以向量逐元素相乘为例，演示 Triton 算子开发方法，然后将其扩展到矩阵乘法以及分块矩阵乘法。

逐元素相乘 代码 7.1 使用 Triton 实现了向量的逐元素相乘算子，其关键步骤如下：

- 使用 `@triton.jit` 装饰器将 `vecmul_kernel` 标记为 GPU Kernel，由编译器生成高效的底层实现。
- 参数 `BLOCK_SIZE`: `tl.constexpr` 表示该值在编译期确定，使编译器能够进行循环展开和访存优化。
- 在 Kernel 内部，`tl.program_id(0)` 获取当前 program instance 在第 0 维网格中的编号，其作用类似于 CUDA 中的 `blockIdx.x`。每个 program instance 负责处理一段连续的数据，其全局索引通过如下方式计算：
$$\text{offsets} = \text{pid} \times \text{BLOCK_SIZE} + \text{tl.arange}(0, \text{BLOCK_SIZE})$$
- 基于索引构造掩码 `mask = offsets < N`，用于避免越界访问。当掩码为 `False` 时，对应位置不会发生实际内存操作，从而避免显式分支判断。
- 基于 `offsets` 与 `mask`，通过 `tl.load` 和 `tl.store` 对数据进行批量读取与写回，实现向量化访存。

```
1 import triton
2 import triton.language as tl
3
4 @triton.jit
5 def vecmul_kernel(
6     x_ptr, y_ptr, c_ptr,
7     N,                # 元素总数
8     BLOCK_SIZE: tl.constexpr # 每个 block 处理的元素数
9 ):
10     # 获取当前 block 的 ID (相当于 blockIdx.x)
11     pid = tl.program_id(0)
12
13     # 计算当前 block 负责的全局偏移
14     offsets = pid * BLOCK_SIZE + tl.arange(0, BLOCK_SIZE)
15
16     # 构建掩码，防止越界访问
17     mask = offsets < N
18
19     # 加载数据 (自动处理内存合并)
20     x = tl.load(x_ptr + offsets, mask=mask)
21     y = tl.load(y_ptr + offsets, mask=mask)
22
23     # 执行逐元素乘法
24     output = x * y
25
26     # 写回结果
27     tl.store(c_ptr + offsets, output, mask=mask)
```

代码 7.1: Triton 实现逐元素相乘

矩阵乘法 设矩阵 $A \in \mathbb{R}^{M \times K}$, $B \in \mathbb{R}^{K \times N}$, $C = AB$ 。我们将输出矩阵 C 划分为大小为 $\text{BLOCK_SIZE_M} \times \text{BLOCK_SIZE_N}$ 的子块，每个 program instance 负责计算其中一个子块 ($\text{pid}_m, \text{pid}_n$)。

对于每个子块，其计算可以表示为对应子矩阵的矩阵乘法：

$$C^{(block)} = A^{(block)} \cdot B^{(block)}$$

其中：

- $A^{(block)} \in \mathbb{R}^{\text{BLOCK_SIZE_M} \times K}$ 表示 A 中对应行范围的子矩阵
- $B^{(block)} \in \mathbb{R}^{K \times \text{BLOCK_SIZE_N}}$ 表示 B 中对应列范围的子矩阵

该形式等价于在 K 维上的求和：

$$C_{m,n} = \sum_{k=0}^{K-1} A_{m,k} B_{k,n}$$

代码 7.2 给出了对应实现。

```

1 @triton.jit
2 def matmul_kernel(a_ptr, b_ptr, c_ptr,
3     M, N, K,
4     BLOCK_SIZE_M: tl.constexpr = 64,
5     BLOCK_SIZE_N: tl.constexpr = 64
6 ):
7     # 每个 program instance 对应输出矩阵 C 的一个子块
8     pid_m = tl.program_id(0) # 行方向 block index
9     pid_n = tl.program_id(1) # 列方向 block index
10
11     # 当前 block 对应的行/列索引
12     offs_m = pid_m * BLOCK_SIZE_M + tl.arange(0, BLOCK_SIZE_M) # (BM,)
13     offs_n = pid_n * BLOCK_SIZE_N + tl.arange(0, BLOCK_SIZE_N) # (BN,)
14     # K 维完整索引 (一次性展开)
15     offs_k = tl.arange(0, K) # (K,)
16     # 加载 A 的子矩阵: (BM, K)
17     a = tl.load(a_ptr + offs_m[:, None] * K + offs_k[None, :],
18         mask=(offs_m[:, None] < M) & (offs_k[None, :] < K), other=0.0
19     )
20     # 加载 B 的子矩阵: (K, BN)
21     b = tl.load(b_ptr + offs_k[:, None] * N + offs_n[None, :],
22         mask=(offs_k[:, None] < K) & (offs_n[None, :] < N), other=0.0
23     )
24     # 矩阵乘法 (在 K 维上完成隐式求和)
25     acc = tl.dot(a, b) # (BM, BN)
26     # 写回结果
27     tl.store(c_ptr + offs_m[:, None] * N + offs_n[None, :], acc,
28         mask=(offs_m[:, None] < M) & (offs_n[None, :] < N)
29     )

```

代码 7.2: 2D Grid 版本 Triton 矩阵乘法

该表达形式上避免了循环，比较简洁。但需要一次性加载大小为 $\text{BLOCK_SIZE_M} \times K$ 和 $K \times \text{BLOCK_SIZE_N}$ 的数据，当 K 较大时会带来较高的寄存器和内存访问开销，成为性能瓶颈。

分块矩阵乘法 下面介绍 Triton 上的分块矩阵乘法实现。其核心思想是将 K 维划分为大小为 $BLOCK_K$ 的子块，并在每次迭代中仅处理一个 K 子块对应的计算贡献。具体而言，每个 program instance 负责计算输出矩阵 C 的一个子块（大小为 $BLOCK_SIZE_M \times BLOCK_SIZE_N$ ），并在 K 维循环中累加局部结果。每一步计算均对应于如下形式的子块矩阵乘法：

$$(BLOCK_SIZE_M \times BLOCK_SIZE_K) \times (BLOCK_SIZE_K \times BLOCK_SIZE_N)$$

其结果被逐步累加至 C 的对应子块中。由于 K 维被划分为多个较小的子块， A 和 B 的局部数据可以在计算过程中被重复利用，从而提升缓存命中率并减少全局内存访问开销，最终在 GPU 上获得更高的计算性能。代码 7.3 实现了上述分块矩阵乘法。

```

1 @triton.jit
2 def matmul_kernel(
3     a_ptr, b_ptr, c_ptr,
4     M, N, K,
5     BLOCK_SIZE_M: tl.constexpr = 64,
6     BLOCK_SIZE_N: tl.constexpr = 64,
7     BLOCK_SIZE_K: tl.constexpr = 32,
8 ):
9
10     # 每个 program instance 直接对应一个 2D tile (pid_m, pid_n)
11     pid_m = tl.program_id(0)
12     pid_n = tl.program_id(1)
13
14     # 当前 block 在全局矩阵中的索引范围
15     offs_m = pid_m * BLOCK_SIZE_M + tl.arange(0, BLOCK_SIZE_M)
16     offs_n = pid_n * BLOCK_SIZE_N + tl.arange(0, BLOCK_SIZE_N)
17     offs_k = tl.arange(0, BLOCK_SIZE_K)
18
19     acc = tl.zeros((BLOCK_SIZE_M, BLOCK_SIZE_N), dtype=tl.float32)
20
21     num_k = tl.cdiv(K, BLOCK_SIZE_K) # 计算 K 维度上的子块个数，向上取整
22
23     for k in range(num_k):
24         k_offsets = k * BLOCK_SIZE_K + offs_k
25
26         # A: (BLOCK_SIZE_M, BLOCK_SIZE_K)
27         a = tl.load(a_ptr + offs_m[:, None] * K + k_offsets[None, :],
28                     mask=(offs_m[:, None] < M) & (k_offsets[None, :] < K), other=0.0
29                 )
30
31         # B: (BLOCK_SIZE_K, BLOCK_SIZE_N)
32         b = tl.load(b_ptr + k_offsets[:, None] * N + offs_n[None, :],
33                     mask=(k_offsets[:, None] < K) & (offs_n[None, :] < N), other=0.0
34                 )
35
36         acc += tl.dot(a, b)
37
38     tl.store( c_ptr + offs_m[:, None] * N + offs_n[None, :], acc,
39             mask=(offs_m[:, None] < M) & (offs_n[None, :] < N)
40         )

```

代码 7.3: 2D Grid Triton 分块矩阵乘法

7.2.2 调用算子

由于 Triton 使用 Python 编写，因此调用非常方便。代码 7.4 和 7.5 分别给出了向量逐元素相乘以及矩阵乘法内核主机端调用过程。

其中，需要着重说明的是 `grid` 用于定义 Triton kernel 的并行执行规模。在 Triton 中，每个 kernel 实例（program instance）负责处理一个数据块，而 `grid` 决定了需要启动多少个 instance。从形式上看，`grid` 支持一维、二维甚至三维结构，其本质与 CUDA 中的 `gridDim` 类似。

```
1 # Host 端函数：封装 Triton Kernel 调用过程
2 def vecmul(x, y):
3     assert x.shape == y.shape, "向量维度不匹配"
4     out = torch.empty_like(x) # 创建输出向量，与输入形状相同
5     N = x.numel() # 计算待处理的元素总数
6
7     # 定义 grid 规模：根据 BLOCK_SIZE 计算所需 program instance 数量
8     grid = lambda meta: (triton.cdiv(N, meta['BLOCK_SIZE']))
9
10    # 启动 Triton Kernel
11    # 等价于：vecmul_kernel[triton.cdiv(N, BLOCK_SIZE)](x, y, out, N, BLOCK_SIZE=1024)
12    vecmul_kernel[grid](x, y, out, N, BLOCK_SIZE=1024)
13
14    return out
```

代码 7.4: Host 端调用封装：Triton 实现逐元素相乘

```
1 # Host 端函数：封装 Triton 2D 矩阵乘法 kernel 调用
2 def matmul(a, b):
3     assert a.shape[1] == b.shape[0], "矩阵维度不匹配"
4     M, K = a.shape
5     K_, N = b.shape
6     out = torch.empty((M, N), device=a.device, dtype=a.dtype)
7
8     # 2D grid: 每个 program instance 对应一个 (tile_m, tile_n)
9     grid = lambda META: (
10         triton.cdiv(M, META['BLOCK_SIZE_M']),
11         triton.cdiv(N, META['BLOCK_SIZE_N']),
12     )
13
14    # 启动 Triton kernel
15    matmul_kernel[grid](a, b, out, M, N, K, BLOCK_SIZE_M=64, BLOCK_SIZE_N=64)
16
17    return out
```

代码 7.5: Host 调用封装：Triton 2D Grid 矩阵乘法

在 Triton 的实际实现中，也可以将二维 `grid` 展平成一维进行调度，再在 kernel 内通过 `program_id` 手动恢复二维索引，以简化调度逻辑，提升编译器优化空间。

7.3 Triton 的 Autotune 机制

在前面的实现中,我们在调用 kernel 时通常需要手动指定超参数,例如 `BLOCK_SIZE_M`、`BLOCK_SIZE_N`、`BLOCK_SIZE_K` 等。这些参数对性能影响非常显著,但其最优组合往往依赖具体硬件 (GPU 型号)、数据规模以及访存模式。传统手动调参方式需要对不同配置进行 benchmark 来选择最优组合。这种方式虽然有效,但参数迁移性差,在不同 GPU 上往往需要重新调优。

为了解决这一问题, Triton 提供了 autotune 机制,可以在运行时结合 JIT 编译自动搜索最优 kernel 配置。也就是说, kernel 并不是在编译期一次性生成唯一版本,而是会根据不同配置动态生成多个编译后的 kernel 版本 (compiled variants),并在实际执行时进行选择。

以分块矩阵乘法为例,开发者可以定义多个候选配置:

```
1 @triton.autotune(  
2     configs=[  
3         triton.Config({'BLOCK_SIZE_M': 64, 'BLOCK_SIZE_N': 64, 'BLOCK_SIZE_K': 32}),  
4         triton.Config({'BLOCK_SIZE_M': 128, 'BLOCK_SIZE_N': 64, 'BLOCK_SIZE_K': 32}),  
5         triton.Config({'BLOCK_SIZE_M': 64, 'BLOCK_SIZE_N': 128, 'BLOCK_SIZE_K': 32}),  
6         triton.Config({'BLOCK_SIZE_M': 128, 'BLOCK_SIZE_N': 128, 'BLOCK_SIZE_K': 32}),  
7     ],  
8     key=['M', 'N', 'K'],  
9 )  
10 @triton.jit  
11 def matmul_kernel(  
12     a_ptr, b_ptr, c_ptr,  
13     M, N, K,  
14     BLOCK_SIZE_M: tl.constexpr = 64,  
15     BLOCK_SIZE_N: tl.constexpr = 64,  
16     BLOCK_SIZE_K: tl.constexpr = 32,  
17 ):
```

在这里, `@triton.jit` 表示该 kernel 会在运行时触发 JIT 编译。不同的 autotune 配置会生成不同的已编译 kernel 实例,这些实例在第一次被调用时才会被编译并缓存,从而避免重复编译开销。

除了上述参数, Triton 还提供用于控制线程并行度的 `num_warps` 和用于调节软件流水线深度的 `num_stages` 参数。在实际应用中,这些参数也可以加入搜索空间,从而进一步提升性能。

Autotune 的核心流程如下:

- 1) 根据 shape 输入配置生成候选 kernel 配置集合;
- 2) 通过 JIT 编译为多个不同的 kernel 编译版本;
- 3) 分别在 GPU 上执行并测量运行时间;
- 4) 选择最快的配置,并将对应的编译结果缓存;
- 5) 后续相同 shape 直接复用已编译的最优 kernel。

因此, autotune 本质上是一个“运行时 JIT + 编译期搜索”的混合优化过程。Triton autotune 将传统人工调参问题转化为自动搜索问题,并通过 JIT 编译机制动态生成和复用多个 kernel 编译版本,从而显著降低 kernel 优化门槛。但需要注意, autotune 会带来首次运行开销:因为需要触发多次 JIT 编译与 benchmark。当搜索空间过大时,这一过程会显著增加启动延迟。因此,在实际工程中,常见策略是先手动设计合理的搜索空间,再交由 autotune 自动选择最优解。

7.4 在 PyTorch 中使用自定义算子

在实际深度学习系统中，我们常需用自定义算子替代框架原生实现，以优化性能、支持新功能或进行算法实验。例如，在注意力机制（Attention）、融合算子（fused operator）以及特殊数据布局计算中，使用 Triton 可以显著减少内存访问开销并提升执行效率。

总体而言，将 Triton 算子集成到 PyTorch 中可以分为以下三个步骤：

- 1) **实现算子**：使用 Triton 编写底层 kernel，实现具体计算逻辑，如矩阵乘法或卷积；
- 2) **封装为 PyTorch 接口**：通过 `torch.autograd.Function` 对 kernel 进行封装，定义前向与反向传播，并进一步封装为 `nn.Module`，以便在模型中复用；
- 3) **在模型中使用算子**：将封装好的模块嵌入到具体网络结构中，从而参与端到端训练与推理。

需要注意的是，Triton 主要负责高性能前向计算，而反向传播通常可以直接使用 PyTorch 的矩阵运算实现，从而降低开发复杂度。

本节以经典的 LeNet-5 模型为例，展示如何将一个用 Triton 编写的自定义全连接层算子集成到 PyTorch 模型中，并参与端到端训练。

7.4.1 全连接层算子

全连接层本质上是矩阵乘法加偏置操作。设输入为 $X \in \mathbb{R}^{B \times \text{in_features}}$ ，权重为 $W \in \mathbb{R}^{\text{out_features} \times \text{in_features}}$ ，偏置为 $b \in \mathbb{R}^{\text{out_features}}$ ，则输出为：

$$Y = XW^T + b$$

该计算可以直接复用前文的分块矩阵乘法实现。具体而言，全连接层的核心操作 XW^T 等价于一个矩阵乘法，因此可以在代码 7.3 的基础上，对输入矩阵与权重矩阵进行分块计算；在此基础上，只需在输出矩阵的每一行上加上偏置向量 b ，即可得到最终结果。对应的 Triton 实现如代码 7.6 所示。

```

1 @triton.jit
2 def linear_kernel(in_ptr, weight_ptr, out_ptr, bias_ptr,
3                 B, in_features, out_features,
4                 BLOCK_SIZE_B: tl.constexpr = 32,
5                 BLOCK_SIZE_OUT: tl.constexpr = 64,
6                 BLOCK_SIZE_K: tl.constexpr = 32):
7     """
8     in_ptr: [B, in_features]
9     weight_ptr: [out_features, in_features]
10    out_ptr: [B, out_features]
11    bias_ptr: [out_features]
12    """
13    # 2D grid: 每个 program instance 负责一个 Tile (Batch, Out Feature)
14    pid_batch = tl.program_id(0)
15    pid_out = tl.program_id(1)
16    offs_batch = pid_batch * BLOCK_SIZE_B + tl.arange(0, BLOCK_SIZE_B) # (BB,)
17    offs_out = pid_out * BLOCK_SIZE_OUT + tl.arange(0, BLOCK_SIZE_OUT) # (BO,)
18    offs_k = tl.arange(0, BLOCK_SIZE_K) # (BK,)
19    acc = tl.zeros((BLOCK_SIZE_B, BLOCK_SIZE_OUT), dtype=tl.float32)
20    num_k = tl.cdiv(in_features, BLOCK_SIZE_K)
21    for k in range(num_k):
22        k_offsets = k * BLOCK_SIZE_K + offs_k # (BK,)
23
24        # 加载 X 子块: (BB, BK)
25        x_block = tl.load(in_ptr + offs_batch[:, None] * in_features + k_offsets[None, :],
26                          mask=(offs_batch[:, None] < B) &
27                              (k_offsets[None, :] < in_features), other=0.0
28                          )
29
30        # 加载 W 子块: (BK, BO);
31        w_block = tl.load(weight_ptr + offs_out[None, :] * in_features + k_offsets[:, None],
32                          mask=(offs_out[None, :] < out_features) & (k_offsets[:, None] < in_features),
33                          other=0.0
34                          )
35
36        # 子块矩阵乘法累加
37        acc += tl.dot(x_block, w_block)
38
39    # 加 bias
40    if bias_ptr is not None:
41        bias = tl.load(bias_ptr + offs_out, mask=offs_out < out_features, other=0.0)
42        acc += bias
43
44    # 写回
45    tl.store(out_ptr + offs_batch[:, None] * out_features + offs_out[None, :], acc,
46            mask=(offs_batch[:, None] < B) & (offs_out[None, :] < out_features)
47            )

```

代码 7.6: Triton 实现全连接层算子（分块矩阵乘法版本）

7.4.2 算子封装

为了在 PyTorch 中直接使用 Triton kernel，需要将其封装为 PyTorch 算子，主要包括以下两个步骤：

- **兼容自动微分**：通过继承 `torch.autograd.Function` 并实现 `forward` 和 `backward` 方法，将 Triton kernel 集成到 PyTorch 计算图中，从而支持端到端梯度计算。
- **模块化封装**：通过 `torch.nn.Module` 对算子进行封装，使其能够像原生 PyTorch 层一样被调用。

```
1 class TritonLinearFunction(torch.autograd.Function):
2     @staticmethod
3     def forward(ctx, input, weight, bias=None):
4         B, in_features = input.shape
5         out_features = weight.shape[0]
6         out = torch.empty((B, out_features), device=input.device, dtype=input.dtype)
7         # 2D grid: batch 维 × 输出特征维
8         grid = lambda meta: ((B + meta['BLOCK_SIZE_B'] - 1) // meta['BLOCK_SIZE_B'],
9                               (out_features + meta['BLOCK_SIZE_OUT'] - 1) // meta['BLOCK_SIZE_OUT'],
10                              )
11         # 调用 Triton kernel
12         linear_kernel[grid](input, weight, out, bias, B, in_features, out_features,
13                             BLOCK_SIZE_B=32, BLOCK_SIZE_OUT=64)
14         # 保存反向传播所需张量
15         ctx.save_for_backward(input, weight, bias)
16         return out
17
18     @staticmethod
19     def backward(ctx, grad_output):
20         input, weight, bias = ctx.saved_tensors # 取出 forward 保存的张量
21         grad_input = grad_weight = grad_bias = None # 初始化梯度
22         if ctx.needs_input_grad[0]:
23             # [B, out] @ [out, in] -> [B, in]
24             grad_input = grad_output @ weight # 输入梯度: dX = dY @ W (传递给前一层)
25         if ctx.needs_input_grad[1]:
26             # [out, B] @ [B, in] -> [out, in]
27             grad_weight = grad_output.t() @ input # 权重梯度: dW = dY^T @ X
28         if bias is not None and ctx.needs_input_grad[2]:
29             # [B, out] -> sum over batch -> [out]
30             grad_bias = grad_output.sum(0) # bias 梯度: 对 batch 维求和
31         return grad_input, grad_weight, grad_bias
32
33 class TritonLinear(torch.nn.Module):
34     def __init__(self, in_features, out_features, bias=True):
35         super().__init__()
36         self.in_features = in_features
37         self.out_features = out_features
38         self.weight = torch.nn.Parameter(torch.randn(out_features, in_features))
39         self.bias = torch.nn.Parameter(torch.randn(out_features)) if bias else None
40
41     def forward(self, x):
42         return TritonLinearFunction.apply(x, self.weight, self.bias)
```

代码 7.7: PyTorch 自定义全连接层模块

7.4.3 集成到 LeNet-5

下面将自定义算子集成到经典 LeNet-5 网络中，如代码 7.8所示。

```
1 class LeNet5(torch.nn.Module):
2     def __init__(self):
3         super().__init__()
4
5         self.conv1 = torch.nn.Conv2d(1, 6, 5)
6         self.pool = torch.nn.AvgPool2d(2)
7         self.conv2 = torch.nn.Conv2d(6, 16, 5)
8
9         # 使用 Triton 实现的全连接层
10        self.fc1 = TritonLinear(16*5*5, 120)
11        self.fc2 = TritonLinear(120, 84)
12        self.fc3 = TritonLinear(84, 10)
13
14    def forward(self, x):
15        x = self.pool(torch.relu(self.conv1(x)))
16        x = self.pool(torch.relu(self.conv2(x)))
17        x = x.view(-1, 16*5*5)
18        x = torch.relu(self.fc1(x))
19        x = torch.relu(self.fc2(x))
20        x = self.fc3(x)
21        return x
```

代码 7.8: 集成 Triton 算子的 LeNet-5

练习

- 1) 用 Triton 实现 Average Pooling 算子，并集成到 LeNet-5 中。
- 2) 结合上节课的内容，对比分析 CUDA 和 Triton 的方案实现，包括：
 - 抽象设计与开发复杂度
 - 使用便捷性
- 3) 结合第三章的内容，分析 Triton 装饰器（@triton.jit）的实现原理。

8 算子融合

本章学习目标：

- ** 理解神经网络中的常见性能瓶颈及其分析方法；
- *** 掌握算子融合的基本原理与优化动机；
- ** 了解 PyTorch 中算子融合的实现方式及其优化机制；

8.1 问题分析

8.1.1 访存与执行开销分析

以代码 8.1 所示的 MLP 模型为例，我们从内存访问等方面分析其性能瓶颈。

```
1 class MLP(nn.Module):
2     def __init__(self, feature=1, hidden=64):
3         super().__init__()
4         # 第一层参数:  $1 \rightarrow \text{hidden}$ 
5         self.weight1 = nn.Parameter(torch.randn(feature, hidden)) #  $(F, H)$ 
6         self.bias1 = nn.Parameter(torch.randn(hidden)) #  $(H,)$ 
7
8         # 第二层参数:  $\text{hidden} \rightarrow 1$ 
9         self.weight2 = nn.Parameter(torch.randn(hidden, 1)) #  $(H, 1)$ 
10        self.bias2 = nn.Parameter(torch.randn(1)) #  $(1,)$ 
11
12    def forward(self, x):
13        x = x @ self.weight1 + self.bias1 #  $(B, H)$ 
14        x = torch.relu(x)
15        x = x @ self.weight2 + self.bias2 #  $(B, 1)$ 
16        return x
```

代码 8.1: MLP 模型

设输入为 $x \in \mathbb{R}^{B \times F}$ ，隐藏层维度为 H 。该 MLP 前向传播主要包含两次矩阵乘法：

- 第一层： $(B \times F) \cdot (F \times H) \rightarrow (B \times H)$
- 第二层： $(B \times H) \cdot (H \times 1) \rightarrow (B \times 1)$

如果不考虑算子融合，可以将矩阵乘法、bias 加法和 ReLU 视为多个独立算子。在 GPU 上，每个算子通常对应一次 kernel 启动，并需要从全局内存读取输入数据，计算完成后再将结果写回全局内存，供后续算子使用。因此，算子之间的中间结果需要在全局内存中显式存储与传递。

在该模型中， $(B \times H)$ 的张量正是不同算子之间传递的中间激活，其访存次数如表 8.1 所示。从整体规模来看，访存主要由输入 $O(BF)$ 、权重 $O(FH)$ 以及中间激活 $O(BH)$ 构成。其中，中间激活在多个算子之间被反复读取与写回，是主要的数据搬运来源。当 B 和 H 较大时，这部分开销会占主导，使模型更容易受到内存带宽限制。该问题通常可以通过算子融合减少中间结果的读写来缓解。

表 8.1: MLP 前向传播访存分析

阶段	读取 (Read)	写入 (Write)	主要张量
第一层矩阵乘法 XW_1	$BF + FH$	$B \times H$	x, W_1
第一层 bias 加法	$B \times H + H$	$B \times H$	中间激活, b_1
ReLU 激活	$B \times H$	$B \times H$	中间激活
第二层矩阵乘法 XW_2	$B \times H + H$	B	激活, W_2
第二层 bias 加法	$B + 1$	B	输出, b_2
总计	$\approx BF + FH + 3BH + 2H + 2B$		$\approx 2BH + 2B$

在 GPU 上, 这一问题还受到片上存储容量的影响。以 nVidia Ampere 架构为例, 每个 SM 上共享内存约为 164 KB。例如, 当 $B = 32, H = 64$ 时, 中间激活大小约为 8 KB (FP32), 从容量上可以放入共享内存。但在实际执行中, 还需要同时存放输入、权重以及其它线程块的数据, 因此能够用于缓存中间激活的空间是有限的。随着 B 或 H 增大, 中间激活规模按 $O(BH)$ 增长, 很快就无法完全驻留在共享内存中, 只能频繁在全局内存与共享内存之间进行数据交换, 从而增加访存开销。该问题通常通过分块 (tiling) 等方法提升局部数据复用 (例如分块矩阵乘法)。

8.1.2 算子融合

算子融合 (Operator Fusion) 是深度学习框架中的一种重要优化技术, 其核心思想是将多个连续的算子合并为一个计算单元, 从而减少中间张量的读写次数、降低内存访问开销, 并提高计算效率。

例如, 代码 8.1 中的全连接层由矩阵乘法与偏置加法两个独立操作组成:

$$y = xW + b$$

在 PyTorch 中, 这两步已经被实现为一个融合算子 `nn.Linear`, 从而减少一次内存读写操作, 提高计算效率。代码 8.2 给出基于 `nn.Linear` 的标准实现。

```

1 class MLP(nn.Module):
2     def __init__(self, feature=1, hidden=64):
3         super().__init__()
4         self.fc1 = nn.Linear(feature, hidden)
5         self.fc2 = nn.Linear(hidden, 1)
6
7     def forward(self, x):
8         x = self.fc1(x)
9         x = F.relu(x)
10        x = self.fc2(x)
11        return x

```

代码 8.2: MLP 模型: 使用 `nn.Linear` 替换矩阵乘法和加法

改写后, 原本由 5 个独立算子产生的全局内存读写, 被减少为 3 个融合算子的读写操作, 从而显著降低了访存开销。在此基础上, 还可以进一步构造更大的融合算子 (如将全连接层与 ReLU 合并), 以继续减少中间结果的读写次数。接下来, 我们将进一步探讨自动化的算子融合技术。

8.2 计算图优化

在深度学习中，模型的计算过程（包括前向计算与反向传播）通常可表示为有向无环图（DAG），称为计算图（Computation Graph）。每个节点代表一个算子，边代表张量数据流。计算图优化是指在保持语义不变的前提下，对计算图进行变换，以通过算子融合减少算子数量、通过降低内存占用提升缓存利用率等。

8.2.1 在 PyTorch 中开启计算图优化

在 PyTorch 中，可以通过 `torch.compile` 启用计算图级别优化，从而实现算子融合与执行加速。如代码 8.3 所示，在模型实例化并迁移至 GPU 后，对模型进行编译优化，后续直接使用返回的编译模型。

```
1 model = MLP().to(device)
2 model = torch.compile(model, backend="inductor", mode="default")
```

代码 8.3: 编译 MLP 模型

`torch.compile` 是 PyTorch 2.x 提供的图编译接口，其函数签名支持多个关键参数，用于控制计算图捕获方式、优化策略及执行后端。主要参数如下：

- `model`: 待编译的 PyTorch 模型或函数，是必选参数。
- `backend="inductor"`: 指定编译后端，默认使用 `TorchInductor`。
- `mode=None`: 用于控制编译优化策略的预设模式，在编译开销与运行性能之间进行权衡。常见选项包括：
 - `"default"`: 默认模式，在编译时间与运行性能之间取得平衡；
 - `"reduce-overhead"`: 减少 Python 调度与框架开销，适用于小模型或频繁调用场景；
 - `"max-autotune"`: 通过更激进的 kernel 搜索获得最高运行性能，但编译时间显著增加。
- `fullgraph=False`: 控制是否强制进行完整计算图捕获。当设为 `True` 时，若执行过程中发生 graph break（如 Python 控制流无法静态追踪），将直接报错，从而保证图优化的完整性，但会降低兼容性。
- `dynamic=False`: 控制是否启用动态形状支持。若为 `True`，模型允许输入张量 shape 在运行时变化，但可能限制部分静态图优化（如算子融合与内存优化）。

8.2.2 PyTorch 计算图优化流程

`torch.compile` 的优化流程以 FX Graph 作为统一的中间表示（IR），整体由 TorchDynamo（图捕获）、AOTAutograd（自动微分图分解）与 TorchInductor（图优化与代码生成）等组件协同完成，如图 8.1 所示。

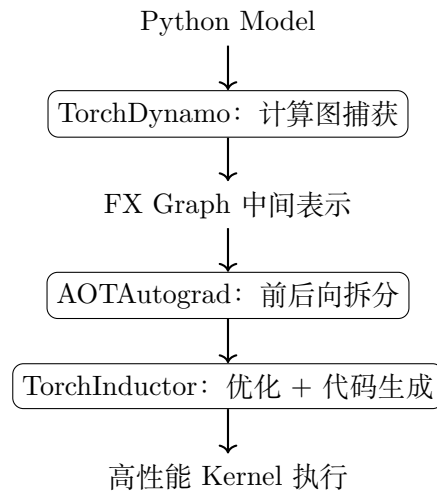


图 8.1: PyTorch 计算图优化组件的协作流程

- **TorchDynamo (捕获)**: 负责在运行时捕获 Python 程序中的张量操作，将动态图转换为静态计算图 (FX Graph)。其目标是在不改变用户代码的前提下，实现图级分析与优化的入口。
- **FX Graph (中间表示)**: 作为统一的中间表示 (IR)，描述模型的计算结构。该阶段支持符号追踪 (symbolic tracing) 以及图变换 (graph rewrite)，为后续优化提供基础。
- **AOTAutograd (前后向拆分)**: 将自动求导过程提前 (ahead-of-time) 展开，把前向计算与反向梯度计算分离成两个独立的计算图，从而提升优化空间与执行效率。
- **TorchInductor (优化 + 代码生成)**: 作为 PyTorch 编译器后端，对计算图进行多层级优化，并基于中间表示进行调度，最终生成高性能后端代码 (GPU 上生成 Triton kernel)。

为了分析编译过程中的图结构与优化细节，可以通过开启 `TORCH_COMPILE_DEBUG` 环境变量来导出中间表示与生成代码。

```
TORCH_COMPILE_DEBUG=1 python mlp.py
```

一般会生成如下文件结构，其中 `forward` 与 `backward` 目录分别对应前向计算图与反向梯度计算图的编译结果，涵盖从高层计算图表示到低层执行代码的多个阶段。其中，`fx_graph_readable.py` 表示优化前的 FX Graph，`fx_graph_transformed.py` 表示经过图变换优化后的计算图结构，而 `output_code.py` 则对应算子融合后生成的高性能执行代码。

```

|-- aot_model__0_debug.log           # AOTAutograd 调试日志
|-- model__0_forward_1.0/           # 前向计算图编译结果
|   |-- fx_graph_readable.py        # FX Graph (前向)
|   |-- fx_graph_transformed.py     # 优化后的 FX Graph
|   |-- ir_pre_fusion.txt           # fusion 前 IR 表示
|   |-- ir_post_fusion.txt          # fusion 后 IR 表示
|   |-- output_code.py              # 生成的 kernel 代码
`-- model__0_backward_3.1/          # 反向计算图编译结果
    |-- fx_graph_readable.py        # FX Graph (反向)
    |-- fx_graph_transformed.py     # 优化后的 FX Graph
    |-- ir_pre_fusion.txt           # fusion 前 IR 表示
    |-- ir_post_fusion.txt          # fusion 后 IR 表示
    |-- output_code.py              # 生成的 kernel 代码
  
```

8.2.3 forward 优化分析

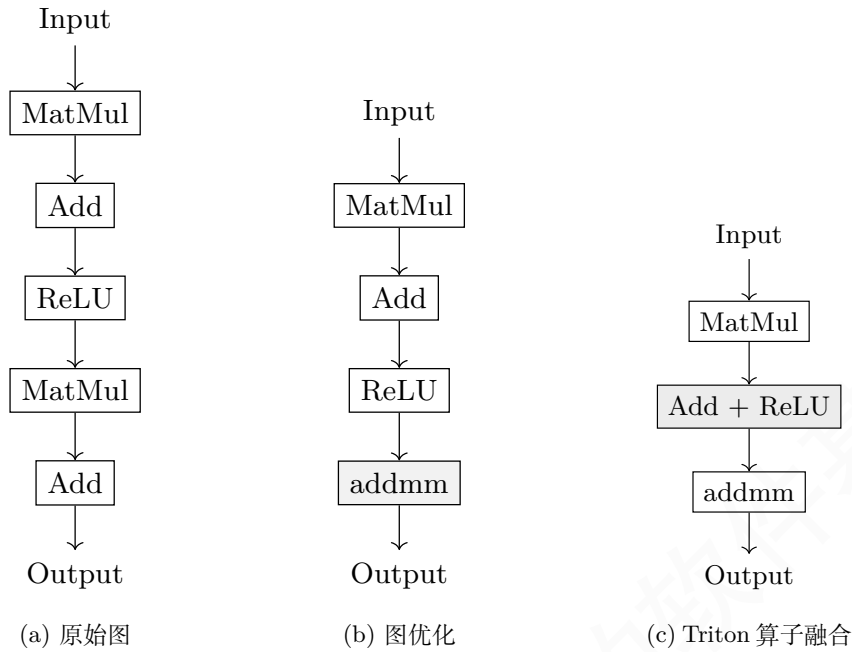


图 8.2: 前向计算图优化过程

优化前的 FX Graph 代码 8.4展示了 `fx_graph_readable.py` 文件的内容。该计算图的返回值列表 `[add_1, relu, permute_1, permute_2]` 为后续反向传播与编译优化提供了必要的计算上下文。各返回值的具体作用如下表所示：

表 8.2: MLP 前向图返回值在梯度计算中的作用

返回值	形状	梯度计算	作用说明
<code>add_1</code>	$[128, 1]$	$\frac{\partial \mathcal{L}}{\partial \text{add_1}}$	模型最终输出张量，用于计算损失函数 $\mathcal{L}(\text{add_1}, y)$ 。
<code>relu</code>	$[128, 64]$	$\frac{\partial \mathcal{L}}{\partial \mathbf{W}_2}$	隐藏层输出，用于计算第二层权重梯度： $\frac{\partial \mathcal{L}}{\partial \mathbf{W}_2} = \text{relu}^\top \cdot \frac{\partial \mathcal{L}}{\partial \text{add_1}}$
<code>permute_1</code>	$[1, 64]$	$\frac{\partial \mathcal{L}}{\partial \text{relu}}$	第二层权重转置 $\mathbf{W}_2^\top$ ，用于梯度回传： $\frac{\partial \mathcal{L}}{\partial \text{relu}} = \frac{\partial \mathcal{L}}{\partial \text{add_1}} \cdot \text{permute_1}$
<code>permute_2</code>	$[1, 128]$	$\frac{\partial \mathcal{L}}{\partial \mathbf{W}_1}$	输入张量转置，用于计算第一层权重梯度： $\frac{\partial \mathcal{L}}{\partial \mathbf{W}_1} = \text{permute_2} \cdot \frac{\partial \mathcal{L}}{\partial \text{relu}}$

```

1 class GraphModule(torch.nn.Module):
2     def forward(self,
3         primals_1: "f32[1, 64]", # weight1: 第一层权重 (in_features=1 -> hidden=64)
4         primals_2: "f32[64]", # bias1: 第一层偏置
5         primals_3: "f32[64, 1]", # weight2: 第二层权重 (hidden=64 -> out_features=1)
6         primals_4: "f32[1]", # bias2: 第二层偏置
7         primals_5: "f32[128, 1]" # input: 输入数据 (batch_size=128, features=1)
8     ):
9         # ----- 第一层: Linear + ReLU -----
10        # 对应 mlp.py:21: x = x @ self.weight1 + self.bias1 # (B, H)
11        # 矩阵乘法: (128,1) @ (1,64) -> (128,64)
12        mm: "f32[128, 64]" = torch.ops.aten.mm.default(primals_5, primals_1)
13        primals_1 = None # FX 内存优化标记: 显式置空, 提示编译器/运行时可提前释放该张量显存
14        # 加法 (广播): (128,64) + (64,) -> (128,64)
15        add: "f32[128, 64]" = torch.ops.aten.add.Tensor(mm, primals_2)
16        mm = primals_2 = None # 释放中间变量, 降低峰值内存占用
17        # 对应 mlp.py:22: x = torch.relu(x)
18        relu: "f32[128, 64]" = torch.ops.aten.relu.default(add)
19        add = None
20
21        # ----- 第二层: Linear -----
22        # 对应 mlp.py:23: x = x @ self.weight2 + self.bias2 # (B, 1)
23        # 矩阵乘法: (128,64) @ (64,1) -> (128,1)
24        mm_1: "f32[128, 1]" = torch.ops.aten.mm.default(relu, primals_3)
25        # 加法 (广播): (128,1) + (1,) -> (128,1)
26        add_1: "f32[128, 1]" = torch.ops.aten.add.Tensor(mm_1, primals_4)
27        mm_1 = primals_4 = None
28
29        # ----- 用于梯度计算 -----
30        permute_1: "f32[1, 64]" = torch.ops.aten.permute.default(primals_3, [1, 0]) # weight2.T
31        primals_3 = None
32        permute_2: "f32[1, 128]" = torch.ops.aten.permute.default(primals_5, [1, 0]) # input_x.T
33        primals_5 = None
34
35        return [add_1, relu, permute_1, permute_2]

```

代码 8.4: MLP 模型优化前的前向 FX Graph (fx_graph_readable.py 文件)

优化后的 FX Graph 代码 8.5展示了计算图优化后的结果。

```

1 class GraphModule(torch.nn.Module):
2     def forward(self, ... ):
3         ...
4         # ----- 第二层: bias + (relu @ weight2) -----
5         addmm_default: "f32[32, 1]" = torch.ops.aten.addmm.default(
6             primals_4, # bias2
7             relu, # input
8             primals_3 # weight2
9         )
10        ...

```

代码 8.5: MLP forward 的 FX Graph 优化结果 (fx_graph_transformed.py 文件)

算子融合结果 代码 8.6展示了最终算子融合后的执行结果。TorchInductor 在该阶段完成了从 FX Graph 到 Triton Kernel 的 lowering, 并通过 kernel fusion 将 add + relu 的融合。

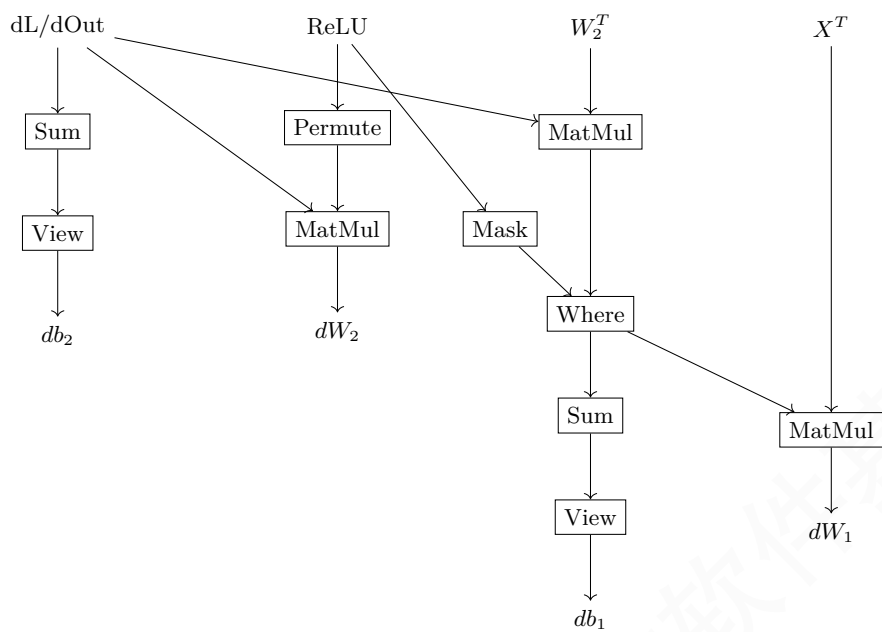
```

1 @triton_heuristics.pointwise(
2     size_hints=[8192],
3     triton_meta={'kernel_name': 'triton_poi_fused_add_relu_0', ...}
4 )
5 @triton.jit
6 def triton_(in_out_ptr0, in_ptr0, xnumel, XBLOCK: tl.constexpr):
7     xnumel = 8192
8     xoffset = tl.program_id(0) * XBLOCK
9     xindex = xoffset + tl.arange(0, XBLOCK)[: ]
10    xmask = xindex < xnumel
11    x2 = xindex
12    x0 = xindex % 64
13    tmp0 = tl.load(in_out_ptr0 + (x2), None)
14    tmp1 = tl.load(in_ptr0 + (x0), None, eviction_policy='evict_last')
15    # ----- 融合执行: add + relu -----
16    tmp2 = tmp0 + tmp1 # add
17    tmp3 = tl.full([1], 0, tl.int32)
18    tmp4 = triton_helpers.maximum(tmp3, tmp2) # ReLU: max(0, x)
19
20    tl.store(in_out_ptr0 + (x2), tmp4, None)
21
22 def call(args):
23     primals_1, primals_2, primals_3, primals_4, primals_5 = args
24     with torch.cuda._DeviceGuard(0):
25         torch.cuda.set_device(0)
26         buf0 = empty_strided_cuda((128, 64), (64, 1), torch.float32)
27         extern_kernels.mm(primals_5, primals_1, out=buf0)
28         del primals_1
29         buf1 = buf0
30         del buf0
31
32     # ----- 使用融合后的Triton算子: add + relu -----
33     stream0 = get_raw_stream(0)
34     triton_poi_fused_add_relu_0.run(
35         buf1,          # in-out tensor
36         primals_2,     # bias
37         8192,          # total elements
38         grid=grid(8192),
39         stream=stream0
40     )
41     del primals_2
42
43     buf3 = empty_strided_cuda((128, 1), (1, 1), torch.float32)
44     extern_kernels.addmm(primals_4, buf1, primals_3, alpha=1, beta=1, out=buf3)
45     del primals_4
46
47     return (buf3, buf1, ...)

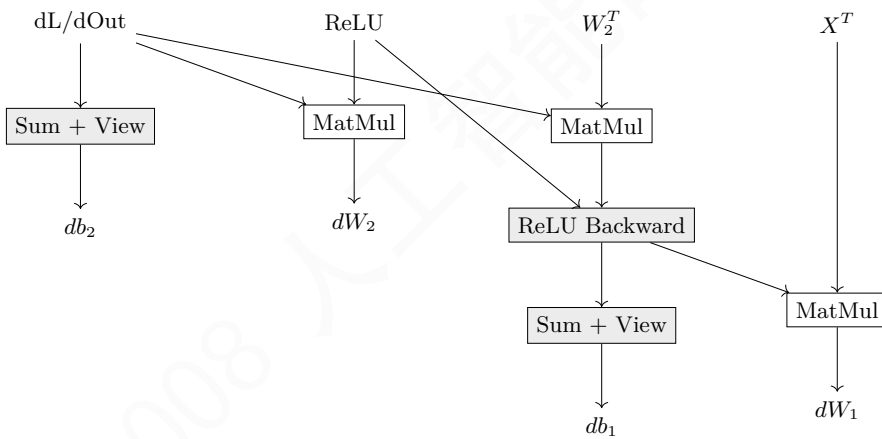
```

代码 8.6: MLP forward 的最终优化结果 (output_code.py 文件)

8.2.4 backward 优化分析



(a) 原始 Backward FX 图



(b) Triton 反向算子融合

图 8.3: 反向计算图优化过程

优化前的 FX Graph 代码 8.7是反向传播的 FX Graph，其结构直接对应梯度计算过程。

```

1 class GraphModule(torch.nn.Module):
2     def forward(self,
3         relu: "f32[128, 64]",          # forward中间激活值 (ReLU前输入)
4         permute_1: "f32[1, 64]",        # W2^T (第二层权重转置)
5         permute_2: "f32[1, 128]",       # X^T (第一层输入转置)
6         tangents_1: "f32[128, 1]"      # dL/dOutput (loss对输出梯度)
7     ):
8         # ----- dL/db2 = sum_batch -----
9         # bias梯度: 对batch维度求和 (128 + 1)
10        sum_1: "f32[1, 1]" = torch.ops.aten.sum.dim_IntList(tangents_1, [0], True)
11        view: "f32[1]" = torch.ops.aten.view.default(sum_1, [1])
12        sum_1 = None
13
14        # ----- dL/dW2 = relu^T @ dL/dout -----
15        # 第二层权重梯度 (GEMM)
16        permute: "f32[64, 128]" = torch.ops.aten.permute.default(relu, [1, 0])
17        mm_2: "f32[64, 1]" = torch.ops.aten.mm.default(permute, tangents_1)
18        permute = None
19
20        # ----- dL/drelu = dL/dout @ W2^T -----
21        mm_3: "f32[128, 64]" = torch.ops.aten.mm.default(tangents_1, permute_1)
22        tangents_1 = permute_1 = None
23
24        # ----- dL/dx = dL/drelu * 1(relu > 0) -----
25        # ReLU backward (逐元素mask)
26        le: "b8[128, 64]" = torch.ops.aten.le.Scalar(relu, 0)
27        full_default: "f32[]" = torch.ops.aten.full.default([], 0.0,
28            dtype=torch.float32,
29            layout=torch.strided,
30            device=torch.device('cuda', 0),
31            pin_memory=False
32        )
33        where: "f32[128, 64]" = torch.ops.aten.where.self(le, full_default, mm_3)
34        le = full_default = mm_3 = None
35
36        # ----- dL/db1 = sum_batch -----
37        # 第一层bias梯度 (128 + 1)
38        sum_2: "f32[1, 64]" = torch.ops.aten.sum.dim_IntList(where, [0], True)
39        view_1: "f32[64]" = torch.ops.aten.view.default(sum_2, [64])
40        sum_2 = None
41
42        # ----- dL/dW1 = X^T @ dL/dx -----
43        # 第一层权重梯度 (GEMM)
44        mm_4: "f32[1, 64]" = torch.ops.aten.mm.default(permute_2, where)
45        permute_2 = where = None
46
47        return [mm_4, view_1, mm_2, view, None]

```

代码 8.7: MLP backward 的 FX Graph

算子融合结果 代码 8.8展示了 TorchInductor 在反向传播阶段的算子融合与 kernel lowering 结果。系统将 FX Graph 中的算子组合映射为多个 Triton kernel，实现计算图级别优化。

```

1 @triton_heuristics.persistent_reduction(size_hints=[1, 32],
2     triton_meta={'kernel_name': 'triton_per_fused_sum_0', ...})
3 ...
4 @triton_heuristics.pointwise(size_hints=[2048],
5     triton_meta={'kernel_name': 'triton_poi_fused_threshold_backward_1', ...})
6 ...
7 @triton_heuristics.reduction(size_hints=[64, 32],
8     triton_meta={'kernel_name': 'triton_red_fused_sum_2', ...})
9 ...
10 def call(args):
11     relu, permute_1, permute_2, tangents_1 = args
12     ...
13     with torch.cuda._DeviceGuard(0):
14         torch.cuda.set_device(0)
15         # -----  $dL/db_2 = \text{sum\_batch}$  -----
16         buf0 = empty_strided_cuda((1, 1), (1, 1), torch.float32)
17         stream0 = get_raw_stream(0)
18         # 算子融合1:  $\text{sum}_1 + \text{view}$ 
19         triton_per_fused_sum_0.run(tangents_1, buf0, 1, 128, grid=grid(1), stream=stream0)
20
21         # -----  $dL/dW_2 = \text{relu}^T @ dL/dout$  -----
22         buf1 = empty_strided_cuda((64, 1), (1, 1), torch.float32)
23         extern_kernels.mm(reinterpret_tensor(relu, (64, 128), (1, 64), 0), tangents_1, out=buf1)
24
25         # -----  $dL/d\text{relu} = dL/dout @ W_2^T$  -----
26         buf2 = empty_strided_cuda((128, 64), (64, 1), torch.float32)
27         extern_kernels.mm(tangents_1, permute_1, out=buf2)
28         del permute_1; del tangents_1
29
30         # reuse buffer for ReLU backward
31         buf3 = buf2; del buf2
32
33         # -----  $dL/dx = dL/d\text{relu} * 1(\text{relu} > 0)$  -----
34         # 算子融合2:  $\text{where} + \text{mask}$ 
35         triton_poi_fused_threshold_backward_1.run(buf3, relu, 8192, grid=grid(8192), stream=stream0)
36         del relu
37
38         # -----  $dL/db_1 = \text{sum\_batch}$  -----
39         buf4 = empty_strided_cuda((1, 64), (64, 1), torch.float32)
40         # 算子融合23:  $\text{sum}_2 + \text{view}$ 
41         triton_red_fused_sum_2.run(buf3, buf4, 64, 128, grid=grid(64), stream=stream0)
42
43         # -----  $dL/dW_1 = X^T @ dL/dx$  -----
44         buf5 = empty_strided_cuda((1, 64), (64, 1), torch.float32)
45         extern_kernels.mm(permute_2, buf3, out=buf5)
46         del buf3; del permute_2
47
48     return (buf5, ...)

```

代码 8.8: MLP backward 的优化结果

8.3 性能分析工具

理想情况下，能够把算力用满，不因算子调度和访存影响性能最理想。在 Linux 环境下，可以使用 nVidia 提供的 profiling 工具获取程序运行的统计信息，从而分析性能瓶颈。常用工具是 nsys，可以统计 CPU/GPU 时间开销占比、kernel 调度等信息。

安装方法 上述工具通常随 CUDA Toolkit 一同安装。

```
sudo apt install nvidia-cuda-toolkit
```

使用 nsys

```
nsys profile python mlp.py
```

执行后会生成 .nsys-rep 文件，可通过以下命令查看摘要：

```
nsys stats report.qdrep
```

```
1  ** OS Runtime Summary (osrtsum):
2  Time(%)   Total(ns)   Calls   Avg(ns)   Name
3  -----
4  73.6      6.55e10      1035    6.32e7    pthread_cond_wait
5  8.3       7.37e9       77      9.57e7    poll
6  6.3       5.61e9       5527    1.02e6    read
7  5.9       5.27e9       85      6.20e7    pthread_cond_timedwait
8  3.0       2.66e9       13      2.05e8    sem_wait
9  ...
10 ** CUDA API Summary (cudaapisum):
11 Time(%)   Total(ns)   Calls   Avg(ns)   Name
12 -----
13 84.7      1.88e8      11010   1.71e4    cudaLaunchKernel
14 6.3       1.40e7      2000    7.00e3    cuLaunchKernel
15 5.2       1.16e7      3       3.86e6    cudaFree
16 0.4       8.50e5      65      1.31e4    cudaMemcpyAsync
17 ...
18 ** CUDA GPU Kernel Summary (gpukernsum):
19 Time(%)   Total(ns)   Inst    Avg(ns)   Kernel
20 -----
21 7.9       3.08e6      1000    3.07e3    gemvx
22 7.0       2.72e6      1000    2.72e3    gemm (cublas)
23 6.3       2.43e6      500     4.86e3    reduce_kernel
24 6.1       2.35e6      1000    2.35e3    triton_
25 6.0       2.31e6      1000    2.31e3    elementwise
26 5.8       2.26e6      500     4.53e3    multi_tensor_apply
27 5.6       2.17e6      1009    2.15e3    fill_kernel
28 ...
```

代码 8.9: nsys 统计结果

从上述统计结果可以观察到，cudaLaunchKernel 调用占据了绝大部分 CUDA API 时间 (84.7%)，每次调用的平均开销约为 1.7×10^4 ns (约 17 μ s)。从 GPU Kernel 统计可以看出，大量 kernel 的执行时间仅为 2 ~ 5 μ s。这类短时、高频的 kernel 反映出 GPU 计算粒度过细，单个 kernel 的计算时间甚至低于其调度开销，很可能导致 GPU 计算资源未能得到充分利用。

练习

1) 对下列 MLP 模型应用 `torch.compile` 进行编译优化，并分析其性能表现。

- 代码对比：分别给出优化前与优化后的模型实现，对比其结构与调用方式的差异；
- 性能测试：在不同 batch size（如 128、1024、4096）下，测量模型的运行时间或吞吐量，对比优化前后的性能变化，并分析其趋势；
- 性能分析：使用 `nsys` 对程序执行过程进行 profiling，尝试分析为什么在小 batch size 下优化效果可能不明显甚至下降。

```
1 class MLP(nn.Module):
2     def __init__(self, feature=1, hidden=1024):
3         super().__init__()
4         self.w1 = nn.Parameter(torch.randn(feature, hidden))
5         self.b1 = nn.Parameter(torch.randn(hidden))
6         self.w2 = nn.Parameter(torch.randn(hidden, hidden))
7         self.b2 = nn.Parameter(torch.randn(hidden))
8         self.w3 = nn.Parameter(torch.randn(hidden, 1))
9         self.b3 = nn.Parameter(torch.randn(1))
10
11     def forward(self, x):
12         x = x @ self.w1 + self.b1
13         x = torch.relu(x)
14         x = x @ self.w2 + self.b2
15         x = torch.relu(x)
16         x = x @ self.w3 + self.b3
17         return x
```

代码 8.10: MLP 模型：使用 `nn.Linear` 替换矩阵乘法和加法

2) 对 LeNet-5 模型应用 `torch.compile` 进行编译优化，对比优化前后的代码和运行性能。

Part III

大模型软件及其应用

9 大语言模型

本章学习目标：

- *** 理解词嵌入（Word Embedding）的基本原理与作用
- *** 理解注意力机制（Attention）的核心思想
- *** 理解基于 Decoder-only Transformer 的语言建模方法、自回归生成过程及其实现

大语言模型是一类以语言建模任务为核心、在大规模语料上训练的神经网络模型，其目标是通过自回归方式学习自然语言中词元（token）之间的统计与语义结构，从而实现文本的理解与生成。在实践中，现代大语言模型通常采用 Decoder-only Transformer 架构，并通过预测下一个词元来进行训练。

9.1 词元

9.1.1 词嵌入

对语言进行建模的第一步是将语言转化为可计算的数学表示。由于神经网络只能处理数值输入，自然语言中的词或符号需要被映射到向量空间中。这种将离散符号映射为连续向量表示的方法称为词嵌入（word embedding）。在词嵌入中，每个词元被表示为一个 d 维实向量，从而嵌入在同一个向量空间 $\mathbb{R}^d$ 中。

在最原始的表示方式中，一个词元可以用 one-hot 向量表示。设词表大小为 V ，则每个词元被表示为一个 V 维向量，其中仅有一个位置为 1，其余为 0。然而，这种表示方式存在明显缺陷：维度高且稀疏，且不同词之间没有任何语义关联。为了解决这一问题，引入了词嵌入。词嵌入本质上可以看作是对 one-hot 向量的一次线性变换。设 embedding 矩阵为 $W \in \mathbb{R}^{V \times d}$ ，则一个 token i 的向量表示可以写为：

$$\mathbf{e}_i = \mathbf{o}_i W$$

其中 $\mathbf{o}_i$ 是对应的 one-hot 向量。由于 $\mathbf{o}_i$ 只有第 i 个位置为 1，上式等价于直接取矩阵 W 的第 i 行。因此，在实际实现中并不会显式构造 one-hot 向量，而是通过查表操作获得对应的 embedding。词嵌入的关键在于其“稠密性”和“可学习性”。与 one-hot 表示不同，embedding 向量是低维且稠密的，并且其数值是通过训练自动学习得到的，从而能够在向量空间中编码词语之间的语义与语法关系。

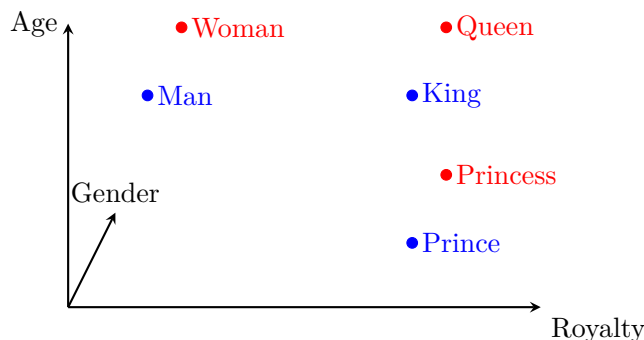


图 9.1: 词嵌入示意图

如果词嵌入的效果足够好，不同词之间的语义与语法关联关系可以在向量空间中得到体现。例如，在英语中，“King - Man + Woman $\approx$ Queen”，表示“King”和“Queen”之间的性别关系可以通过向量差

来表达。同样， $\text{King} - \text{Prince} \approx \text{Queen} - \text{Princess}$ ，表示成年与年轻之间的年龄关系在王室词汇中也能被捕捉。

那么，如何构建这样的向量空间，并将离散的词或 token 映射到其中呢？核心思想是：不人为规定词向量的具体含义，而是通过学习任务让模型自动发现这些表示。在现代大语言模型中，词嵌入一般不再是独立训练的模块，而是作为整个模型的一部分进行端到端训练。具体来说，embedding 矩阵 W 被视为模型的第一层参数，与后续的神经网络共同优化。在训练过程中，模型通常基于语言建模目标（如预测下一个 token 的概率），通过最小化损失函数来更新所有参数，包括 embedding。

值得注意的是，embedding 只是信息表示的起点。经过多层非线性变换（例如自注意力机制），每个词元会被映射为上下文相关的表示。因此，同一个词在不同语境中的最终向量表示是不同的。这种上下文相关的稠密向量才是大模型真正用于理解和生成语言的核心表示。

9.1.2 分词

在大语言模型中，词嵌入的输入单位通常并不是完整单词。词表 (vocabulary) 中既包含单个字符，也包含完整单词或更细粒度的子词 (subword)。在实际应用中，现代大模型普遍采用基于子词的分词方法，其核心思想之一是“最长匹配”：在词表中优先寻找能够匹配当前文本前缀的最长词元。

例如，假设词表中同时存在：

p, l, a, y, i, n, g, play, ing

当输入字符串为：

playing

时，分词器会优先匹配最长的词元 `play` 和 `ing`，而不是先匹配较短的 `p`。

这种分词方式的好处是在有限词表规模下，同时兼顾以下几个方面：

- 高频词能够整体保留；
- 低频词能够拆分为更小的子词；
- 未登录词 (OOV) 仍然可以被表示；
- 对拼写错误或错词具有一定鲁棒性。

9.2 注意力机制

Transformer 的核心是注意力（Attention）机制，其思想是通过计算序列中各词元之间的相关性，实现信息的加权聚合。在该机制中，每个输入向量（词嵌入）会被映射为 Query (Q)、Key (K) 和 Value (V)，并基于 Query 与 Key 的相似度计算注意力权重，从而对 Value 进行加权求和。标准的缩放点积注意力定义为：

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V,$$

其中，

$$Q \in \mathbb{R}^{n \times d_k}, \quad K \in \mathbb{R}^{n \times d_k}, \quad V \in \mathbb{R}^{n \times d_v},$$

n 表示序列长度， d_k 表示 Query/Key 的特征维度， d_v 表示 Value 的特征维度。缩放因子 $\sqrt{d_k}$ 用于避免点积结果随维度增大而过大，从而稳定 softmax 的梯度分布。

设输入序列为若干词元表示 $x_1, x_2, \dots, x_n \in \mathbb{R}^{d_{\text{model}}}$ ，其中 d_{model} 是词嵌入向量的维度。在进入注意力计算之前，每个 token 通过三组线性映射分别生成 Query、Key 和 Value：

$$q_i = x_i W_Q,$$

$$k_i = x_i W_K,$$

$$v_i = x_i W_V,$$

其中投影矩阵为：

$$W_Q \in \mathbb{R}^{d_{\text{model}} \times d_k}, \quad W_K \in \mathbb{R}^{d_{\text{model}} \times d_k}, \quad W_V \in \mathbb{R}^{d_{\text{model}} \times d_v}.$$

因此：

$$q_i \in \mathbb{R}^{d_k}, \quad k_i \in \mathbb{R}^{d_k}, \quad v_i \in \mathbb{R}^{d_v},$$

即在单个 attention head 中，Query/Key 被映射到用于相似度计算的子空间 $\mathbb{R}^{d_k}$ ，而 Value 位于 $\mathbb{R}^{d_v}$ 中用于信息聚合。

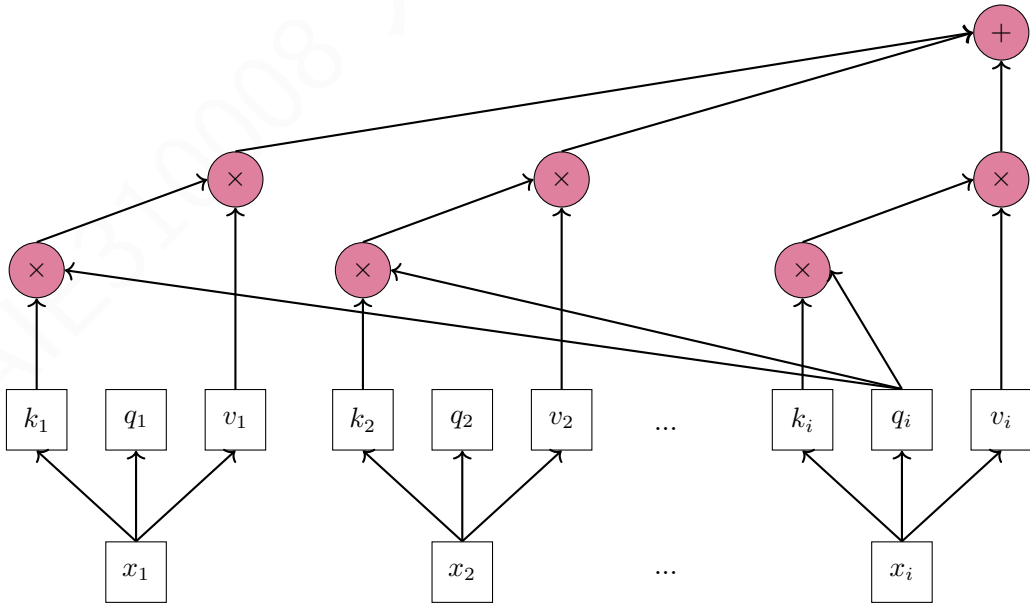


图 9.2: Token x_i 的注意力计算过程

图 9.2 展示了以第 i 个词元的嵌入向量为 Query 的注意力计算过程，具体步骤如下：

1) 设输入序列为 $x_1, x_2, \dots, x_i \in \mathbb{R}^{d_{\text{model}}}$ 。通过线性映射得到：

$$q_i = x_i W_Q,$$

并对所有历史 token $j \leq i$ 生成 Key 和 Value：

$$k_j = x_j W_K, \quad v_j = x_j W_V,$$

其中 $q_i, k_j \in \mathbb{R}^{d_k}$, $v_j \in \mathbb{R}^{d_v}$ 。

2) 计算 Query 与 Key 的点积相似度：

$$s_{ij} = q_i \cdot k_j,$$

图中紫色 $\times$ 节点表示该运算；

3) 通过 softmax 归一化得到注意力权重：

$$\alpha_{ij} = \text{softmax}\left(\frac{s_{ij}}{\sqrt{d_k}}\right), \quad \sum_{j=1}^i \alpha_{ij} = 1;$$

4) 对 Value 进行加权求和，得到输出：

$$\text{out}_i = \sum_{j=1}^i \alpha_{ij} v_j \in \mathbb{R}^{d_v},$$

图中紫色 $+$ 节点表示该聚合过程。

代码 9.1 使用 PyTorch 实现了注意力机制，完整模拟了缩放点积注意力的计算流程。该实现首先通过三个独立的线性映射将输入特征投影为 Query、Key 和 Value 表示，然后通过矩阵乘法计算 Query 与 Key 之间的相似度得分，并进行缩放以提高数值稳定性。在引入因果掩码后，模型被限制为只能关注当前位置及其之前的历史信息，从而满足自回归建模的约束条件。随后，对相似度分数进行 softmax 归一化以获得注意力权重，并对 Value 向量进行加权求和得到最终输出表示。

```

1 class SelfAttention(nn.Module):
2     def __init__(self, d_model=256, d_k=256, d_v=256):
3         super().__init__()
4
5         # Query / Key / Value 的线性投影
6         # W_Q: (d_model -> d_k)
7         # W_K: (d_model -> d_k)
8         # W_V: (d_model -> d_v)
9         self.q_proj = nn.Linear(d_model, d_k)
10        self.k_proj = nn.Linear(d_model, d_k)
11        self.v_proj = nn.Linear(d_model, d_v)
12
13        # 输出映射 (d_v -> d_model)
14        self.out_proj = nn.Linear(d_v, d_model)
15
16    def forward(self, x):
17        # x: (B, T, d_model)
18        B, T, _ = x.shape
19
20        # 1. 线性映射得到 Q, K, V
21        Q = self.q_proj(x) # (B, T, d_k)
22        K = self.k_proj(x) # (B, T, d_k)
23        V = self.v_proj(x) # (B, T, d_v)
24
25        # 2. 计算注意力分数 QK^T
26        scores = Q @ K.transpose(-2, -1) # (B, T, T)
27
28        # scaled dot-product
29        scores = scores / (Q.shape[-1] ** 0.5) # sqrt(d_k)
30
31        # 3. 因果掩码: 对于第 i 行, 只允许看到 j <= i 的位置
32        mask = torch.tril(torch.ones(T, T, device=x.device)) # 生成 T×T 的下三角全 1 矩阵
33        scores = scores.masked_fill(mask == 0, float("-inf"))
34
35        # 4. softmax
36        attn = F.softmax(scores, dim=-1) # (B, T, T)
37
38        # 5. 加权求和
39        out = attn @ V # (B, T, d_v)
40
41        # 6. 输出投影
42        return self.out_proj(out) # (B, T, d_model)

```

代码 9.1: 基于 PyTorch 的自注意力实现

9.3 Decoder-only Transformer

Decoder-only Transformer 是当前大语言模型 (Large Language Models, LLMs) 的主流架构，其核心特点是仅由 Transformer 的解码器 (Decoder) 堆叠而成，并通过自回归方式进行序列建模。在该结构中，模型以自左向右的方式逐步预测下一个词元，即在给定历史上下文 $x_{1:i-1}$ 的条件下建模 $P(x_i | x_{<i})$ 。

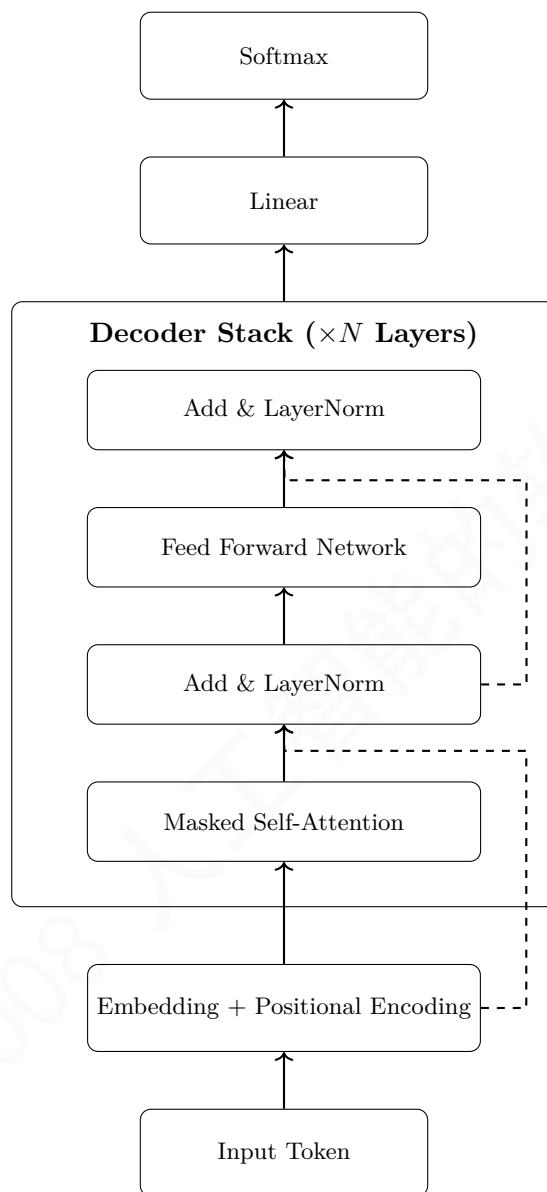


图 9.3: Decoder-only Transformer 结构示意图

如图 9.3所示，Decoder-only 结构由多层相同的 Decoder Block 堆叠而成。每个 Block 包含 Masked Self-Attention 与 Feed-Forward Network 两个子层，并结合残差连接与 LayerNorm 以稳定深层网络的训练过程。其中，Masked Self-Attention 通过因果掩码限制信息流，使得每个 token 仅能访问自身及其之前的历史 token，从而满足自回归建模的条件。

代码 9.2给出了单个 Decoder Block 对应的 PyTorch 实现，其结构与图 9.3严格一致。

```
1 class DecoderBlock(nn.Module):
2     def __init__(self, d_model=256, d_k=256, d_v=256, d_ff=1024):
3         super().__init__()
4
5         # Masked Self-Attention
6         self.attn = SelfAttention(d_model, d_k, d_v)
7
8         # Feed-Forward Network
9         self.ffn = FeedForward(d_model, d_ff)
10
11        # LayerNorm (Post-Norm)
12        self.ln1 = nn.LayerNorm(d_model)
13        self.ln2 = nn.LayerNorm(d_model)
14
15    def forward(self, x):
16        # x: (B, T, d_model)
17
18        # 1. Attention → Residual → LayerNorm
19        x = self.ln1(x + self.attn(x))
20
21        # 2. FFN → Residual → LayerNorm
22        x = self.ln2(x + self.ffn(x))
23
24        return x
25
26
27 class FeedForward(nn.Module):
28     def __init__(self, d_model, d_ff):
29         super().__init__()
30         # Position-wise FFN: (d_model → d_ff → d_model)
31         self.net = nn.Sequential(
32             nn.Linear(d_model, d_ff),
33             nn.ReLU(),
34             nn.Linear(d_ff, d_model)
35         )
36
37    def forward(self, x):
38        # x: (B, T, d_model)
39        return self.net(x)
```

代码 9.2: Decoder Block 的 PyTorch 实现 (Post-Norm 结构)

代码 9.3给出了 Decoder-only Transformer 的整体实现。在输入端，模型首先将离散的词元序列映射为连续的向量表示，并叠加位置信息以引入序列顺序。其中，token embedding 的实现可视为对 one-hot 表示的一种高效替代：设词表大小为 V ，embedding 矩阵为 $W \in \mathbb{R}^{V \times d}$ ，则一个词元的 one-hot 向量与 W 相乘即可得到其向量表示，而在实际实现中，`nn.Embedding` 通过查表操作直接取出矩阵对应行，从而避免显式构造 one-hot 向量并提高计算效率。随后，表示向量依次通过多个 Decoder Block 进行逐层特征变换。在输出端，模型通过 LayerNorm 对最终隐藏状态进行归一化，再映射到词表空间得到 logits 分布，用于预测下一个词元的概率分布。

```

1 class DecoderOnlyTransformer(nn.Module):
2     def __init__(
3         self, vocab_size, d_model=256, d_k=256, d_v=256, d_ff=1024, num_layers=6, max_len=512
4     ):
5         super().__init__()
6
7         # 词嵌入：将离散 token id 映射为连续向量表示；one-hot 的等价实现（查表代替矩阵乘法）
8         self.token_emb = nn.Embedding(vocab_size, d_model)
9
10        # 位置嵌入：为序列位置提供可学习的位置信息
11        self.pos_emb = nn.Embedding(max_len, d_model)
12
13        # Decoder Blocks 堆叠
14        self.blocks = nn.ModuleList([
15            DecoderBlock(d_model, d_k, d_v, d_ff) for _ in range(num_layers)
16        ])
17
18        # 输出层（映射到词表空间）
19        self.lm_head = nn.Linear(d_model, vocab_size, bias=False)
20
21    def forward(self, idx):
22        # B 为 batch size, T 为序列长度
23        # idx_{b,t} 表示第 b 个样本第 t 个位置的 token id, 取值范围为 [0, vocab_size)
24        B, T = idx.shape
25
26        # 位置索引: (B, T)
27        pos = torch.arange(T, device=idx.device).unsqueeze(0) # 生成 [[0, 1, 2, ..., T-1]]
28        pos = pos.expand(B, T) # 扩展到 (B, T)
29
30        x = self.token_emb(idx) + self.pos_emb(pos)
31
32        # 通过多层 Decoder Block
33        for block in self.blocks:
34            x = block(x)
35
36        # logits: (B, T, vocab_size)
37        logits = self.lm_head(x)
38
39        return logits

```

代码 9.3: Decoder-only Transformer 的 PyTorch 实现

9.4 模型训练与测试

9.4.1 模型训练

本节实现基于 Decoder-only Transformer 的语言模型训练流程。模型采用下一个词元预测作为训练目标，通过最大化序列的条件概率分布 $P(x_t | x_{<t})$ 来学习语言建模能力。训练过程主要分为两个阶段：

- 数据准备：对原始文本进行词元化，并构造固定长度的训练样本；
- 模型训练：基于定长数据样本进行自回归训练。

代码 9.4 实现了数据读取与训练样本构造过程。首先从本地文件 `shakespeare.txt`¹ 中读取原始语料，并基于字符级 tokenizer 将文本映射为离散的 token id 序列，从而将自然语言转化为可计算的整数表示。在此基础上，采用滑动窗口方式从长序列中截取固定长度为 T （即 `block_size`）的连续片段，构造训练样本对 (x, y) 。其中输入序列 x 表示当前位置的上下文 token 序列，而目标序列 y 为其在时间维度上右移一位后的对齐结果，即 $y_t = x_{t+1}$ ，用于监督模型学习 next-token prediction 任务。最后，DataLoader 将这些样本组织为 mini-batch 形式，以支持并行计算与稳定训练，从而提升模型在大规模语料上的优化效率。

```
1 with open("shakespeare.txt", "r", encoding="utf-8") as f:
2     text = f.read()
3
4 chars = sorted(list(set(text))) # 字符级 tokenizer
5 vocab_size = len(chars)
6
7 char2idx = {ch: i for i, ch in enumerate(chars)}
8 idx2char = {i: ch for i, ch in enumerate(chars)}
9
10 # 文本 → token id 序列
11 data = torch.tensor([char2idx[ch] for ch in text], dtype=torch.long)
12
13 # 构造训练样本 (x, y)
14 class NextTokenDataset(Dataset):
15     def __init__(self, data, block_size):
16         self.data = data
17         self.block_size = block_size
18
19     def __len__(self):
20         return len(self.data) - self.block_size - 1
21
22     def __getitem__(self, idx):
23         x = self.data[idx : idx + self.block_size] # 从长序列中截取长度为 block_size 的输入序列 x
24         y = self.data[idx + 1 : idx + self.block_size + 1] # 目标序列 y 是 x 的“右移一位版本”
25         return x, y
26
27 block_size = 128
28 batch_size = 32
29 dataset = NextTokenDataset(data, block_size)
30 dataloader = DataLoader(dataset, batch_size=batch_size, shuffle=True)
```

代码 9.4: 文本数据读取与训练样本构造

¹<https://raw.githubusercontent.com/karpathy/char-rnn/master/data/tinyshakespeare/input.txt>

代码 9.5 实现了 Decoder-only Transformer 的训练过程。模型接收输入序列 x 并输出每个位置的词表 logits，通过交叉熵损失函数与目标序列 y 计算误差，并使用 AdamW 优化器进行参数更新。整个训练过程以 mini-batch 方式进行，从而提升训练效率与稳定性。

```
1 model = DecoderOnlyTransformer(vocab_size=vocab_size, d_embed=256, d_ff=1024,
2     num_layers=6, max_len=block_size).to(device)
3 optimizer = torch.optim.AdamW(model.parameters(), lr=3e-4, weight_decay=1e-2)
4 num_epochs = 10
5
6 # 训练循环
7 for epoch in range(num_epochs):
8     model.train()
9     total_loss = 0.0
10
11     for x, y in dataloader:
12         x = x.to(device)
13         y = y.to(device)
14
15         # 前向传播
16         logits = model(x) # (B, T, V)
17
18         # Cross Entropy Loss
19         loss = F.cross_entropy(
20             logits.view(-1, logits.size(-1)), # (B*T, V)
21             y.view(-1)                        # (B*T,)
22         )
23
24         # 反向传播与参数更新
25         optimizer.zero_grad()
26         loss.backward()
27         optimizer.step()
28
29         total_loss += loss.item()
30
31     print(f"Epoch {epoch+1}, loss = {total_loss / len(dataloader):.4f}")
```

代码 9.5: Decoder-only Transformer 的训练过程

9.4.2 模型测试

在模型训练完成后，我们通过自回归方式进行文本生成测试。代码 9.6 实现了基于贪心解码的生成过程。模型首先将输入提示文本编码为词元序列，然后在上下文窗口 T 的限制下逐步预测下一个词元，并将预测结果拼接回输入序列中，实现逐步生成。生成过程可在达到最大生成步数或满足终止条件（如生成 EOS token）时提前停止。

```
1 def generate(model, prompt, char2idx, idx2char, max_new_tokens=200, block_size=128):
2     model.eval()
3
4     # 将输入文本编码为 token id 序列
5     idx = torch.tensor([char2idx[c] for c in prompt], dtype=torch.long).unsqueeze(0) # (1, T)
6
7     with torch.no_grad():
8         for _ in range(max_new_tokens):
9
10            # 截取最近 block_size 个 token 作为上下文窗口
11            idx_cond = idx[:, -block_size:] # (1, T)
12
13            # 前向传播得到 logits
14            logits = model(idx_cond) # (1, T, V)
15
16            # 仅取最后一个时间步的预测分布
17            logits = logits[:, -1, :] # (1, V)
18
19            # greedy decoding: 选择概率最大的 token
20            probs = torch.softmax(logits, dim=-1)
21            next_idx = torch.argmax(probs, dim=-1, keepdim=True) # (1, 1)
22
23            # 如果生成 EOS token, 则提前停止
24            if next_idx.item() == char2idx[eos_token]:
25                break
26
27            # 拼接回输入序列
28            idx = torch.cat([idx, next_idx], dim=1)
29
30            # 解码为文本
31            out = "".join([idx2char[i] for i in idx[0].tolist()])
32        return out
```

代码 9.6: Decoder-only Transformer 的文本生成

练习

- 1) 分析 Decoder-only Transformer 的参数量构成，并推导其随模型维度 (d_{model})、层数 (N) 以及前馈隐藏维度 (d_{ff}) 变化的规模关系。
- 2) 在 Kaggle 或本地环境中实现一个简化版 Decoder-only Transformer (可基于字符级或子词级 tokenizer)，完成训练与文本生成任务，并分析不同超参数 (如层数、上下文长度、embedding 维度) 对训练效率和生成效果的影响。

10 大模型训练

本章学习目标：

- ** 理解大模型训练的主要挑战
- *** 理解 LoRA 的原理
- ** 了解并行训练的思想：Data Parallel、Tensor Parallel 和 Pipeline Parallelism

10.1 大模型的训练挑战

随着深度学习模型规模的不断扩大，LLM 的训练面临日益复杂的计算与系统挑战。相比传统深度学习模型，大模型通常具有数十亿甚至数千亿参数，其训练过程对计算资源、存储资源以及通信效率均提出了更高要求。由于模型参数规模巨大，单个计算设备通常难以完成训练，因此需要依赖多 GPU 或 TPU 集群进行分布式训练。同时，模型参数、梯度以及中间激活值会占用大量显存，并带来显著的存储与内存带宽开销。此外，在数据并行与模型并行等训练范式中，不同计算设备之间需要频繁进行参数与梯度同步，通信延迟与带宽限制逐渐成为制约训练效率的关键瓶颈。另一方面，大模型训练通常具有较长的训练周期与较高的成本。即使采用高性能硬件与多种优化策略，完成一次大规模预训练仍可能需要数周甚至数月时间。因此，如何降低训练成本、提高训练效率以及减少显存占用，成为当前大模型训练研究的重要方向。

公司	模型	发布时间	参数量	上下文长度	训练 Tokens	训练规模与成本
OpenAI	GPT-3 [1]	2020	175B	2K	~1T 级别	数千 GPU (V100/A100)；训练计算量约 $\sim 3 \times 10^{23}$ FLOPs (估计)；数百万美元级训练成本
Meta	LLaMA 3 [2]	2024	8B / 70B / 405B	8K-128K	~10T 级别	万卡级 GPU 集群
DeepSeek	DeepSeek-V3 [3]	2024	671B (37B activated)	128K	~10T 级别	千卡级 H800 GPU 集群；训练成本为数百万 GPU-hours 级别

表 10.1: 代表性大模型训练系统参数对比（公开信息与工程估计）

当前互联网数据中真正可用于大模型训练的高质量文本数据相对有限。研究表明 [4]，在经过清洗、去重、质量过滤以及版权筛查后，可用于大规模语言模型预训练的高质量公开文本数据规模大约为数十万亿 (10-100T) 词元量级。例如，FineWeb 数据集包含约 15T 英文词元¹；RedPajama V2 数据集则达到约 30T 词元规模²。

本章主要从两类技术路径展开讨论：一类是参数高效微调方法，通过仅更新少量参数以降低模型微调的计算与存储开销；另一类是并行与分布式训练技术，通过数据并行、模型并行以及流水线并行等方式提升大模型训练效率。

¹FineWeb: <https://huggingface.co/datasets/HuggingFaceFW/fineweb>.

²RedPajama V2: <https://github.com/togethercomputer/RedPajama-Data>.

10.2 参数高效微调

参数高效微调 (Parameter-Efficient Fine-Tuning, PEFT) 技术, 通过仅训练少量额外参数, 在保持模型性能的同时大幅降低资源消耗。本节主要讲基于 LoRA 的参数高效微调技术。本节主要讲 Low-Rank Adaptation (LoRA) 这种方法。

基于如下经验观察: 在下游任务微调过程中, 预训练大模型权重的有效更新 ΔW 往往具有较低的内在秩 (low intrinsic rank), 即模型性能的提升主要依赖于少数几个主方向上的参数变化。因此, 无需学习完整的 ΔW , 而可以用低秩矩阵对其进行近似, 从而显著减少可训练参数数量。

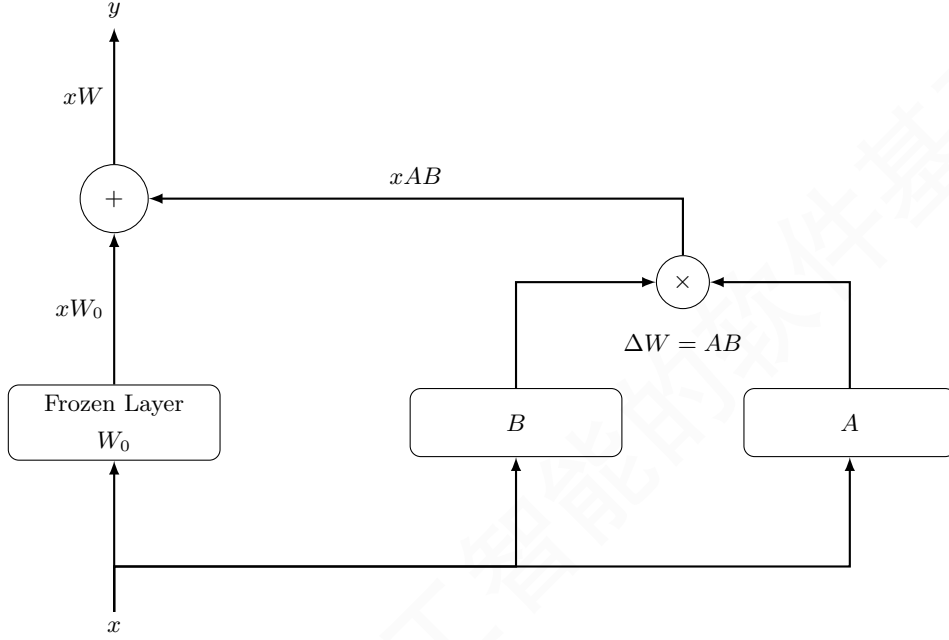


图 10.1: LoRA 原理示意图

如图 10.1所示, LoRA 不直接更新完整的权重矩阵 W , 而是在冻结原始预训练参数 W_0 的基础上, 引入两个低秩矩阵 A 与 B 来表示权重增量:

$$\Delta W = AB$$

其中, 若输入向量为

$$x \in \mathbb{R}^{1 \times d},$$

原始权重矩阵为

$$W_0 \in \mathbb{R}^{d \times k},$$

则通常令

$$A \in \mathbb{R}^{d \times r}, \quad B \in \mathbb{R}^{r \times k},$$

其中 $r \ll \min(d, k)$ 为低秩维度 (rank)。

因此:

$$\Delta W = AB \in \mathbb{R}^{d \times k},$$

新的权重可表示为:

$$W = W_0 + \Delta W = W_0 + AB.$$

在前向传播过程中, 模型输出变为:

$$xW = xW_0 + xAB.$$

其中：

- xW_0 由冻结的预训练参数计算得到；
- xAB 表示 LoRA 学习到的低秩增量部分。

由于仅需训练矩阵 A 与 B ，可训练参数数量从原本的

$$d \times k$$

降低为

$$r(d + k),$$

当 r 很小时，参数规模与显存占用都将显著下降。例如，当 $d = k = 4096$ 、 $r = 8$ 时，可训练参数量仅约为原来的 0.76%。

LoRA 的另一个重要优点是：其不会修改原始预训练模型参数，因此不同任务对应的 LoRA 权重可以独立保存与切换。实际部署时，只需加载基础模型与对应的 LoRA 参数即可快速适配不同任务，从而大幅降低存储与部署成本。

在 Transformer 中，LoRA 通常应用于注意力机制中的线性投影层，例如 Query、Key、Value 或 Output Projection 等模块。其中最常见的是对 Query 与 Value 矩阵进行低秩适配，因为实验表明这些位置对下游任务性能最为敏感。

此外，为了稳定训练，LoRA 通常还会引入缩放系数 α ，将权重更新写为：

$$W = W_0 + \frac{\alpha}{r} AB.$$

其中 α 用于控制低秩更新项的幅度，从而避免训练初期对原模型分布造成过大扰动。

总体而言，LoRA 通过“冻结大部分参数 + 仅学习低秩增量”的方式，在保持模型性能的同时，显著降低了微调过程中的参数量、显存消耗与训练成本，因此已成为当前大模型微调中最常用的参数高效微调方法之一。

10.3 并行和分布式训练

随着大语言模型参数规模不断增长,单张 GPU 已难以满足训练所需的显存与计算需求。例如,一个 7B 参数规模的 Transformer 模型,若采用 FP16 精度,仅模型参数本身就需要约: $7 \times 10^9 \times 2 \text{ Bytes} \approx 14 \text{ GB}$ 。并且训练过程中还需要额外存储梯度和优化器状态等信息,因此实际训练显存往往会达到参数量的数倍。

名称	并行方式	说明
数据并行 (Data Parallel)	训练数据	每张 GPU 持有完整模型副本, 处理不同数据 batch
全分片数据并行 (Fully Sharded Data Parallel)	存储参数	参数与梯度等信息在多 GPU 间分片存储, 计算时按需聚合
张量并行 (Tensor Parallel)	(横向) 算子计算	将单层算子按张量维度切分到多 GPU 上并行计算
流水线并行 (Pipeline Parallel)	(纵向) 层级并行	将模型按层划分为多个 stage, 不同 GPU 负责不同层的计算

表 10.2: 主流分布式训练并行方式对比

10.3.1 数据并行

在单机多卡场景中,如果单张 GPU 能够容纳完整模型参数,则可以通过数据并行加速训练。数据并行的基本思想是: 每张 GPU 持有一份完整模型副本,并分别处理不同的 mini-batch 数据,从而提升整体训练吞吐量,缩短训练时间。

需要注意的是,数据并行并不能解决模型过大导致的显存不足问题,因为每张 GPU 仍需存储完整模型参数与优化器状态。

DataParallel (DP) PyTorch 提供的基础数据并行接口为 `torch.nn.DataParallel`, 其使用方式如下:

```
1 model = torch.nn.DataParallel(model)
2 model = model.cuda()
```

其执行流程如下:

- 主 GPU 负责参数广播与结果汇总;
- 输入 mini-batch 被拆分并分发到多个 GPU;
- 各 GPU 独立执行前向与反向传播;
- 各 GPU 的梯度在主 GPU 上聚合后进行参数更新。

该方法实现简单、代码改动少,适合小规模实验。但由于存在主卡通信与汇总瓶颈,扩展性较差,因此已逐渐不被推荐用于正式训练任务。

Distributed Data Parallel (DDP) 目前 PyTorch 官方推荐使用 DDP 作为标准数据并行方式。DDP 基于 NVIDIA 的 NCCL (NVIDIA Collective Communications Library) 后端实现高效 GPU 间通信,采用去中心化的 All-Reduce 机制进行梯度同步,从而避免传统参数服务器架构中的中心节点瓶颈,具有更好的扩展性与计算效率。

其典型训练流程如下:

```

1 import os
2 import torch
3 import torch.nn as nn
4 import torch.distributed as dist
5 from torch.nn.parallel import DistributedDataParallel as DDP
6 from torch.utils.data import DataLoader
7 from torch.utils.data.distributed import DistributedSampler
8
9 # 1. 初始化分布式环境
10 dist.init_process_group(backend="nccl")
11
12 # 2. 绑定当前进程与GPU
13 local_rank = int(os.environ["LOCAL_RANK"])
14 torch.cuda.set_device(local_rank)
15 device = torch.device("cuda", local_rank)
16
17 # 3. 构建模型并封装DDP
18 model = LLM().to(device)
19 model = DDP(model, device_ids=[local_rank])
20
21 # 4. 数据划分
22 dataset = MyDataset()
23 sampler = DistributedSampler(dataset, shuffle=True)
24 loader = DataLoader(dataset, batch_size=32, sampler=sampler)
25
26 # 5. 优化器与损失函数
27 optimizer = torch.optim.Adam(model.parameters(), lr=1e-4)
28 criterion = nn.CrossEntropyLoss()
29
30 # 6. 训练循环
31 for epoch in range(3):
32     sampler.set_epoch(epoch)
33     for x, y in loader:
34         x, y = x.to(device), y.to(device)
35         logits = model(x)
36         loss = criterion(logits, y)
37         optimizer.zero_grad()
38         loss.backward() # DDP 自动触发 All-Reduce 同步梯度
39         optimizer.step()
40
41 # 7. 清理环境
42 dist.destroy_process_group()

```

通常通过 `torchrun` 启动多进程训练：

```
1 torchrun --nproc_per_node=2 train.py
```

其中：

- `nproc_per_node` 表示使用的 GPU 数量；
- 每张 GPU 对应一个独立进程；
- PyTorch 自动完成进程间通信初始化。

10.3.2 参数并行

在大语言模型训练中，模型参数规模通常远超单卡显存容量，因此无法通过标准的数据并行方式进行扩展。参数相关的并行方法本质上是通过切分模型中的张量降低单卡的存储或计算压力。但需要注意的是，不同方法切分的对象与目的并不相同。

在 Decoder-only Transformer 中，参数主要集中在 Self-Attention 和 Feed-Forward 网络的线性变换中，例如：

$$W_Q, W_K, W_V \in \mathbb{R}^{d_{\text{model}} \times (H d_k)}$$

其中 H 表示 attention head 数量。这一结构使得模型天然具有“按 head 或按矩阵维度切分”的可能性。

Fully Sharded Data Parallel (FSDP) FSDP 的核心思想是对“参数存储”进行切分，而不是改变计算方式。在实现上，模型所有参数（包括 attention 与 FFN 层）会被 flatten 成连续向量，并在数据并行维度上均匀切分，使得每张 GPU 仅保存约 $\frac{1}{N}$ 的参数、梯度以及优化器状态。

在前向传播时，当某一层参与计算（例如 attention 层中的 Q/K/V 线性映射），其参数分片会通过 all-gather 临时恢复为完整矩阵：

$$q_i = x_i W_Q, \quad k_i = x_i W_K, \quad v_i = x_i W_V$$

计算完成后立即释放完整参数，仅保留本地分片。在反向传播过程中，梯度通过 reduce-scatter 操作同步并重新切分。

因此，FSDP 可以理解为：模型参数被按存储切开，计算时再临时拼回完整形式，计算完成后重新分片存储。其本质目标是降低显存占用，使模型可以在多 GPU 上存得下。

Tensor Parallelism (TP) 与 FSDP 不同，Tensor Parallel 并不以“存储切分”为核心，而是直接对计算过程进行切分。其基本思想是：将同一层中的矩阵运算拆分到不同 GPU 上独立计算，而不是先恢复完整参数再计算。

以 attention 中的线性映射为例，输出维度通常按 head 进行划分：

$$W_Q, W_K, W_V \in \mathbb{R}^{d_{\text{model}} \times (H d_k)}$$

在 Tensor Parallel 中，这些矩阵可以沿 head 维度或列维度拆分为多个子矩阵，例如：

$$W_Q = [W_Q^{(1)}, W_Q^{(2)}, \dots, W_Q^{(N)}]$$

每张 GPU 只负责计算对应部分：

$$q_i^{(n)} = x_i W_Q^{(n)}$$

最终再对各个 head 的输出进行拼接或聚合。

因此，Tensor Parallel 可以理解为：模型参数不再作为整体存在，而是直接参与分布式计算，每张 GPU 只完成一部分矩阵乘法。其本质目标是降低单层计算压力，使大矩阵乘法可以在多 GPU 上并行执行。

Pipeline Parallelism (PP) PP 从“模型深度”维度对模型进行划分。其核心思想是：将一个深层 Decoder-only Transformer 按层划分为多个连续的子网络（stage），并分配到不同 GPU 上执行。

以标准 Transformer 为例，模型由多层 Decoder Block 堆叠而成：

$$\text{Block}_1 \rightarrow \text{Block}_2 \rightarrow \dots \rightarrow \text{Block}_N$$

在 Pipeline Parallel 中，这些层被划分为若干段，例如：

$$\{\text{Block}_1 \sim \text{Block}_k\}, \quad \{\text{Block}_{k+1} \sim \text{Block}_{2k}\}, \quad \dots$$

每个 GPU 只负责其中一段网络的前向与反向计算。

在前向传播过程中，输入 token 表示首先进入第一段计算（如前几层 attention 与 FFN），随后其输出作为激活值传递给下一 GPU，并继续经过后续层处理，直到得到最终 logits。这一过程类似于工业生产中的流水线，因此称为“pipeline”。

10.3.3 实际训练中的典型组合

在当前主流大语言模型训练中，通常不会单独使用某一种并行方式，而是将多种技术组合使用，以同时缓解显存、计算与通信三方面的瓶颈。一般而言，DDP 或 FSDP 用于实现数据并行与参数状态分片，Tensor Parallel 用于切分单层矩阵计算以提升算子级并行度，Pipeline Parallel 用于将深层 Transformer 按层划分到不同 GPU，从而扩展模型在深度方向上的可训练规模。此外，还常配合多种系统级优化策略，例如 ZeRO 用于进一步降低优化器状态与梯度的显存占用，FP16/BF16 混合精度用于减少计算与存储开销，而 Gradient Checkpointing 则通过以计算换显存的方式降低前向激活的存储压力。总体而言，现代大语言模型训练已经不再是单纯的模型优化问题，而是一个高度系统化的分布式工程问题，其核心约束来自 GPU 间通信效率、显存层级管理、参数与计算的切分策略以及集群网络带宽等多个因素的综合权衡。

10.4 使用 Hugging Face 训练大模型

Hugging Face³ 已成为当前自然语言处理乃至整个 AI 领域最重要的开源生态之一。它不仅提供数以万计的预训练模型与公开数据集，还构建了一套完整的大模型开发工具链，极大降低了大模型训练与部署门槛。

Hugging Face 生态主要由多个 Python 库组成，并与 PyTorch 深度集成：

- **transformers**：提供 Transformer 模型实现与预训练权重，支持 LLaMA 等主流模型；
- **datasets**：提供统一的数据集加载与预处理接口，支持流式读取与大规模数据处理；
- **tokenizers**：提供高性能 tokenizer 实现，用于文本与词元序列之间的转换；
- **peft**：提供参数高效微调（PEFT）方法，包括 LoRA、Prefix Tuning、Prompt Tuning 等；
- **Trainer**：提供高层训练接口，自动封装训练循环、梯度更新、mixed precision、checkpoint 保存以及 DDP、FSDP 等分布式训练功能。

Hugging Face 提供了一套端到端的大模型开发与训练框架，涵盖模型加载、数据处理、分布式训练、参数高效微调（PEFT）以及模型发布等环节。研究者与工程人员可以专注于模型结构与任务本身，而无需手动实现数据并行、混合精度训练、梯度检查点等底层机制。

³Hugging Face 在线平台：<https://huggingface.co>

下面以 LLaMA-3 模型为例，演示如何使用 Hugging Face 与 PEFT 对大语言模型进行 LoRA 微调。

```
1 from transformers import AutoModelForCausalLM, AutoTokenizer, TrainingArguments, Trainer
2 from peft import LoraConfig, get_peft_model
3 from datasets import load_dataset
4 import torch
5
6 # AutoTokenizer: 自动加载与 LLaMA-3 对应的 tokenizer
7 tokenizer = AutoTokenizer.from_pretrained("meta-llama/Meta-Llama-3-8B")
8 # AutoModelForCausalLM: 自动加载 Decoder-only Transformer
9 model = AutoModelForCausalLM.from_pretrained("meta-llama/Meta-Llama-3-8B",
10       torch_dtype=torch.bfloat16 # BF16 混合精度降低显存与计算开销
11 )
12 # 配置 LoRA (Low-Rank Adaptation)
13 # r: 低秩矩阵 rank, 决定新增参数规模
14 # lora_alpha: LoRA 缩放系数, 用于控制低秩更新幅度
15 # target_modules: 指定在哪些线性层插入 LoRA adapter
16 peft_config = LoraConfig(r=8, lora_alpha=32,
17       target_modules=["q_proj", "v_proj"],
18       task_type="CAUSAL_LM"
19 )
20 # 注入 LoRA adapter: 原始预训练参数默认被冻结, 仅 LoRA 参数参与训练
21 model = get_peft_model(model, peft_config)
22 # 输出可训练参数比例
23 model.print_trainable_parameters()
24
25 # 从 Hugging Face Hub 加载 instruction tuning 数据集
26 dataset = load_dataset("databricks/databricks-dolly-15k")
27
28 # 训练参数
29 training_args = TrainingArguments(
30     output_dir="./llama3-lora",
31     per_device_train_batch_size=1, # 单张 GPU 的 batch size
32     gradient_accumulation_steps=8, # 梯度累积: 每 8 个 step 更新一次参数, 模拟更大的 batch size
33     bf16=True, # 启用 BF16 混合精度训练
34     gradient_checkpointing=True, # backward 时重新计算部分 activation
35     logging_steps=10,
36     save_steps=500,
37     num_train_epochs=1
38 )
39
40 # Hugging Face 高层训练接口
41 trainer = Trainer(model=model, args=training_args, train_dataset=dataset["train"])
42 # 启动训练
43 trainer.train()
44 # 保存 LoRA adapter (仅包含低秩增量参数)
45 model.save_pretrained("./llama3-lora-adapter")
46 # merge LoRA 权重回原始模型
47 merged_model = model.merge_and_unload()
48 # 保存 merge 后的完整模型
49 merged_model.save_pretrained("./llama3-lora-merged")
```

代码 10.1: 使用 Hugging Face + PEFT 对 LLaMA-3 进行 LoRA 微调

上述代码展示了当前主流的大模型参数高效微调流程。由于 LoRA 仅训练少量低秩参数，因此相比全参数微调，其显存占用与训练成本都会显著降低。

使用 FSDP 在 `TrainingArguments` 中增加如下配置：

```
1 training_args = TrainingArguments(  
2     ...  
3     # FSDP: 对参数、梯度与 optimizer state 进行分片存储  
4     fsdp="full_shard auto_wrap",  
5     # 自动对 Transformer Layer 进行 FSDP 包装  
6     fsdp_transformer_layer_cls_to_wrap="LlamaDecoderLayer",  
7 )
```

代码 10.2: 在 Hugging Face Trainer 中启用 FSDP

通常可通过如下命令启动多 GPU 分布式训练：

```
1 torchrun --nproc_per_node=4 train.py
```

使用 TP TP 在 Hugging Face 的 Trainer 中通常不直接支持，而是依赖底层大规模分布式训练框架实现，例如微软的 DeepSpeed⁴ 或 NVIDIA 的 Megatron-LM⁵。以 DeepSpeed 为例，在实际使用中通常通过配置文件启用对应的并行策略，而无需对模型结构进行手动修改。例如：

```
1 training_args = TrainingArguments(  
2     ...  
3     deepspeed="ds_config.json"  
4 )
```

使用 PP 在 Hugging Face 中可以直接通过设备映射方式将不同 Transformer 层分配到不同 GPU 上执行。例如：

```
1 from accelerate import dispatch_model  
2 from transformers import AutoModelForCausalLM  
3  
4 model = AutoModelForCausalLM.from_pretrained("meta-llama/Meta-Llama-3-8B")  
5  
6 device_map = {  
7     "model.layers.0": 0,  
8     "model.layers.1": 0,  
9     "model.layers.2": 1,  
10    "model.layers.3": 1  
11 }  
12  
13 model = dispatch_model(model, device_map=device_map)
```

⁴<https://github.com/deepspeedai/deepspeed>

⁵<https://github.com/nvidia/megatron-lm>

参考文献

- [1] Tom Brown et al. “Language Models are Few-Shot Learners.” *NeurIPS*, 2020.
- [2] AI at Meta. “The LLaMA 3 Herd of Models.” *arXiv preprint arXiv:2407.21783*, 2024.
- [3] Aixin Liu et al. “DeepSeek-V3 Technical Report.” *arXiv preprint arXiv:2412.19437*, 2024.
- [4] Villalobos, et al. “Position: Will we run out of data? Limits of LLM scaling based on human-generated data.” *ICML*, 2024.

练习

- 1) 使用 PyTorch DDP 对 LeNet-5 在 MNIST 数据集上进行分布式训练，对比不同 GPU 数量下的训练时间与模型准确率。
- 2) 使用 Hugging Face 与 PEFT 框架，对 LLaMA-3-8B 或其它开源 Transformer 模型进行 LoRA 微调，尝试修改 LoRA rank（如 $r = 4, 8, 16$ ），并对比不同 rank 下的可训练时间、显存占用以及模型效果。

11 大模型推理

本章学习目标：

- ★★★ 理解模型参数量化的基本原理
- ★★★ 理解 KV Cache 在自回归推理中的加速机制
- ★★★ 理解 FlashAttention 的计算与内存优化思想
- ★★ 了解 vLLM 和 SGLang 工具和关键优化

由于大模型参数规模庞大且中间计算状态占用显存较高，其推理过程往往受到内存带宽与计算资源的双重限制，导致延迟增加与吞吐下降。因此，提升推理效率成为大模型系统工程中的核心问题之一。本节主要介绍三种主流的推理优化技术：参数量化、KV Cache 以及 FlashAttention。

11.1 参数量化

参数量化是通过降低模型参数与计算中间变量的数值精度，从而减少显存占用并提升推理效率的重要技术。其核心思想是在尽可能保持模型性能的前提下，将高精度浮点表示（如 FP32）映射为低比特浮点数甚至整数表示。

11.1.1 整数与浮点数

在深度学习系统中，数值类型不仅决定了模型参数的存储开销，也直接影响硬件计算复杂度、数值精度以及推理效率。常见的数据格式主要包括浮点数与整数两类。

整数 整数类型采用定点 (Fixed-Point) 表示方式，即每一个比特位具有固定的数值含义。对于有符号整数 (Signed Integer)，通常采用二进制补码 (Two's Complement) 表示，其最高位为符号位 (0 表示正数，1 表示负数)；其余位用于表示整数数值。

以 INT32 的整数 7 为例，其二进制表示为：

00000000 00000000 00000000 00000111

负数通常采用二进制补码表示。对于-7，首先写出正数 7 的二进制，随后按位取反

11111111 11111111 11111111 11111000

再加 1 即可得到：

11111111 11111111 11111111 11111001

因此，对于一个 n bit 的有符号整数，其可表示范围为： $[-2^{n-1}, 2^{n-1} - 1]$

浮点数 浮点数 (Floating-Point) 可以支持小数或更大的数值范围，其基本结构通常为：

$$\text{Value} = (-1)^s \times m \times 2^e$$

其中：

- s : 符号位 (Sign Bit)
- e : 指数 (Exponent)
- m : 尾数 (Mantissa 或 Fraction)

FP32 (单精度浮点数) 是传统深度学习训练的标准格式, 其结构如下:

$$1 \text{ bit Sign} + 8 \text{ bit Exponent} + 23 \text{ bit Mantissa}$$

以十进制小数 0.1 为例, 将其转换为 FP32 需要首先将其转换为二进制小数:

$$0.1_{10} = 0.00011001100110011 \dots_2$$

可以看到, 0.1 在二进制中是无限循环小数, 因此无法被 FP32 精确表示。将其规格化:

$$0.0001100110011 \dots = 1.1001100110011 \dots \times 2^{-4}$$

因此得到, 0.1 转化为 FP32 表示后的指数为-4, 加上 IEEE754 标注中规定的偏置 127 后实际指数为 123 (01111011_2), 尾数部分是 1001100110011001100_2 。因此, FP32 中 0.1 的完整表示约为:

$$0 \mid 01111011 \mid 1001100110011001100$$

由于尾数位数有限, FP32 实际保存的并不是精确的 0.1, 而是:

$$0.10000000149011612$$

因此, 浮点数计算通常会存在误差。这也是为什么在深度学习训练中, 经常需要关注数值稳定性问题。表 11.1 总结了常见的一些数据类型。

表 11.1: 常见 FP 与 INT 数据格式结构与数值范围对比

数据类型	总位宽	符号位	指数位	尾数位	数值范围
FP32	32 bit	1	8	23	$\approx \pm 3.4 \times 10^{38}$
FP16	16 bit	1	5	10	$\approx \pm 6.55 \times 10^4$
FP8 (E4M3FN)	8 bit	1	4	3	$\approx [-240, 240]$
FP8 (E5M2)	8 bit	1	5	2	$\approx [-57344, 57344]$
INT32	32 bit	1	—	—	$[-2^{31}, 2^{31} - 1]$
INT16	16 bit	1	—	—	$[-2^{15}, 2^{15} - 1]$
INT8	8 bit	1	—	—	$[-2^7, 2^7 - 1]$
INT4	4 bit	1	—	—	$[-2^3, 2^3 - 1]$

参数量化 参数量化 (Quantization) 是指将浮点数转换为整数表示, 或采用更低比特宽度的数据类型来表示数值的一种模型压缩与加速技术。其主要目标包括两个方面: 一是提升模型推理阶段的运算性能, 二是降低模型的存储开销与显存占用。

相比浮点运算, 整数运算无需处理指数对齐、尾数规格化以及浮点舍入等复杂操作, 因此硬件实现更加简单, 能够在相同芯片资源下实现更高的计算并行度与吞吐率。因此, 将同等位宽的浮点运算转换为整数运算, 通常可以显著提升推理效率。此外, 无论是浮点数还是整数, 采用更低位宽的数据类型都能够进一步降低计算开销。例如, 相比 FP32, FP16、FP8、INT8 以及 INT4 的数据位宽更小, 因此在矩阵乘法过程中能够减少显存带宽压力, 并提高缓存利用率与计算吞吐率。另一方面, 低比特表示还能显著减少

模型参数存储空间。例如，相比 FP16，INT8 的存储开销降低约 50%，而 INT4 则仅需约 25% 的存储空间。这对于大模型部署尤为重要，因为模型权重通常占据主要显存开销。因此，现代大模型系统通常会结合低比特整数计算与混合精度技术，在尽可能保持模型精度的前提下，实现更高的推理速度与更低的部署成本。

当前主流的大模型量化方法多采用训练后量化策略，即在模型训练完成后直接对权重进行量化处理，而无需重新训练模型或对模型结构进行修改。参数量化通常通过缩放与离散映射的方式实现。

- **浮点 → 低精度浮点 (FP32 → FP16 / FP8)**: 该类量化过程不引入缩放操作，仅通过截断尾数位与指数位实现精度压缩，同时保留浮点结构。因此其数值误差主要来源于舍入，而非量化映射误差。
- **浮点 → 整数 (FP → INT8 / INT4)**: 由于同比特浮点数表示的数值范围远大于整数，该类量化需要缩放操作，将浮点空间映射到离散整数空间。数学形式为：

$$q = \text{round} \left(\frac{x}{s} \right)$$

其中：

- x : 原始浮点数
- q : 量化后的整数表示
- s : 缩放因子

从可量化性的角度来看，神经网络主要由线性变换（如矩阵乘法）构成，这类运算在数值上对均匀缩放具有较好的等价性，使得参数在不同精度表示下仍能保持相对稳定的计算行为。同时，归一化 (Normalization) 与 ReLU 等非线性操作通过对激活分布进行重标定或截断，在一定程度上改善了数值分布的稳定性与动态范围，从而降低了低比特量化对模型精度的影响。因此，在上述结构特性共同作用下，模型在低精度表示下通常仍能够保持较好的数值稳定性与表达能力。

11.2 KV Cache

在大语言模型的自回归生成过程中，模型逐个词元生成输出。若每次生成新词元都重新计算所有历史词元的 Key 和 Value，将导致大量重复的线性投影计算。KV Cache (Key-Value Cache) 技术通过缓存历史 Key/Value，使得在生成第 t 个词元时无需重复计算 $x_{1:t-1}$ 的 Key 和 Value，仅需计算当前词元的表示，从而显著降低推理过程中的重复计算开销，是 LLM 推理加速的核心机制。

问题：自回归生成的重复计算 考虑生成第 t 个词元时，输入序列为 $x_{1:t} = [x_1, x_2, \dots, x_t]$ 。标准注意力计算为：

$$\text{Attention}(Q_t, K_{1:t}, V_{1:t}) = \text{softmax}\left(\frac{Q_t K_{1:t}^\top}{\sqrt{d}}\right) V_{1:t}$$

其中， $Q_t = x_t W_Q$ 表示当前词元的 query； $K_{1:t} = [x_1; \dots; x_t] W_K$ 表示历史词元的 key； $V_{1:t} = [x_1; \dots; x_t] W_V$ 表示历史词元的 value。

在自回归生成过程中，当生成第 $t+1$ 个词元时， $K_{1:t}$ 与 $V_{1:t}$ 与上一轮计算结果完全相同，但标准实现仍会重复计算 $K_{1:t+1}$ 和 $V_{1:t+1}$ ，导致计算冗余，并使整体推理复杂度随序列长度呈平方级增长（主要来自 attention score 计算）。

KV Cache 核心思想 将已计算的 Key 和 Value 缓存在显存中，后续步骤直接复用，仅计算当前 token 的 K_t, V_t 。具体做法如下：

- 初始化空缓存： $\text{Cache}_K = []$, $\text{Cache}_V = []$
- 第 1 步：计算 K_1, V_1 ，存入缓存
- 第 2 步：计算 K_2, V_2 ，追加到缓存
- 第 t 步：仅计算 K_t, V_t 并追加到缓存

此时注意力计算变为：

$$\text{Attention}(Q_t, K_{1:t}^{\text{cache}}, V_{1:t}^{\text{cache}})$$

从计算复杂度来看，KV Cache 将每步 Key/Value 的重复投影计算从 $O(td^2)$ 降低为 $O(d^2)$ （仅当前 token），从而显著降低自回归推理开销。然而，其代价是显存开销随序列长度线性增长。对于 L 层模型、隐藏维度 d 、序列长度 T ，KV Cache 的显存需求为：

$$\text{Memory}_{KV} = 2 \times L \times T \times d \times \text{bytes}$$

示例：以 LLaMA 3 8B ($L = 32$, $d = 4096$) 为例，在 $T = 2048$ 、FP16 条件下，如果不考虑其它优化，KV Cache 显存可近似估算为：

$$2 \times 32 \times 2048 \times 4096 \times 2 \approx 1 \text{ GB}$$

11.3 FlashAttention

传统的缩放点积注意力可以写为：

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d}}\right)V. \quad (11.1)$$

其中 $Q, K, V \in \mathbb{R}^{N \times d}$ ，注意力分数矩阵为：

$$S = \frac{QK^\top}{\sqrt{d}} \in \mathbb{R}^{N \times N}.$$

因此，标准注意力在计算过程中需要显式构造并存储一个 $N \times N$ 的中间矩阵。随着序列长度 N 增大，这种二次规模的显存开销会成为 Transformer 的主要瓶颈。FlashAttention 的核心目标是重新组织计算过程，使得该中间矩阵无需完整存储到显存中，而是以分块（block-wise）和流式（online）的方式进行计算与累积。

对于第 i 个 query，其输出可以展开为：

$$O_i = \frac{\sum_{j=1}^N \exp(S_{ij})V_j}{\sum_{j=1}^N \exp(S_{ij})}. \quad (11.2)$$

这里可以看到，注意力由两部分组成：上方是未归一化的加权和，下方是 softmax 的归一化分母。由于二者都是求和结构，因此可以按 block 进行分解：

$$\sum_{j=1}^N = \sum_{b=1}^B \sum_{j \in \mathcal{J}_b}, \quad \{\mathcal{J}_b\}_{b=1}^B \text{ 为对索引的分块划分.}$$

也就是说，如果能够逐块处理 key/value，而不是一次性构造完整的 $N \times N$ 注意力矩阵，就可以避免中间显存的巨大开销。

关键难点 softmax 依赖全局归一化项，并不满足简单的线性可加性。直接计算 $\exp(S_{ij})$ 在数值上容易出现溢出或下溢问题，因此实际实现中利用 softmax 的平移不变性，将其改写为：

$$P_{ij} = \frac{\exp(S_{ij} - m_i)}{\sum_k \exp(S_{ik} - m_i)}, \quad m_i = \max_j S_{ij}. \quad (11.3)$$

于是 attention 输出可写为：

$$O_i = \frac{\tilde{O}_i}{l_i}, \quad (11.4)$$

其中未归一化加权和为：

$$\tilde{O}_i = \sum_j \exp(S_{ij} - m_i)V_j, \quad (11.5)$$

归一化因子为：

$$l_i = \sum_j \exp(S_{ij} - m_i). \quad (11.6)$$

FlashAttention 的关键在于：即使注意力被拆分为多个 block，也可以在线维护 softmax 的正确归一化结果。对每个 query 行，只需维护三个量：

$$(m_i, l_i, \tilde{O}_i),$$

其中：

- m_i : 当前已经看到的 attention score 最大值;
- l_i : softmax 归一化因子;
- $\tilde{O}_i$: 未归一化的 value 加权和。

当处理新的 block 时, 首先计算该 block 内的局部最大值 m_i^{block} , 并更新全局最大值:

$$m_i^{new} = \max(m_i^{old}, m_i^{block}). \quad (11.7)$$

FlashAttention 的在线更新依赖指数函数的平移性质:

$$\exp(x - m^{new}) = \exp(x - m^{old}) \cdot \exp(m^{old} - m^{new}). \quad (11.8)$$

因此, 当 softmax 的归一化基准从 m^{old} 变化到 m^{new} 时, 旧 block 的所有指数项都只需乘同一个缩放因子即可完成坐标系转换。定义:

$$\alpha = \exp(m_i^{old} - m_i^{new}). \quad (11.9)$$

因此归一化项可以递推更新为:

$$l_i^{new} = l_i^{old} \alpha + \sum_{j \in block} \exp(S_{ij} - m_i^{new}). \quad (11.10)$$

类似地, 未归一化输出也满足相同的缩放关系:

$$\tilde{O}_i^{new} = \tilde{O}_i^{old} \alpha + \sum_{j \in block} \exp(S_{ij} - m_i^{new}) V_j. \quad (11.11)$$

当所有 block 处理完成后, 只需进行一次最终归一化即可得到 attention 输出:

$$O_i = \frac{\tilde{O}_i}{l_i}. \quad (11.12)$$

整个过程中, 算法无需显式存储完整的 $N \times N$ 注意力矩阵, 而是分块完成计算与累积, 从而显著降低 HBM 显存访问开销, 复杂度从 $O(N^2)$ 降低为 $O(N)$ 。

算法总结 FlashAttention 的在线计算过程总结如下:

- 将 K, V 按 block 划分;
- 对每个 block 计算局部 attention score;
- 更新全局最大值 m_i ;
- 对旧结果进行指数缩放修正;
- 累积归一化项与输出值;
- 最终进行一次归一化得到输出。

11.4 大模型部署

实际大模型部署通常依赖专用推理框架，以提升吞吐率、降低延迟并优化显存使用。典型工具包括 vLLM¹ 与 SGLang²。这些系统的核心目标不是改变模型结构，而是优化推理过程中的内存管理与执行调度。

11.4.1 vLLM

vLLM 是一个开源的大语言模型推理系统，由加州大学伯克利分校提出并开源实现，已成为工业界最广泛使用的 LLM serving 框架之一。

Paged Attention vLLM 的核心创新是 PagedAttention，其本质是对 KV Cache 的分页式管理。传统 Transformer 推理中，KV Cache 需要为每个请求分配连续显存： $K, V \in \mathbb{R}^{T \times d}$ 。这种方式的问题在于必须按最大上下文长度预分配显存。由于不同请求长度不同，导致显存浪费。尤其在多请求并发时容易产生显存碎片。

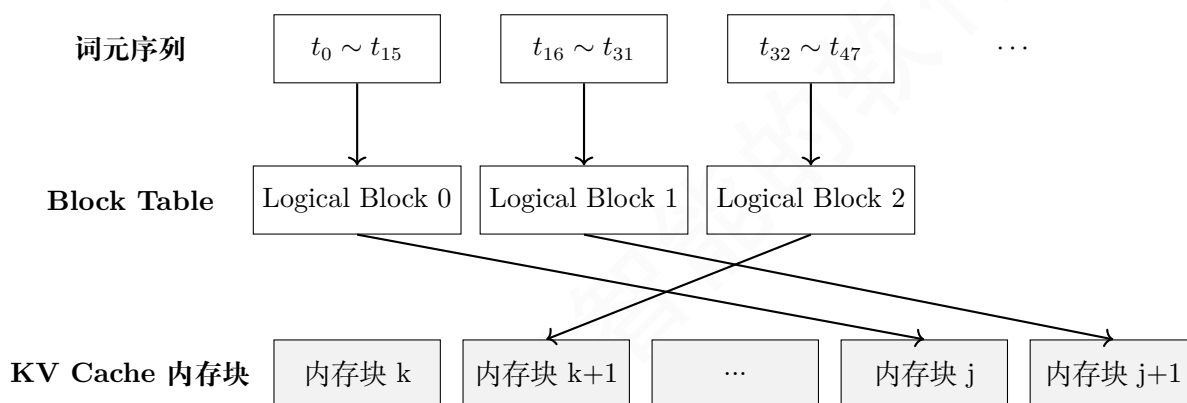


图 11.1: PagedAttention 中的分页式 KV Cache 管理

如图 11.1所示，PagedAttention 的关键思想是：

- 将 KV Cache 切分为固定大小的 block（例如 16 或 32 个词元）
- 每个请求的 KV 不再要求连续存储
- 使用 block table 维护“词元 $\rightarrow$ 内存块”的映射关系

因此，KV Cache 的访问从连续数组变为：词元逻辑索引 $\rightarrow$ block table $\rightarrow$ KV 块内存地址。这种设计使得显存分配更接近操作系统中的“分页内存管理”，从而减少碎片并提升利用率。

安装和使用 安装：

```
pip install vllm
```

启动服务（kaggle T4 环境下测试通过）：

```
python -m vllm.entrypoints.openai.api_server \
  --model Qwen/Qwen2.5-1.5B-Instruct \
```

¹<https://github.com/vllm-project/vllm>

²<https://github.com/sgl-project/sglang>

```
--host 0.0.0.0 \  
--port 8000 \  
--max-num-seqs 2 \  
--max-model-len 1024 \  
--attention-backend TRITON_ATTN \  
--gpu-memory-utilization 0.85 \  
--tensor-parallel-size 1
```

参数说明 vLLM 的关键运行参数包括：

- `--model`: 指定加载的模型名称或本地路径，例如 `Qwen/Qwen2.5-1.5B-Instruct`
- `--host`: 服务绑定地址。0.0.0.0 表示允许局域网或外部机器访问
- `--port`: HTTP API 服务监听端口，例如 8000
- `--max-num-seqs`: 单次调度中允许并发处理的最大请求数，会影响吞吐量与显存占用
- `--max-model-len`: 模型支持的最大上下文长度 (token 数)，该值越大，KV Cache 显存占用越高
- `--attention-backend`: 指定 Attention 计算后端，例如 `TRITON_ATTN` 使用 Triton 实现的高性能 Attention Kernel
- `--gpu-memory-utilization`: GPU 显存使用比例上限，例如 0.85 表示最多使用 85% 显存
- `--tensor-parallel-size`: Tensor Parallelism 使用的 GPU 数量。1 表示单 GPU 运行

启动以后，vLLM 默认会在本地 8000 端口启动一个兼容 OpenAI API 的 HTTP 服务：`http://localhost:8000/v1`

11.4.2 SGLang

SGLang 是由 LMSYS 提出的开源大语言模型推理与编排框架，其设计目标是提升复杂 LLM 应用（如多轮对话和工具调用）的执行效率，并在推理层面进一步优化 KV cache 复用与前缀共享能力。

Radix Attention SGLang 的核心优化机制是 Radix Attention，其本质是一种基于前缀树的 KV cache 复用机制，用于在多请求场景下共享重复的 prompt 前缀计算。

在传统 Transformer 推理中，KV cache 被用于在单个序列内部复用历史词元的 Key-Value 表示，从而避免在自回归生成过程中对历史上下文进行重复计算。具体而言，在同一请求的生成过程中，模型仅需对新增词元计算 attention，而历史词元的 KV 状态可以直接复用，从而显著降低计算复杂度。然而，在实际服务化部署中，多轮对话通常并不以“单一连续序列”的形式存在，而是采用 request-based 的执行范式：每一轮用户输入都会被重新组织为包含 system prompt、历史对话与当前输入的完整 prompt，并作为独立请求提交给推理引擎。因此，即使在同一会话内部，不同轮次之间仍然会产生重复的前缀计算。更进一步，在多用户或高并发场景中，这种前缀重复会跨请求广泛存在，例如 system prompt、模板化指令或 few-shot 示例在不同请求之间高度共享。由于标准 KV cache 的复用范围仅限于单个请求生命周期，因此无法跨请求共享这些重复前缀的 attention 计算。

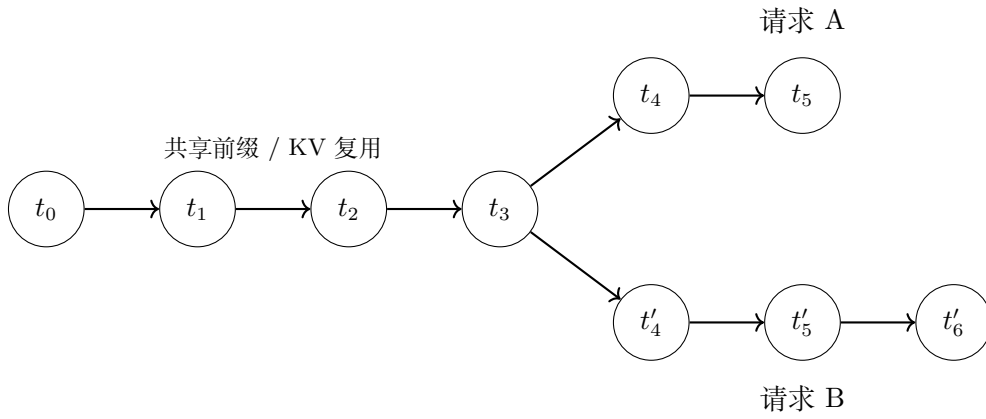


图 11.2: 基于 Radix Tree 的 KV Cache 复用

为了解决这一问题，Radix Attention 引入基于 radix tree 的前缀索引结构，对 token 序列进行结构化组织，从而实现跨请求的 KV cache 共享。如图 11.2 所示，对于请求集合 $\{t^{(i)}\}$ ，当其词元序列在 radix tree 上存在可复用的公共前缀时，即不同请求共享相同的前缀路径，则这些词元对应的 KV 值只需计算一次，并由 radix tree 中的共享节点进行复用，从而减少重复的 attention 计算开销。

为了更好地发挥 RadixAttention 的性能，SGLang 在系统设计上另外引入了多项关键机制，主要包括结构化生成与程序化 Prompt 执行等。

安装与使用 安装：

```
pip install sglang
```

启动 OpenAI 兼容服务：

```
python -m sglang.launch_server \
    --model-path XXX \
    --port 8000
```

参数说明 SGLang 在启动服务时提供了一系列关键配置参数，用于控制模型加载、并行策略以及运行时性能优化，常见参数如下：

- `--model-path`: 指定模型路径或 HuggingFace 模型名称（支持模型列表³）
- `--port`: 服务监听端口，用于提供 OpenAI-compatible HTTP API
- `--host`: 服务绑定地址，默认通常为 0.0.0.0
- `--tensor-parallel-size`: 张量并行 GPU 数量，用于分布式推理加速
- `--dtype`: 计算精度类型，如 fp16、bf16
- `--max-total-tokens`: 系统允许的最大词元总数，用于控制 KV cache 上限
- `--context-length`: 模型支持的最大上下文长度
- `--disable-radix-cache`: 关闭 RadixAttention cache

³https://sgl-project.github.io/supported_models/text_generation/generative_models.html

调用示例 vLLM 和 SGLang 提供的服务均兼容 OpenAI API 接口，因此可以直接使用 OpenAI 官方 Python SDK 进行调用。代码 11.1 展示了一个基于历史消息维护的多轮对话 Agent，实现与本地大模型服务的持续交互。

```
1 from openai import OpenAI
2
3 # vLLM 服务地址
4 client = OpenAI(
5     api_key="EMPTY", # vLLM 默认不校验
6     base_url="http://127.0.0.1:8000/v1",
7 )
8
9 # 对话历史
10 messages = [
11     {
12         "role": "system",
13         "content": "你是一个有帮助的AI助手"
14     }
15 ]
16
17 def chat_with_qwen(user_input: str):
18     # 加入用户消息
19     messages.append({
20         "role": "user",
21         "content": user_input
22     })
23     # 调用 vLLM
24     response = client.chat.completions.create(
25         model="Qwen/Qwen2.5-1.5B-Instruct",
26         messages=messages,
27         temperature=0.7,
28         max_tokens=512,
29     )
30     # 取回复
31     reply = response.choices[0].message.content
32     # 加入历史
33     messages.append({
34         "role": "assistant",
35         "content": reply
36     })
37     return reply
38
39 if __name__ == "__main__":
40     print("=== Local Qwen (vLLM) ===")
41     while True:
42         user_input = input("You: ")
43         if user_input.strip().lower() in {"exit", "quit"}:
44             break
45         reply = chat_with_qwen(user_input)
46         print("Qwen:", reply)
```

代码 11.1: 基于 OpenAI SDK 的本地 vLLM/SGLang 多轮对话 Agent 示例

练习

- 1) 完成一个端到端的大模型推理系统实验，包括模型部署、API 调用与性能分析三个步骤。
 - (a) **模型部署**：使用 vLLM 或 SGLang 在本地部署一个大语言模型服务（Kaggle 推荐使用 Qwen2.5-1.5B-Instruct 模型）。在 Kaggle 环境中进行实验时，建议将服务进程放到后台运行，可先将启动命令写入 shell 脚本，再通过脚本启动服务。
 - (b) **API 调用测试**：使用 Python OpenAI SDK 或 curl 调用本地服务，完成至少一次多轮对话请求，验证服务正常运行。
 - (c) **性能分析**：调整相关优化配置，观察并分析模型的显存占用和响应延迟变化。

12 智能体

本章学习目标：

- ** 理解典型智能体的软件结构和基本原理
- *** 能够阅读智能体代码逻辑并定制智能体

12.1 智能体的软件结构

智能体（Agent）是近年来大语言模型应用中的一个重要概念。传统聊天机器人通常采用“用户输入—模型回答”的模式，只能根据已有知识生成文本。而智能体不仅能够进行对话，还能够主动规划任务、调用外部工具、访问记忆，并根据环境反馈持续调整自己的行为。图12.1展示了一个典型智能体的软件结构。

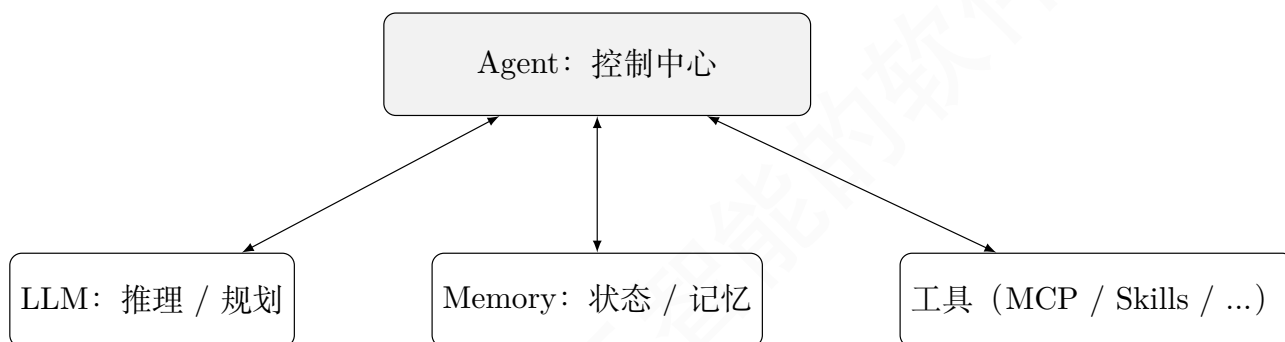


图 12.1: 典型智能体软件栈

- **Agent（控制中心）**：Agent 负责整个系统的调度与控制。它接收用户输入，组织上下文信息，调用大语言模型进行推理，并根据模型输出决定是否需要调用工具。在工具执行结束后，Agent 还需要收集执行结果，并组织最终回复返回给用户。
- **LLM（推理 / 规划）**：LLM 是智能体的核心决策模块，负责理解用户意图、分析当前状态并生成行动计划。例如，当用户要求查询天气、搜索网页或者执行代码时，LLM 需要判断应当调用哪些工具，并决定工具调用顺序。在现代 Agent 系统中，任务规划、任务分解以及推理通常都由 LLM 完成。
- **Memory（状态 / 记忆）**：Memory 用于保存智能体运行过程中的状态信息。最简单的记忆形式是对话历史，即保存用户和系统之前的交互内容。记忆模块使智能体能够在多轮交互中保持上下文一致性。
- **Tools（工具）**：工具模块赋予智能体与外部环境交互的能力。常见工具包括搜索引擎、各种 Web API 以及代码执行器等。通过工具调用，智能体能够执行查询、计算、文件操作等实际任务，而不仅仅是生成文本。近年来出现了多种工具扩展机制。例如，MCP（Model Context Protocol）提供了标准化的工具接入协议；Skill 则为智能体提供额外知识和操作规范，例如软件操作手册或知识库等。二者共同扩展了智能体的能力边界。

12.2 案例：CLI 智能体

本节通过一个最小可运行的 CLI 智能体示例，展示智能体系统的基本工作方式。命令行环境（Linux Shell）的一个典型问题是用户需要记忆大量命令及其参数，这在实际使用中具有较高的学习成本。因此，该智能体的目标是：当用户在命令行中输入自然语言指令时，系统能够自动判断是否需要调用系统命令；在需要时生成并执行对应的 Shell 命令；并将执行结果返回给用户。同时，系统还会对执行结果进行进一步分析与解释，从而降低用户理解命令输出的难度，提升命令行使用的易用性与智能化水平。

智能体的实现如代码12.1所示。

```
1 from openai import OpenAI
2 import os
3 import subprocess
4 import json
5
6 API_KEY = "sk-YOUKEY" #替换为你自己的API_KEY。
7 client = OpenAI(
8     api_key=API_KEY,
9     base_url="https://dashscope.aliyuncs.com/compatible-mode/v1"
10 )
11 messages = [
12     {
13         "role": "system",
14         "content": ""
15         你是一个CLI智能体。
16         如果用户请求涉及系统操作（如ls, dir, cat, pwd等），请返回JSON格式：
17         {
18             "action": "run_shell",
19             "command": "实际要执行的命令"
20         }
21         否则正常回答。
22         ""
23     }
24 ]
25
26 def run_shell(cmd: str) -> str:
27     """执行shell命令"""
28     try:
29         result = subprocess.run(
30             cmd,
31             shell=True,
32             capture_output=True,
33             text=True
34         )
35         return result.stdout + result.stderr
36     except Exception as e:
37         return str(e)
38
39 def chat(user_input: str):
40     messages.append({"role": "user", "content": user_input})
41     response = client.chat.completions.create(
42         model="qwen-plus",
```

```

43     messages=messages,
44     temperature=0.2,         max_tokens=512,
45 )
46 content = response.choices[0].message.content.strip()
47 # 尝试解析是否是工具调用
48 try:
49     data = json.loads(content)
50     if isinstance(data, dict) and data.get("action") == "run_shell":
51         cmd = data["command"]
52         print(f"\n[EXECUTING] {cmd}\n")
53         output = run_shell(cmd)
54         # 把执行结果回传给模型
55         messages.append({"role": "assistant", "content": content})
56         messages.append({
57             "role": "user",
58             "content": f"命令执行结果: \n{output}\n请总结结果。"
59         })
60
61         final = client.chat.completions.create(
62             model="qwen-plus",
63             messages=messages,
64             temperature=0.2,
65             max_tokens=512,
66         )
67         reply = final.choices[0].message.content
68     else:
69         reply = content
70
71 except Exception:
72     # 不是JSON就当普通回答
73     reply = content
74
75 messages.append({"role": "assistant", "content": reply})
76 return reply
77
78 def main():
79     print("=== CLI Agent with Execution ===")
80     while True:
81         user_input = input("\nYou: ")
82         if user_input in ["exit", "quit"]:
83             break
84         reply = chat(user_input)
85         print("\nAgent:", reply)
86
87 if __name__ == "__main__":
88     main()

```

代码 12.1: Linux shell 智能体

练习

- 1) 运行代码 12.1所示的 CLI 智能体程序，观察其在不同指令下的执行效果，并分析其优点与局限性。如果你的开发环境不支持 Linux 系统，可以在大模型辅助下将其改写为 Windows Shell 智能体，并测试其在 Windows 环境下的可用性。